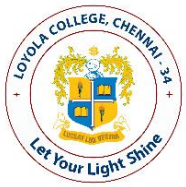
LOYOLA COLLEGE

Chennai – 600 034



Department of Data Science

PRACTICAL RECORD

Course Title : Machine Learning Lab

Dept. No. : 26-PDS-014

Name : MARIYA JOHNET R

Semester : I

Year : 2026-2027

Loyola College, Chennai – 600 034

Department of Data Science

BONAFIDE CERTIFICATE

Dept. No. : 26-PDS-014

Name : MARIYA JOHNET R

Class : 1 M.Sc Data Science

This is to certify that this is the bonafide record of the practical work done in Machine Learning Lab at the Loyola College Data Center, during the year 2026-2027.

Head of the Department **Staff-in-Charge**

Date : 14.10.2026

Submitted for the Machine Learning Lab Data Science Practical examination held on 14.10.2026 at Loyola College, Chennai – 600 034.

Internal Examiner **External Examiner**

CONTENTS

Sl. No.	Title	Date	Page No.
1	Exploratory Data Analysis with Titanic Dataset	01-07-2026	4-10
2	Simple Linear Regression	02-07-2026	11-14
3	Multiple Linear Regression	07-07-2026	15-20
4	Polynomial Regression	10-07-2026	21-25
5	Regularization Techniques	17-07-2026	26-32
6	Logistic Regression	05-08-2026	33-39
7	K-Nearest Neighbour	20-08-2026	40-47
8	Decision Tree Classification	21-08-2026	48-55
9	Random Forest Classification	25-08-2026	56-63
10	Support Vector Machine	28-08-2026	64-69
11	Hyperparameter Tuning (GridSearchCV)	02-09-2026	70-72
12	K-Means Clustering	07-09-2026	73-77
13	Hierarchical Clustering	15-09-2026	78-80
14	DBSCAN Clustering	18-09-2026	81-84
15	Principal Component Analysis	22-09-2026	85-88

Exercise – 01

Exploratory Data Analysis with Titanic Dataset

1. AIM

To perform exploratory data analysis on the Titanic dataset by examining its structure, missing values, statistical summaries, distributions, relationships and visual patterns.

2. ALGORITHM

1. Load the Titanic dataset.

2. Inspect rows, dimensions, data types and missing values.

3. Generate statistical summaries.

4. Perform the cleaning and transformations in the notebook.

5. Create the exploratory visualizations.

6. Interpret the observed patterns.

3. PROGRAM

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

data = 'Dataset/titanic.csv'

df = pd.read_csv(data)

df.head()

df.tail()

df.sample(5)

print("Rows and Columns:", df.shape)

df.info()

df.describe()

df.size

df.columns

df.isnull().sum()

missing_percentage = (df.isnull().sum() / len(df)) * 100

print(missing_percentage.sort_values(ascending=False))
```

```
plt.figure(figsize=(10, 6))

sns.heatmap(
    df.isnull(),
    cbar=False,
    yticklabels=False,
    cmap="viridis"
)

plt.title("Missing Values Heatmap")
plt.show()

# Fill Age missing values
df["Age"] = df["Age"].fillna(df["Age"].median())

# Fill Embarked missing values
df["Embarked"] = df["Embarked"].fillna(df["Embarked"].mode()[0])

# Fill Cabin missing values
df["Cabin"] = df["Cabin"].fillna("Unknown")

print(df.isnull().sum())

print("Number of duplicate rows:", df.duplicated().sum())

df = df.drop_duplicates()

print("Shape after removing duplicates:", df.shape)

plt.figure(figsize=(6, 4))

sns.countplot(data=df, x="Survived")

plt.title("Passenger Survival Count")
plt.xlabel("Survived")
plt.ylabel("Number of Passengers")

plt.show()

survival_counts = df["Survived"].value_counts()

plt.figure(figsize=(6, 6))

plt.pie(
    survival_counts,
    labels=["Did Not Survive", "Survived"],
    autopct="%1.1f%%",
    startangle=90
)

plt.title("Survival Distribution")

plt.show()

plt.figure(figsize=(6, 4))
```

```
sns.countplot(data=df, x="Pclass")

plt.title("Passenger Class Distribution")
plt.xlabel("Passenger Class")
plt.ylabel("Number of Passengers")

plt.show()

plt.figure(figsize=(6, 4))

sns.countplot(data=df, x="Sex")

plt.title("Gender Distribution")
plt.xlabel("Gender")
plt.ylabel("Number of Passengers")

plt.show()

plt.figure(figsize=(7, 5))

sns.countplot(
    data=df,
    x="Sex",
    hue="Survived"
)

plt.title("Survival by Gender")
plt.xlabel("Gender")
plt.ylabel("Number of Passengers")

plt.show()

plt.figure(figsize=(7, 5))

sns.countplot(
    data=df,
    x="Pclass",
    hue="Survived"
)

plt.title("Survival by Passenger Class")
plt.xlabel("Passenger Class")
plt.ylabel("Number of Passengers")

plt.show()

plt.figure(figsize=(8, 5))

sns.boxplot(
    data=df,
    x="Survived",
    y="Fare"
)

plt.title("Fare Distribution by Survival")
plt.xlabel("Survived")
```

```
plt.ylabel("Fare")

plt.show()

g = sns.catplot(
    data=df,
    x="Pclass",
    hue="Survived",
    col="Sex",
    kind="count",
    height=5,
    aspect=1
)

g.fig.suptitle(
    "Survival by Passenger Class and Gender",
    y=1.05
)

plt.show()

plt.figure(figsize=(9, 6))

sns.scatterplot(
    data=df,
    x="Age",
    y="Fare",
    hue="Survived",
    style="Sex",
    size="Pclass",
    sizes=(50, 200)
)

plt.title("Age vs Fare by Survival, Gender and Class")
plt.xlabel("Age")
plt.ylabel("Fare")

plt.show()

numeric_columns = [
    "Survived",
    "Pclass",
    "Age",
    "SibSp",
    "Parch",
    "Fare"
]

correlation = df[numeric_columns].corr()

print(correlation)

plt.figure(figsize=(9, 7))

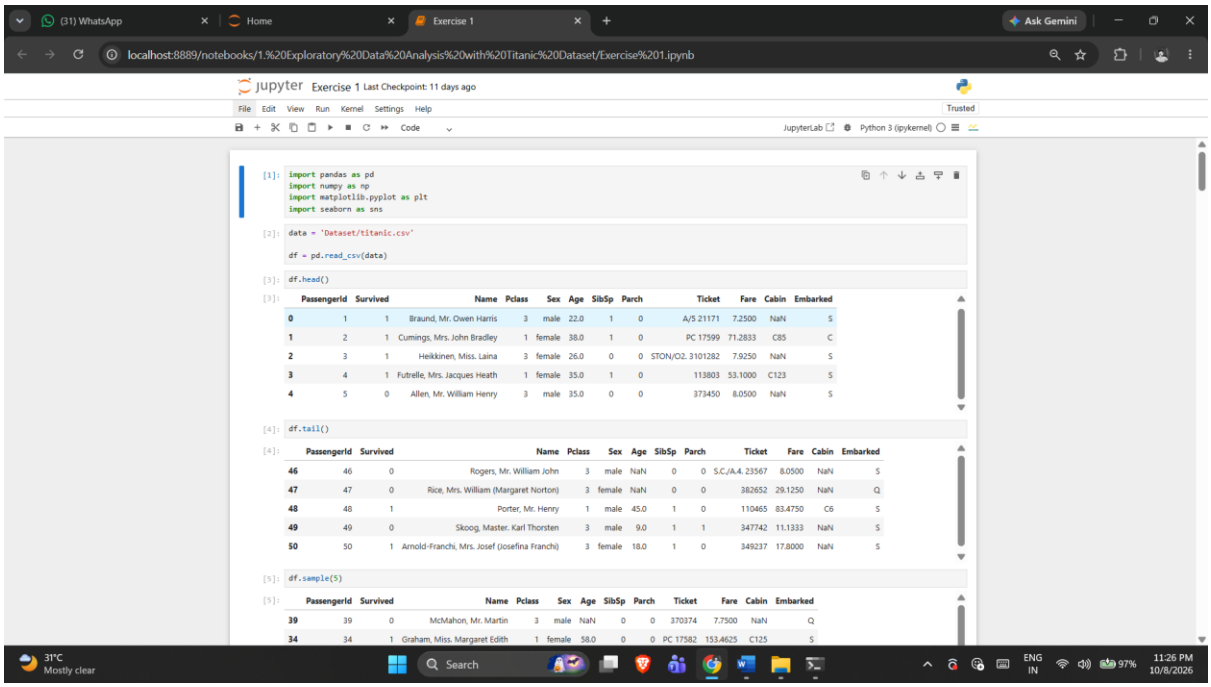
sns.heatmap(
    correlation,
    annot=True,
    cmap="coolwarm",
    fmt=".2f",
```

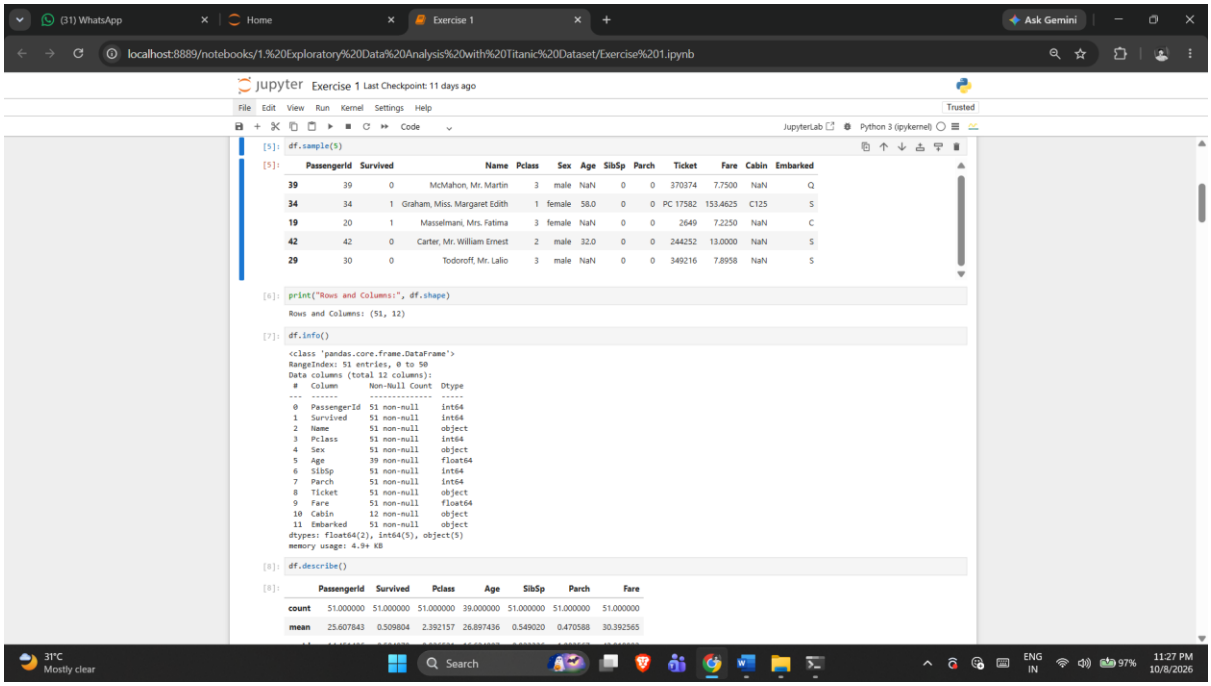
```
    linewidths=0.5
)

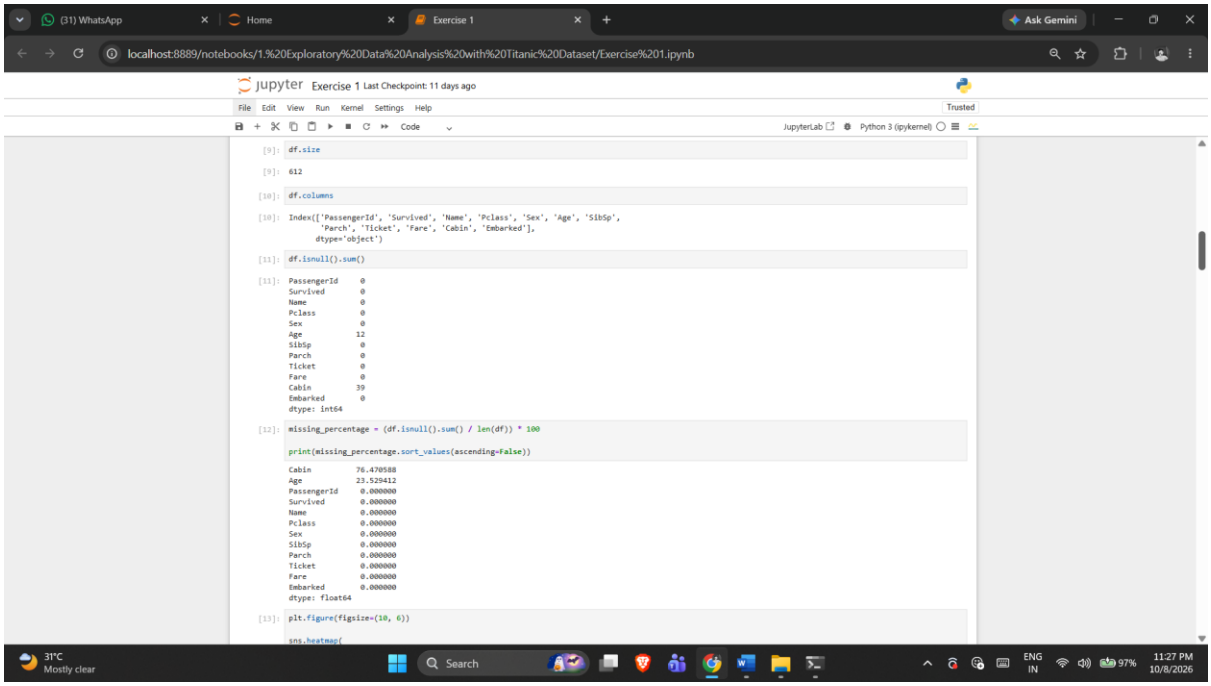
plt.title("Correlation Heatmap")

plt.show()
```

4. OUTPUT







```
[9]: df.size

[9]: 612

[10]: df.columns

[10]: Index(['PassengerId', 'Survived', 'Name', 'Pclass', 'Sex', 'Age', 'SibSp',
       'Parch', 'Ticket', 'Fare', 'Cabin', 'Embarked'],
      dtype='object')

[11]: df.isnull().sum()

[11]: PassengerId     0
      Survived        0
      Name            0
      Pclass          0
      Sex             0
      Age            12
      SibSp           0
      Parch           0
      Ticket          0
      Fare            0
      Cabin          39
      Embarked        0
      dtype: int64

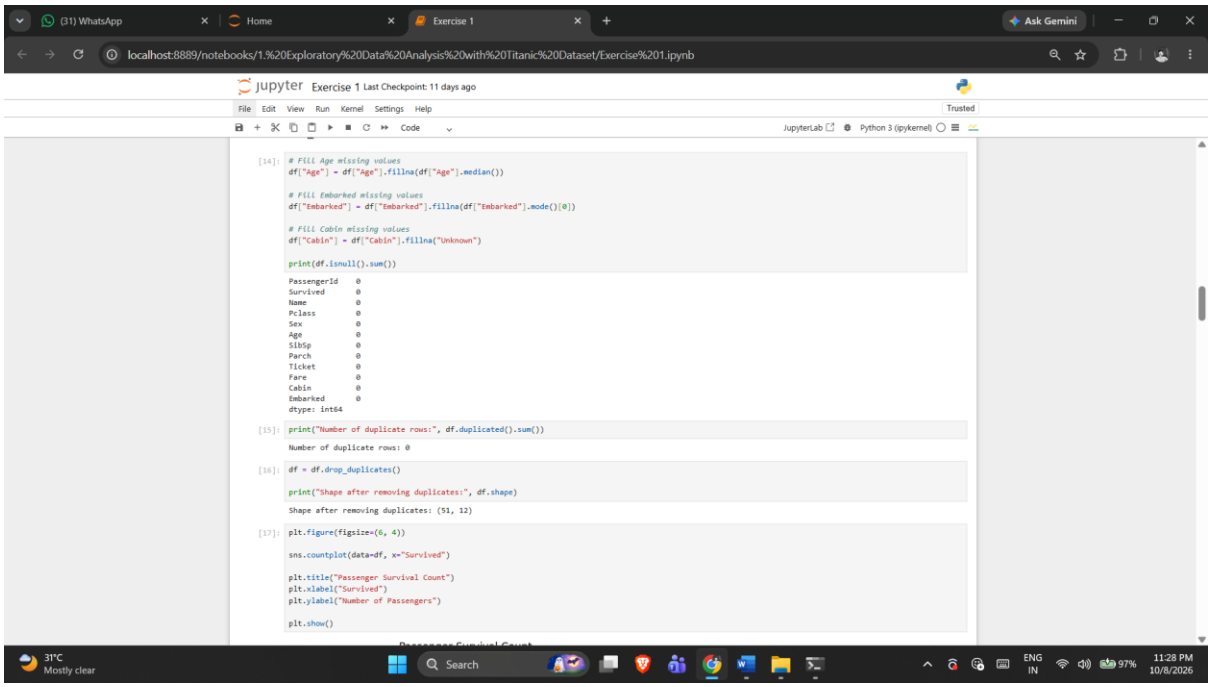
[12]: missing_percentage = (df.isnull().sum() / len(df)) * 100

      print(missing_percentage.sort_values(ascending=False))

      Cabin          76.470588
      Age            23.529412
      PassengerId     0.000000
      Survived        0.000000
      Name            0.000000
      Pclass          0.000000
      Sex             0.000000
      SibSp           0.000000
      Parch           0.000000
      Ticket          0.000000
      Fare            0.000000
      Embarked        0.000000
      dtype: float64

[13]: plt.figure(figsize=(10, 6))

      sns.heatmap(
```



```
[14]: # Fill Age missing values
      df["Age"] = df["Age"].fillna(df["Age"].median())

      # Fill Embarked missing values
      df["Embarked"] = df["Embarked"].fillna(df["Embarked"].mode()[0])

      # Fill Cabin missing values
      df["Cabin"] = df["Cabin"].fillna("Unknown")

      print(df.isnull().sum())

      PassengerId    0
      Survived       0
      Name           0
      Pclass         0
      Sex            0
      Age            0
      SibSp          0
      Parch          0
      Ticket         0
      Fare           0
      Cabin          0
      Embarked       0
      dtype: int64

[15]: print("Number of duplicate rows:", df.duplicated().sum())

      Number of duplicate rows: 0

[16]: df = df.drop_duplicates()

      print("Shape after removing duplicates:", df.shape)

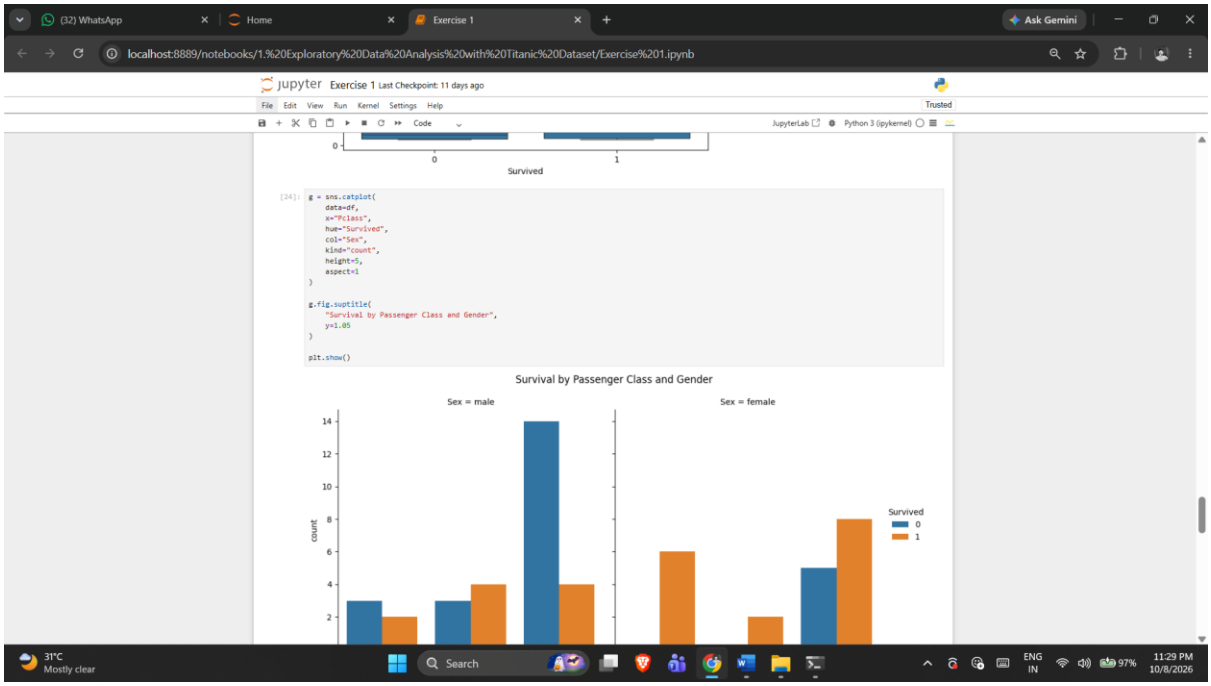
      Shape after removing duplicates: (51, 12)

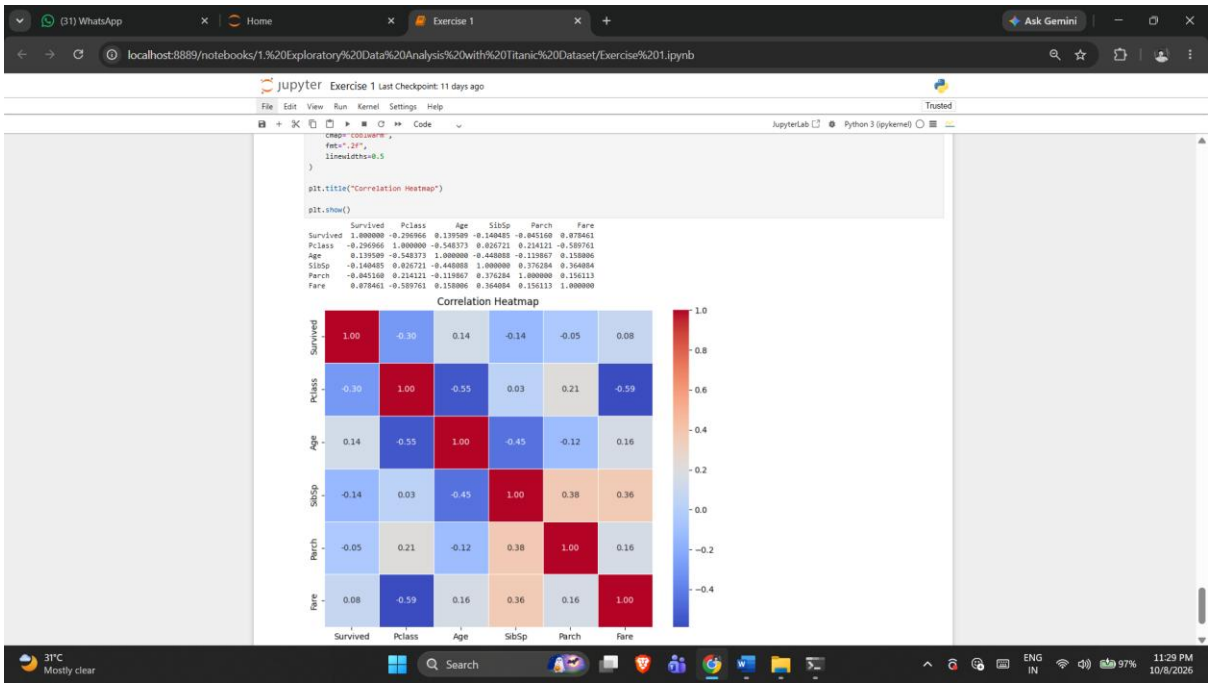
[17]: plt.figure(figsize=(6, 4))

      sns.countplot(data=df, x="Survived")

      plt.title("Passenger Survival Count")
      plt.xlabel("Survived")
      plt.ylabel("Number of Passengers")

      plt.show()
```





5. RESULT

Thus, the Exploratory Data Analysis with Titanic Dataset practical was implemented successfully using the program from the corresponding Jupyter Notebook.

Exercise -2

Simple Linear Regression

1. AIM

To implement and demonstrate Simple Linear Regression using the dataset, preprocessing, model or analysis workflow provided in the corresponding Jupyter Notebook.

2. ALGORITHM

1. Load the regression data.

2. Select the independent and dependent variables.

3. Split the data into training and testing sets.

4. Train Linear Regression.

5. Generate predictions.

6. Calculate the notebook's evaluation metrics.

7. Plot the fitted regression relationship.

3. PROGRAM

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

data = "Dataset/Salary.csv"
df = pd.read_csv(data)
df = df[['YearsExperience','Salary']]

df

df.size

df.shape

df.isnull().sum()

X = df[['YearsExperience']]   # Independent Variable
y = df['Salary']     # Dependent Variable

model = LinearRegression()

model.fit(x1,y1)

x1,x2,y1,y2 = train_test_split(X,y,random_state= 42,test_size=0.2)
```

```
y_pred = model.predict(x2)

print("Actual Values:")
print(y2.values)

print("Predicted Values:")
print(y_pred)

print("Coefficient (Slope):", model.coef_[0])
print("Intercept:", model.intercept_)
print("Mean Squared Error:", mean_squared_error(y_test, y_pred))
print("R2 Score:", r2_score(y_test, y_pred))

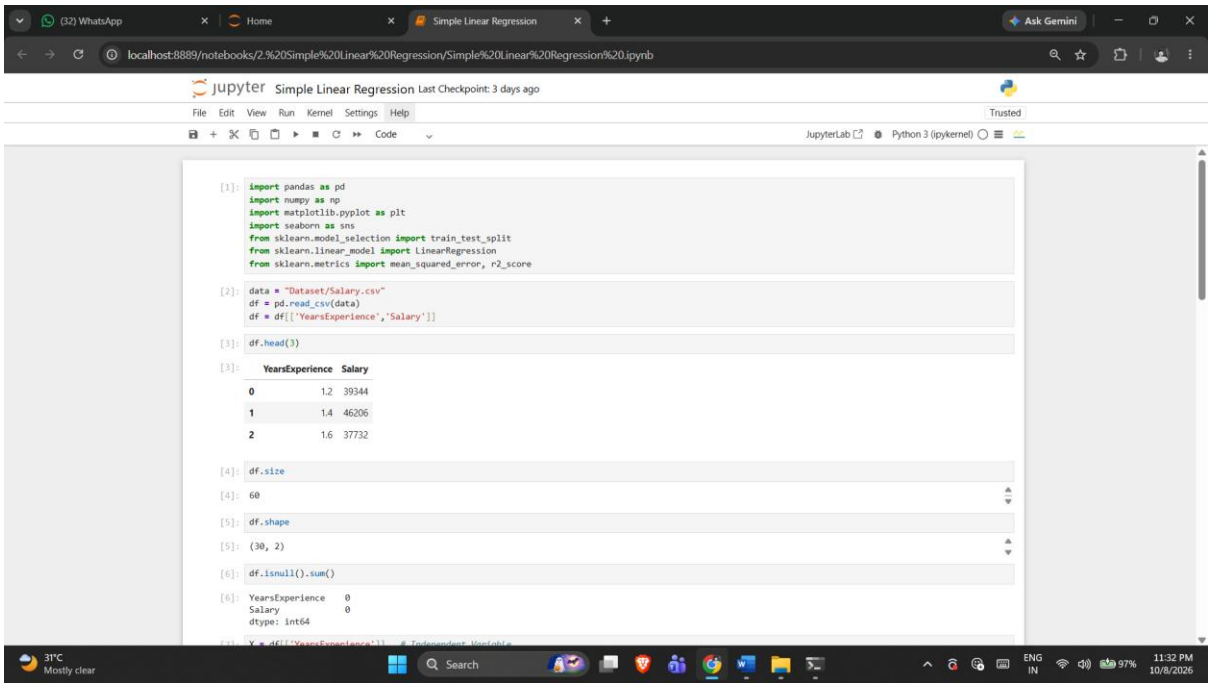
YearsExperience = [[2.1]]
predicted_salary = model.predict(YearsExperience)

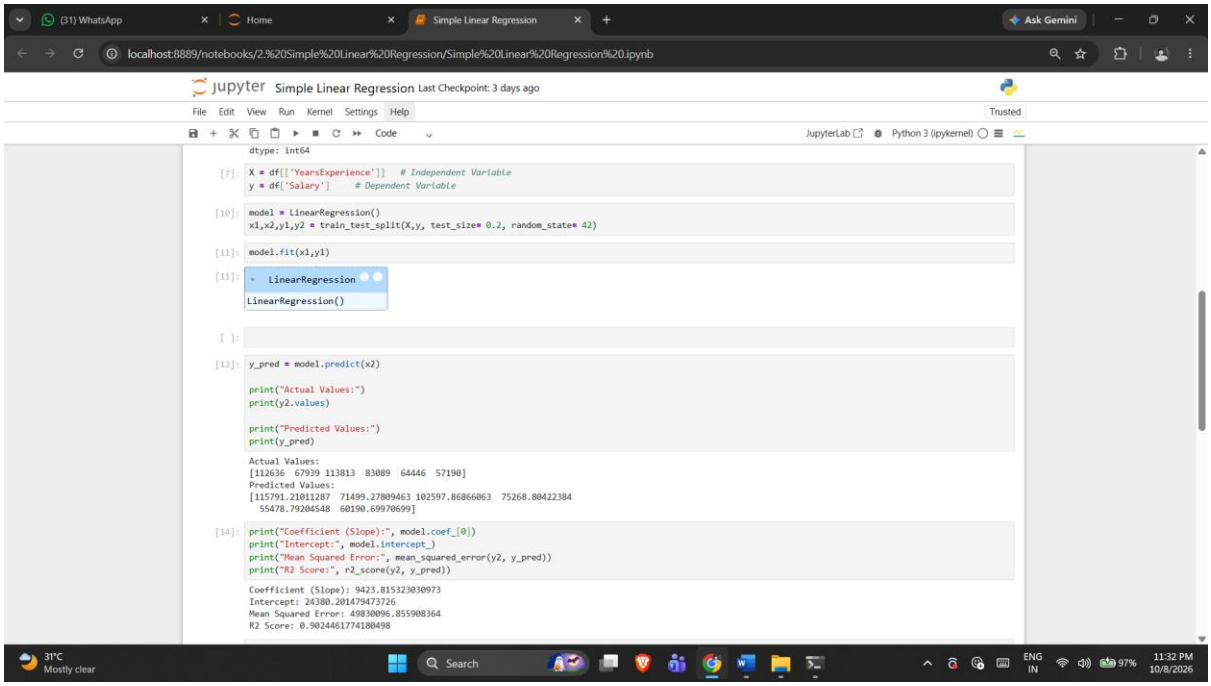
print("Predicted salary for 2.1 years:", predicted_salary[0])

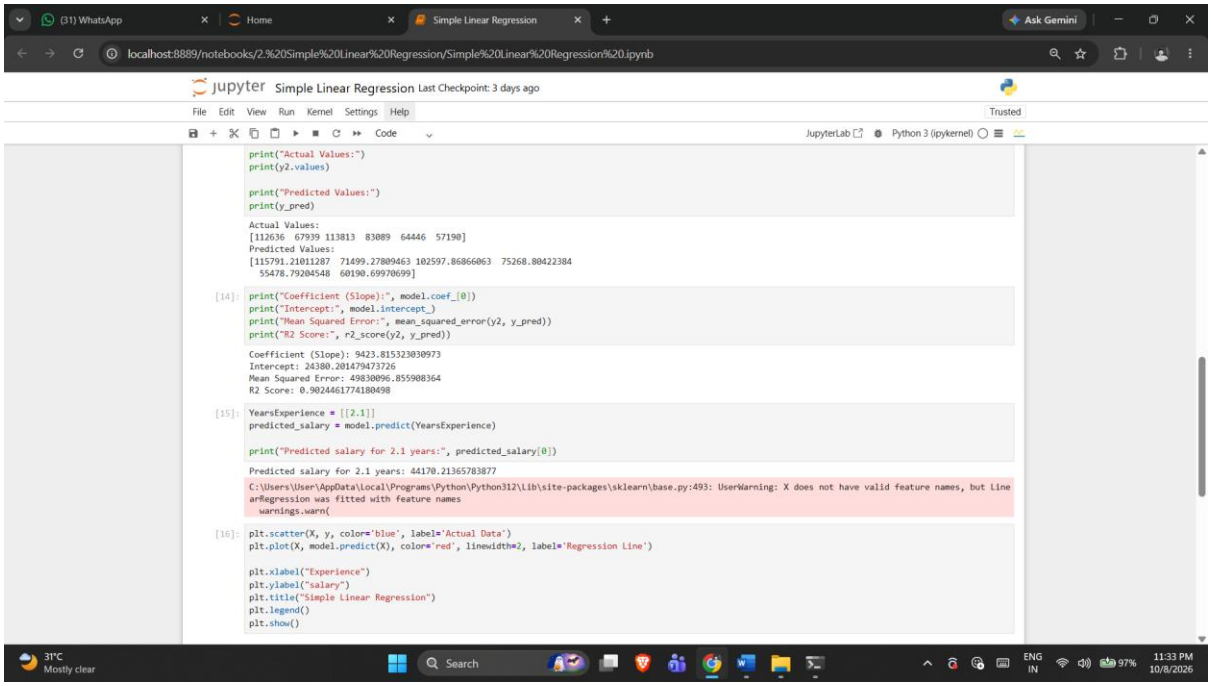
plt.scatter(X, y, color='blue', label='Actual Data')
plt.plot(X, model.predict(X), color='red', linewidth=2, label='Regression Line')

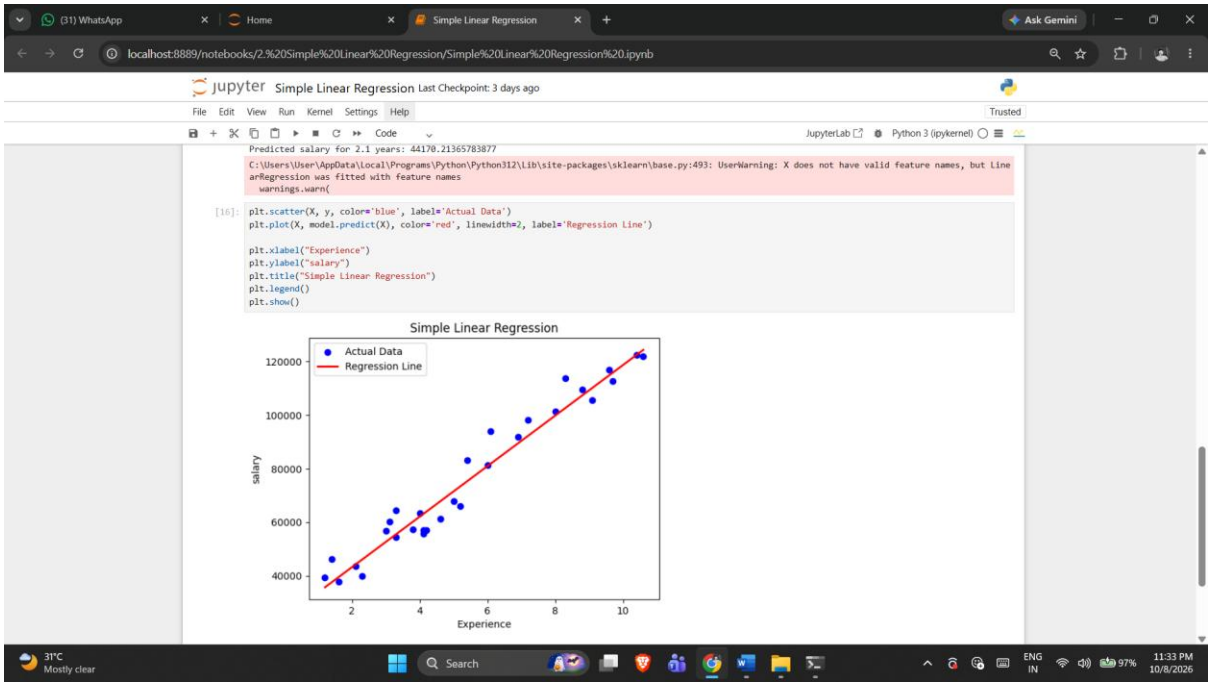
plt.xlabel("Experience")
plt.ylabel("salary")
plt.title("Simple Linear Regression")
plt.legend()
plt.show()
```

4. OUTPUT




(31) WhatsApp
Home
Simple Linear Regression
Ask Gemini
localhost:8889/notebooks/2.%20Simple%20Linear%20Regression/Simple%20Linear%20Regression%20.ipynb
jupyter Simple Linear Regression Last Checkpoint: 3 days ago
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
dtype: int64
[7]: X = df[['YearsExperience']] # Independent Variable
y = df['Salary'] # Dependent Variable
[10]: model = LinearRegression()
x1,x2,y1,y2 = train_test_split(X,y, test_size= 0.2, random_state= 42)
[11]: model.fit(x1,y1)
[11]: LinearRegression
LinearRegression()
[ ]:
[12]: y_pred = model.predict(x2)
print("Actual Values:")
print(y2.values)
print("Predicted Values:")
print(y_pred)
Actual Values:
[112636 67939 113813 83089 64446 57190]
Predicted Values:
[115791.21011287 71499.27809463 102597.86866063 75268.80422384
55478.79204548 60190.69970699]
[14]: print("Coefficient (Slope):", model.coef_[0])
print("Intercept:", model.intercept_)
print("Mean Squared Error:", mean_squared_error(y2, y_pred))
print("R2 Score:", r2_score(y2, y_pred))
Coefficient (Slope): 9423.815323030973
Intercept: 24380.201479473726
Mean Squared Error: 49830096.855908364
R2 Score: 0.9024461774180498
31°C
Mostly clear
Search
ENG
IN
97%
11:32 PM
10/8/2026


(31) WhatsApp
Home
Simple Linear Regression
Ask Gemini
localhost:8889/notebooks/2.%20Simple%20Linear%20Regression/Simple%20Linear%20Regression%20.ipynb
jupyter Simple Linear Regression Last Checkpoint: 3 days ago
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
print("Actual Values:")
print(y2.values)
print("Predicted Values:")
print(y_pred)
Actual Values:
[112636 67939 113813 83089 64446 57190]
Predicted Values:
[115791.21011287 71499.27809463 102597.86866063 75268.80422384
55478.79204548 60190.69970699]
[14]: print("Coefficient (Slope):", model.coef_[0])
print("Intercept:", model.intercept_)
print("Mean Squared Error:", mean_squared_error(y2, y_pred))
print("R2 Score:", r2_score(y2, y_pred))
Coefficient (Slope): 9423.815323030973
Intercept: 24380.201479473726
Mean Squared Error: 49830096.855908364
R2 Score: 0.9024461774180498
[15]: YearsExperience = [[2.1]]
predicted_salary = model.predict(YearsExperience)
print("Predicted salary for 2.1 years:", predicted_salary[0])
Predicted salary for 2.1 years: 44170.21365783877
C:\Users\User\AppData\Local\Programs\Python\Python312\Lib\site-packages\sklearn\base.py:493: UserWarning: X does not have valid feature names, but LinearRegression was fitted with feature names
warnings.warn(
[16]: plt.scatter(X, y, color='blue', label='Actual Data')
plt.plot(X, model.predict(X), color='red', linewidth=2, label='Regression Line')
plt.xlabel("Experience")
plt.ylabel("salary")
plt.title("Simple Linear Regression")
plt.legend()
plt.show()
31°C
Mostly clear
Search
ENG
IN
97%
11:33 PM
10/8/2026



5. RESULT

Thus, the Simple Linear Regression practical was implemented successfully using the program from the corresponding Jupyter Notebook.

Exercise -3

Multiple Linear Regression

1. AIM

To implement and demonstrate Multiple Linear Regression using the dataset, preprocessing, model or analysis workflow provided in the corresponding Jupyter Notebook.

2. ALGORITHM

1. Load the regression dataset.

2. Select the target and multiple predictor variables.

3. Split the data into training and testing sets.

4. Train the multiple linear regression model.

5. Generate predictions.

6. Calculate the regression metrics implemented in the notebook.

7. Display the notebook's visual analysis.

3. PROGRAM

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.linear_model import LinearRegression
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score,
    mean_absolute_percentage_error,
    explained_variance_score,
    max_error
)

data = 'Dataset/Housing.csv'

df= pd.read_csv(data)

df.head()

df.info()

le = LabelEncoder()
for col in df.select_dtypes(include="object").columns:
    df[col] = le.fit_transform(df[col])

X = df.drop("price", axis=1)
y = df["price"]
```

```
x1, x2, y1, y2 = train_test_split(X, y,test_size=0.2,random_state=42)

model = LinearRegression()
model.fit(x1, y1)

y_pred = model.predict(x2)
y_pred

mae = mean_absolute_error(y2, y_pred)
mse = mean_squared_error(y2, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y2, y_pred)
mape = mean_absolute_percentage_error(y2, y_pred)
evs = explained_variance_score(y2, y_pred)
maxerr = max_error(y2, y_pred)

n = x2.shape[0]      # Number of observations
p = x2.shape[1]      # Number of predictors

adj_r2 = 1 - ((1-r2)*(n-1)/(n-p-1))

print("\n========== Regression Evaluation ==========")
print(f"Mean Absolute Error (MAE): {mae:.2f}")
print(f"Mean Squared Error (MSE): {mse:.2f}")
print(f"Root Mean Squared Error (RMSE): {rmse:.2f}")
print(f"R² Score: {r2:.4f}")
print(f"Adjusted R² Score: {adj_r2:.4f}")
print(f"Mean Absolute Percentage Error (MAPE): {mape:.4f}")
print(f"Explained Variance Score: {evs:.4f}")
print(f"Maximum Error: {maxerr:.2f}")

print("\nIntercept:")
print(model.intercept_)



# In[21]:



print("\nCoefficients:")
coef = pd.DataFrame({
    "Feature": X.columns,
    "Coefficient": model.coef_
})
print(coef)

new_house = [[5000,3,2,2,1,1,0,1,0,1,0,1]]

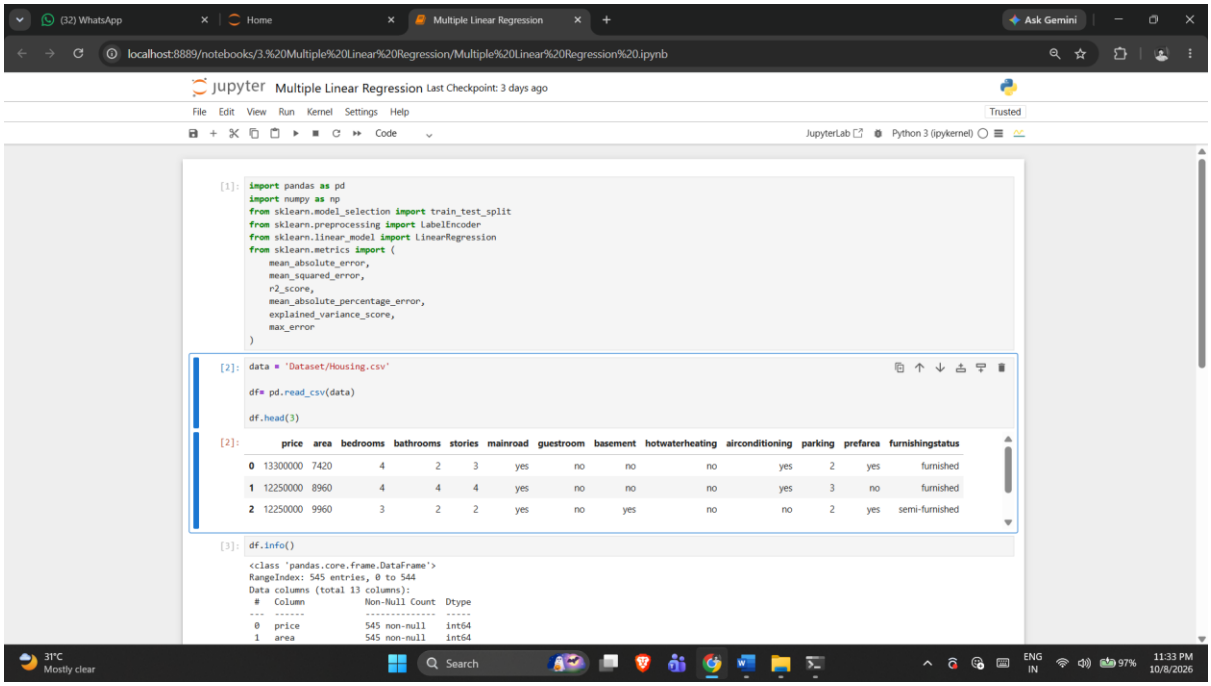
prediction = model.predict(new_house)

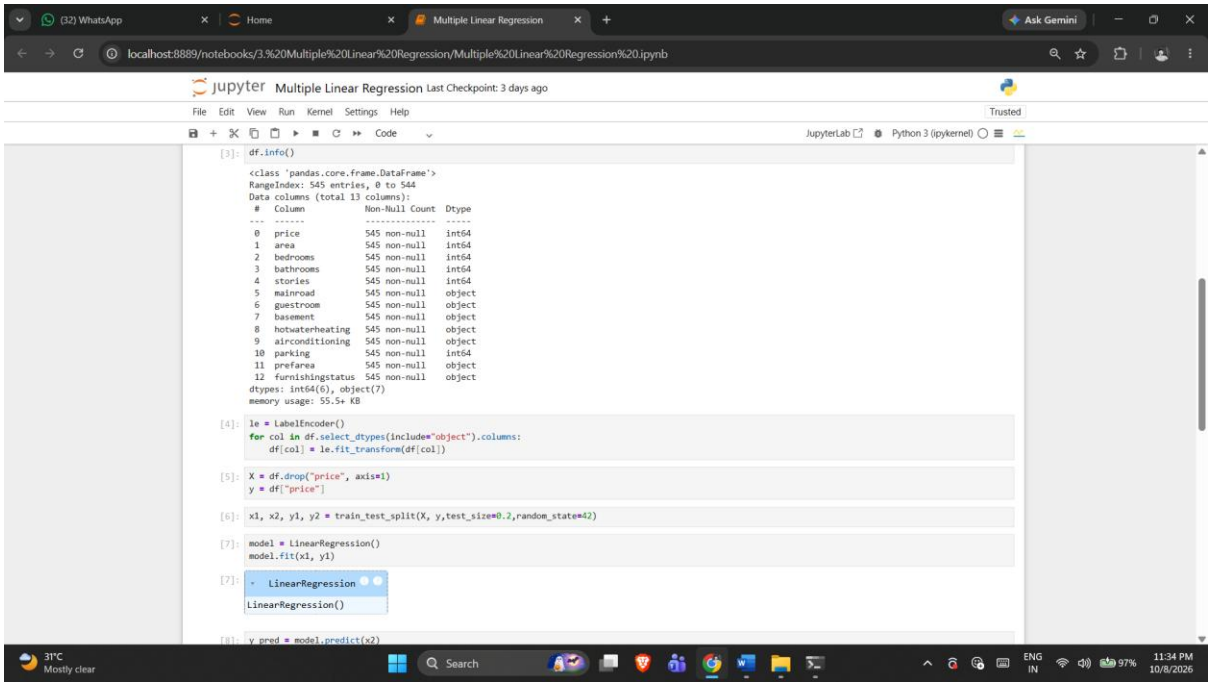
print("\nPredicted House Price:", prediction[0])

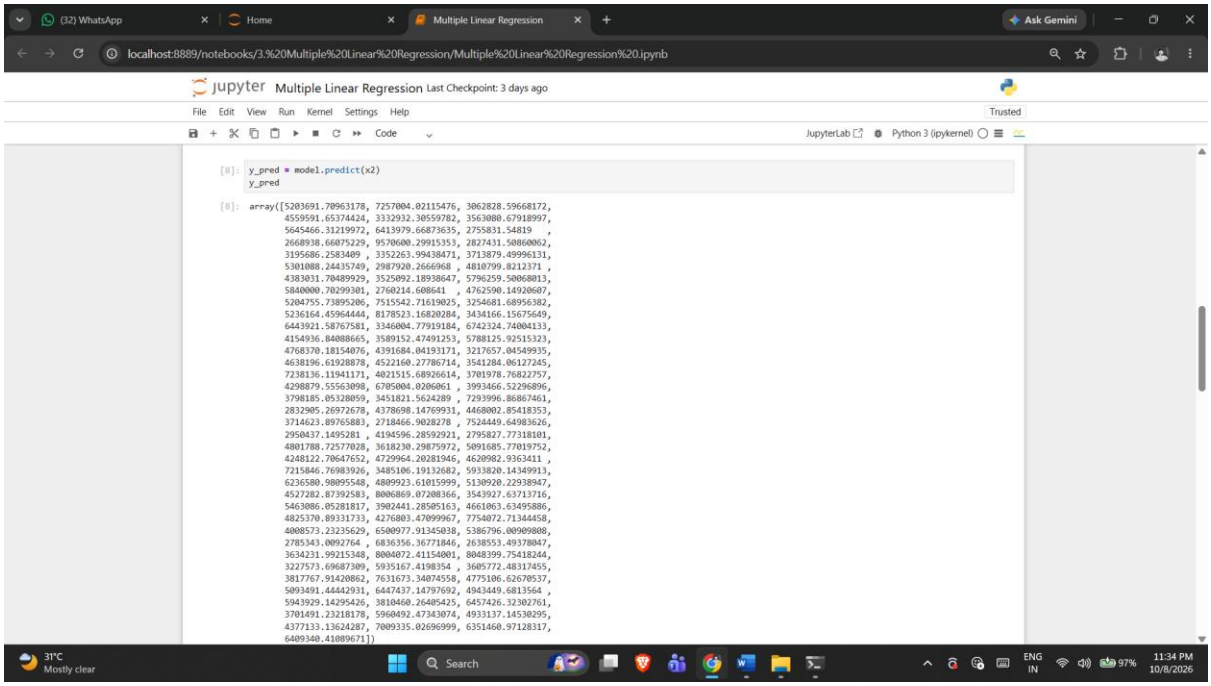
import matplotlib.pyplot as plt
plt.figure(figsize=(6,6))
plt.scatter(y2, y_pred)
plt.plot([y2.min(), y2.max()],
         [y2.min(), y2.max()],
         'r--')
```

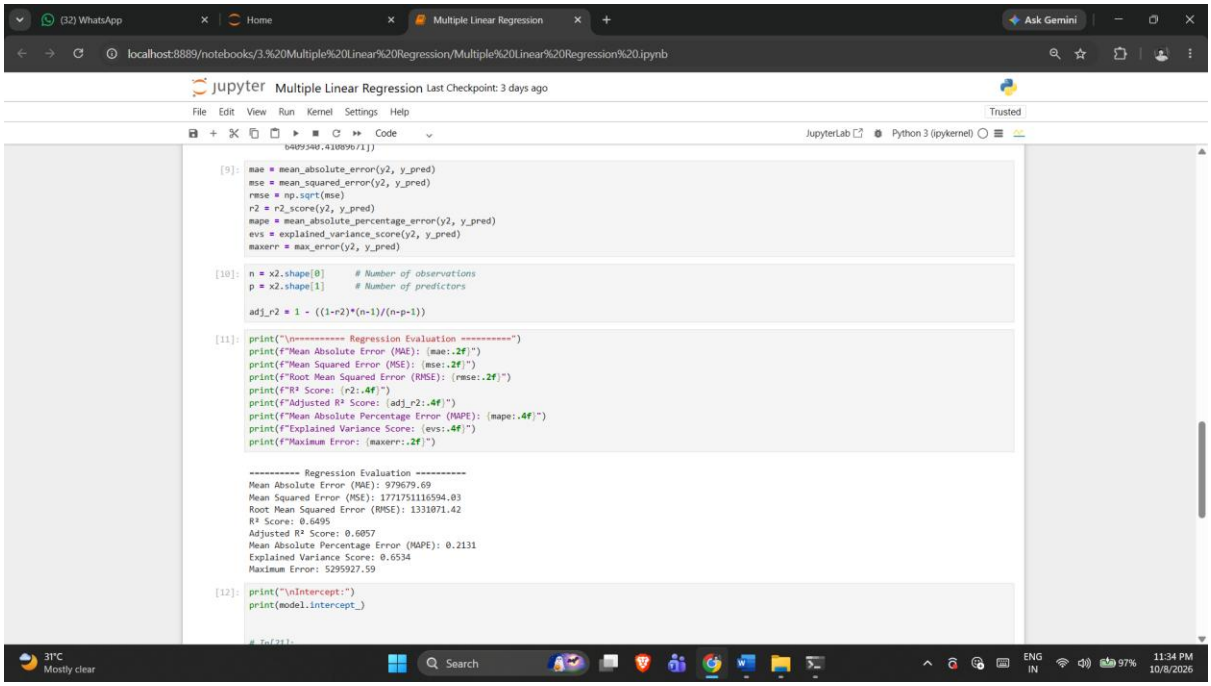
```
plt.xlabel("Actual Price")
plt.ylabel("Predicted Price")
plt.title("Actual vs Predicted Values")
plt.show()
```

4. OUTPUT






(32) WhatsApp
Home
Multiple Linear Regression
Ask Gemini
localhost:8889/notebooks/3.%20Multiple%20Linear%20Regression/Multiple%20Linear%20Regression%20.ipynb
jupyter Multiple Linear Regression Last Checkpoint: 3 days ago
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
[8]: y_pred = model.predict(x2)
y_pred
[8]: array([5203691.70963178, 7257004.02115476, 3062828.59668172,
4559591.65374424, 3332932.30559782, 3563080.67918997,
5645466.31219972, 6413979.66873635, 2755831.54819 ,
2668938.66075229, 9570600.29915353, 2827431.50860062,
3195686.2583409 , 3352263.99438471, 3713879.49996131,
5301088.24435749, 2987920.2666968 , 4810799.8212371 ,
4383031.70489929, 3525092.18938647, 5796259.50068013,
5840000.70299301, 2760214.608641 , 4762590.14920607,
5204755.73895206, 7515542.71619025, 3254681.68956382,
5236164.45964444, 8178523.16820284, 3434166.15675649,
6443921.58767581, 3346004.77919184, 6742324.74004133,
4154936.84088665, 3589152.47491253, 5788125.92515323,
4768370.18154076, 4391684.04193171, 3217657.04549935,
4638196.61928878, 4522160.27786714, 3541284.06127245,
7238136.11941171, 4021515.68926614, 3701978.76822757,
4298879.55563098, 6705004.0206061 , 3993466.52296896,
3798185.05328059, 3451821.5624289 , 7293996.86867461,
2832905.26972678, 4378698.14769931, 4468002.85418353,
3714623.89765883, 2718466.9028278 , 7524449.64983626,
2950437.1495281 , 4194596.28592921, 2795827.77318101,
4801788.72577028, 3618230.29875972, 5091685.77019752,
4248122.70647652, 4729964.20281946, 4620982.9363411 ,
7215846.76983926, 3485106.19132682, 5933820.14349913,
6236580.98095548, 4809923.61015999, 5130920.22938947,
4527282.87392583, 8006869.07208366, 3543927.63713716,
5463086.05281817, 3902441.28505163, 4661063.63495886,
4825370.89331733, 4276803.47099967, 7754072.71344458,
4008573.23235629, 6500977.91345038, 5386796.00909808,
2785343.0092764 , 6836356.36771846, 2638553.49378047,
3634231.99215348, 8004072.41154001, 8048399.75418244,
3227573.69687309, 5935167.4198354 , 3605772.48317455,
3817767.91420862, 7631673.34074558, 4775106.62670537,
5093491.44442931, 6447437.14797692, 4943449.6813564 ,
5943929.14295426, 3810460.26405425, 6457426.32302761,
3701491.23218178, 5960492.47343074, 4933137.14530295,
4377133.13624287, 7009335.02696999, 6351460.97128317,
6409340.41089671])
31°C
Mostly clear
Search
ENG
IN
97%
11:34 PM
10/8/2026


(32) WhatsApp
Home
Multiple Linear Regression
Ask Gemini
localhost:8889/notebooks/3.%20Multiple%20Linear%20Regression/Multiple%20Linear%20Regression%20.ipynb
jupyter Multiple Linear Regression Last Checkpoint: 3 days ago
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
[9]: mae = mean_absolute_error(y2, y_pred)
mse = mean_squared_error(y2, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y2, y_pred)
mape = mean_absolute_percentage_error(y2, y_pred)
evs = explained_variance_score(y2, y_pred)
maxerr = max_error(y2, y_pred)
[10]: n = x2.shape[0] # Number of observations
p = x2.shape[1] # Number of predictors
adj_r2 = 1 - ((1-r2)*(n-1)/(n-p-1))
[11]: print("\n========== Regression Evaluation ==========")
print(f"Mean Absolute Error (MAE): {mae:.2f}")
print(f"Mean Squared Error (MSE): {mse:.2f}")
print(f"Root Mean Squared Error (RMSE): {rmse:.2f}")
print(f"R² Score: {r2:.4f}")
print(f"Adjusted R² Score: {adj_r2:.4f}")
print(f"Mean Absolute Percentage Error (MAPE): {mape:.4f}")
print(f"Explained Variance Score: {evs:.4f}")
print(f"Maximum Error: {maxerr:.2f}")
========== Regression Evaluation ==========
Mean Absolute Error (MAE): 979679.69
Mean Squared Error (MSE): 1771751116594.03
Root Mean Squared Error (RMSE): 1331071.42
R² Score: 0.6495
Adjusted R² Score: 0.6057
Mean Absolute Percentage Error (MAPE): 0.2131
Explained Variance Score: 0.6534
Maximum Error: 5295927.59
[12]: print("\nIntercept:")
print(model.intercept_)
31°C
Mostly clear
Search
ENG
IN
97%
11:34 PM
10/8/2026

(32) WhatsApp | Home | Multiple Linear Regression | Ask Gemini

localhost:8889/notebooks/3.%20Multiple%20Linear%20Regression/Multiple%20Linear%20Regression%20.ipynb

jupyter Multiple Linear Regression Last Checkpoint: 3 days ago

File Edit View Run Kernel Settings Help — Trusted

Code — JupyterLab — Python 3 (ipykernel)

```
Mean Absolute Percentage Error (MAPE): 0.2131
Explained Variance Score: 0.6534
Maximum Error: 5295927.59
```

[12]:
```
print("\nIntercept:")
print(model.intercept_)


# In[21]:


print("\nCoefficients:")
coef = pd.DataFrame({
    "Feature": X.columns,
    "Coefficient": model.coef_
})
print(coef)
```

```
Intercept:
293083.0691506723

Coefficients:
             Feature   Coefficient
0               area  2.358488e+02
1           bedrooms  7.857449e+04
2          bathrooms  1.097117e+06
3            stories  4.062232e+05
4           mainroad  3.668242e+05
5          guestroom  2.331468e+05
6           basement  3.931598e+05
7    hotwaterheating  6.878813e+05
8    airconditioning  7.855506e+05
9            parking  2.257565e+05
10          prefarea  6.299017e+05
11  furnishingstatus -2.103971e+05
```

[13]:
```
new_house = [[5000,3,2,2,1,1,0,1,0,1,0,1]]

prediction = model.predict(new_house)
```

31°C Mostly clear — Search — ENG IN — 97% — 11:34 PM 10/8/2026

(32) WhatsApp | Home | Multiple Linear Regression | Ask Gemini

localhost:8889/notebooks/3.%20Multiple%20Linear%20Regression/Multiple%20Linear%20Regression%20.ipynb

jupyter Multiple Linear Regression Last Checkpoint: 3 days ago

File Edit View Run Kernel Settings Help — Trusted

Code — JupyterLab — Python 3 (ipykernel)

```
2          bathrooms  1.097117e+06
3            stories  4.062232e+05
4           mainroad  3.668242e+05
5          guestroom  2.331468e+05
6           basement  3.931598e+05
7    hotwaterheating  6.878813e+05
8    airconditioning  7.855506e+05
9            parking  2.257565e+05
10          prefarea  6.299017e+05
11  furnishingstatus -2.103971e+05
```

[13]:
```
new_house = [[5000,3,2,2,1,1,0,1,0,1,0,1]]

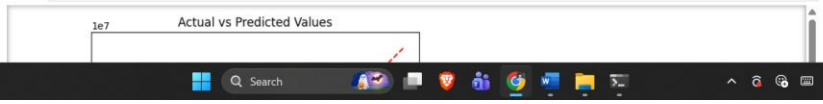
prediction = model.predict(new_house)

print("\nPredicted House Price:", prediction[0])
```

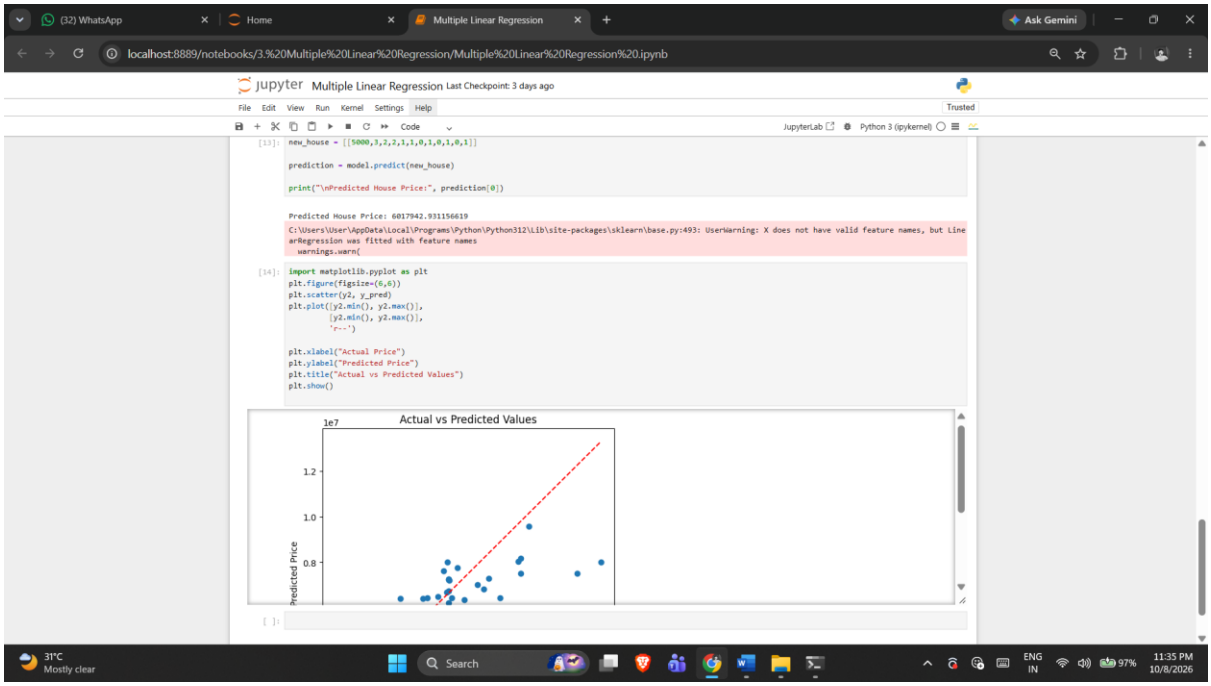
```
Predicted House Price: 6017942.931156619
C:\Users\User\AppData\Local\Programs\Python\Python312\Lib\site-packages\sklearn\base.py:493: UserWarning: X does not have valid feature names, but LinearRegression was fitted with feature names
  warnings.warn(
```

[14]:
```
import matplotlib.pyplot as plt
plt.figure(figsize=(6,6))
plt.scatter(y2, y_pred)
plt.plot([y2.min(), y2.max()],
         [y2.min(), y2.max()],
         'r--')

plt.xlabel("Actual Price")
plt.ylabel("Predicted Price")
plt.title("Actual vs Predicted Values")
plt.show()
```



31°C Mostly clear — Search — ENG IN — 97% — 11:34 PM 10/8/2026



5. RESULT

Thus, the Multiple Linear Regression practical was implemented successfully using the program from the corresponding Jupyter Notebook.

Exercise -4

Polynomial Regression

1. AIM

To implement degree-2 polynomial regression, evaluate its predictions with the notebook's regression metrics, and visualize the fitted curve.

2. ALGORITHM

1. Load and inspect the fish dataset.

2. Encode the Species feature.

3. Separate Weight as target.

4. Split into training and testing sets.

5. Generate degree-2 polynomial features.

6. Train Linear Regression on transformed features.

7. Calculate all metrics implemented in the notebook.

8. Plot the polynomial relationship for Length3 and Weight.

3. PROGRAM

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression

from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score,
    mean_absolute_percentage_error,
    explained_variance_score,
    max_error
)

data = 'Dataset/fish.csv'
df = pd.read_csv(data)

df.head()

df.info()

df.shape

df = pd.get_dummies(df, columns=['Species'], drop_first=True)
```

```
X = df.drop("Weight", axis=1)
y = df["Weight"]

x1, x2, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

poly = PolynomialFeatures(degree=2)

X_train_poly = poly.fit_transform(x1)
X_test_poly = poly.transform(x2)

model = LinearRegression()

model.fit(X_train_poly, y_train)

# Prediction
y_pred = model.predict(X_test_poly)

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)
mape = mean_absolute_percentage_error(y_test, y_pred)
evs = explained_variance_score(y_test, y_pred)
maxerr = max_error(y_test, y_pred)

# Adjusted R²
n = X_test_poly.shape[0]
p = X_test_poly.shape[1] - 1

adj_r2 = 1 - ((1-r2)*(n-1)/(n-p-1))

print("\n========== Evaluation Metrics ==========")

print("MAE :", mae)
print("MSE :", mse)
print("RMSE :", rmse)
print("R2 Score :", r2)
print("Adjusted R2 :", adj_r2)
print("MAPE :", mape)
print("Explained Variance :", evs)
print("Maximum Error :", maxerr)

X = df[['Length3']]
y = df['Weight']
poly = PolynomialFeatures(degree=2)

X_poly = poly.fit_transform(X)

model = LinearRegression()
model.fit(X_poly, y)

X_grid = np.arange(
    X.min().values[0],
```

```
        X.max().values[0],
        0.1
    ).reshape(-1,1)

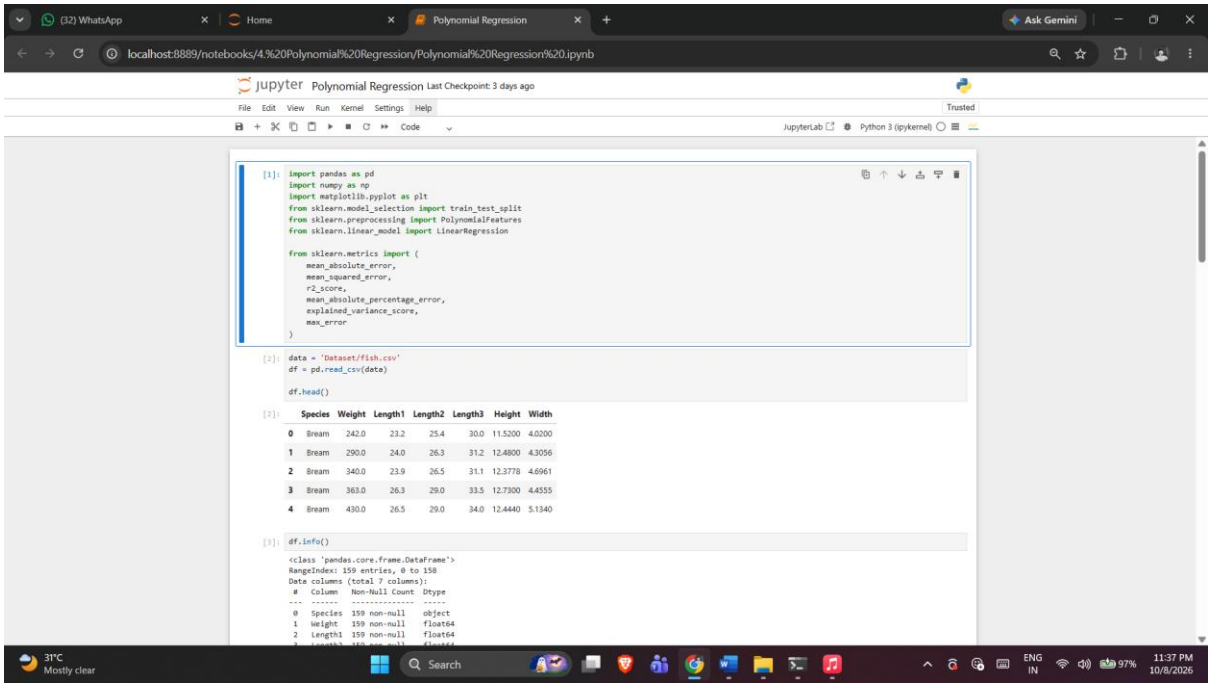
    plt.figure(figsize=(8,6))

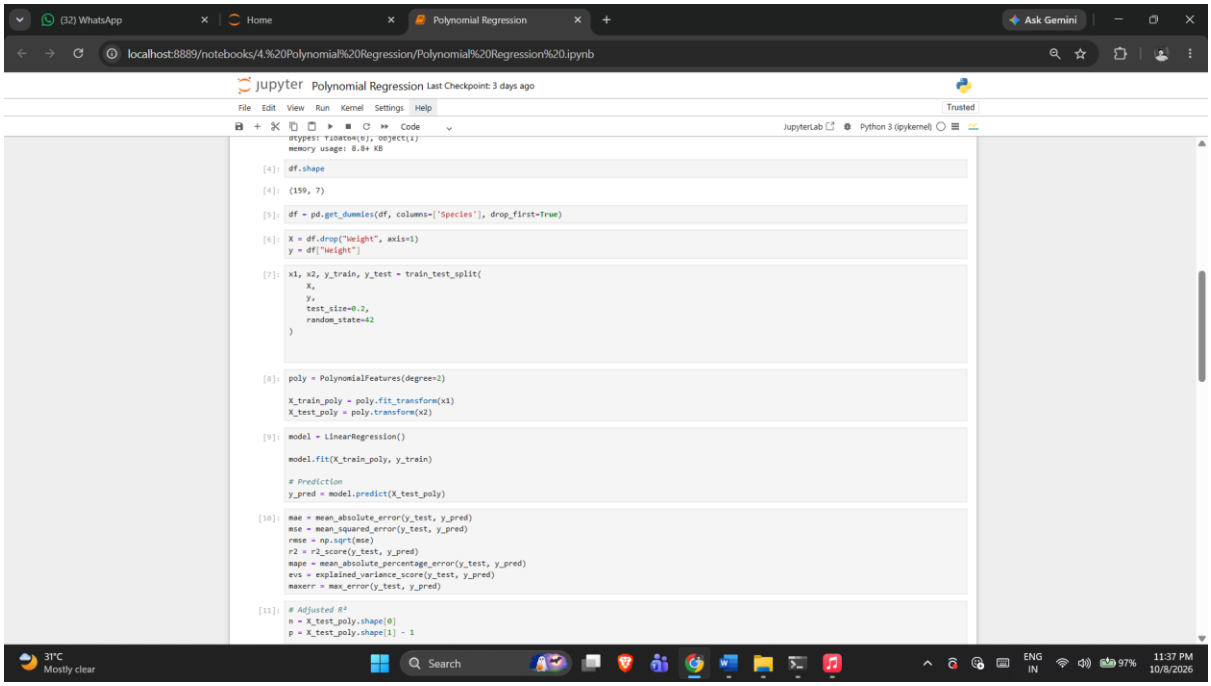
    plt.scatter(X, y, color="blue", label="Actual Data")

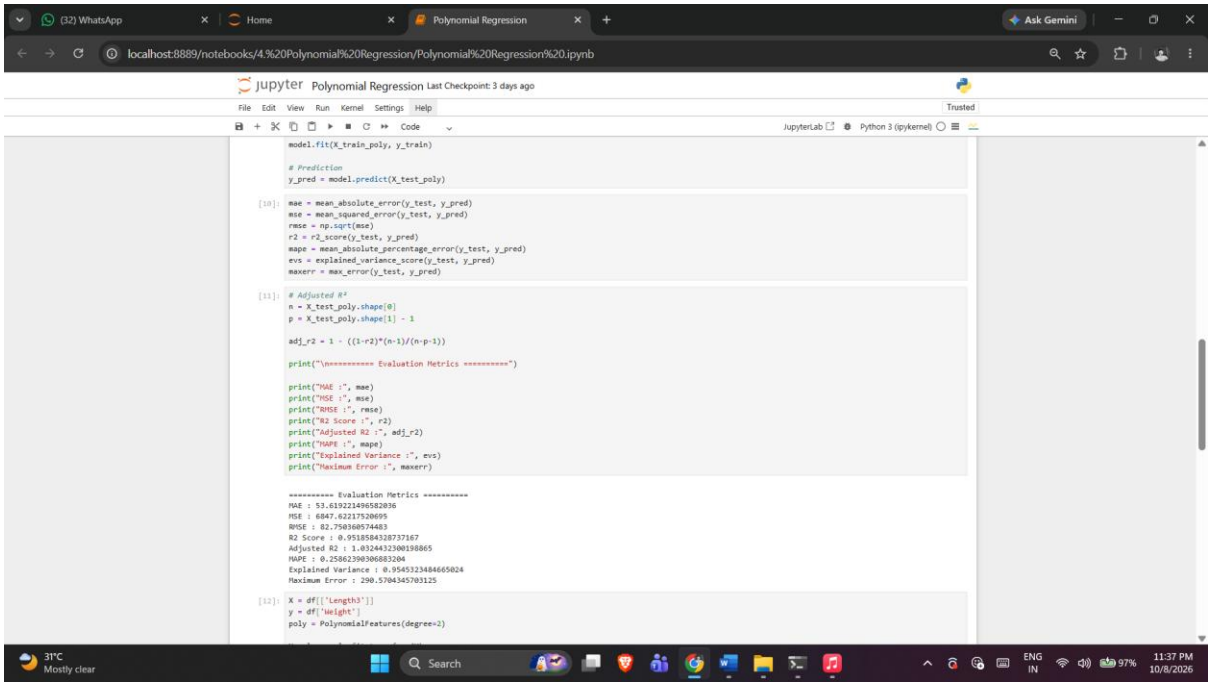
    plt.plot(
        X_grid,
        model.predict(poly.transform(X_grid)),
        color="red",
        linewidth=3,
        label="Polynomial Curve"
    )

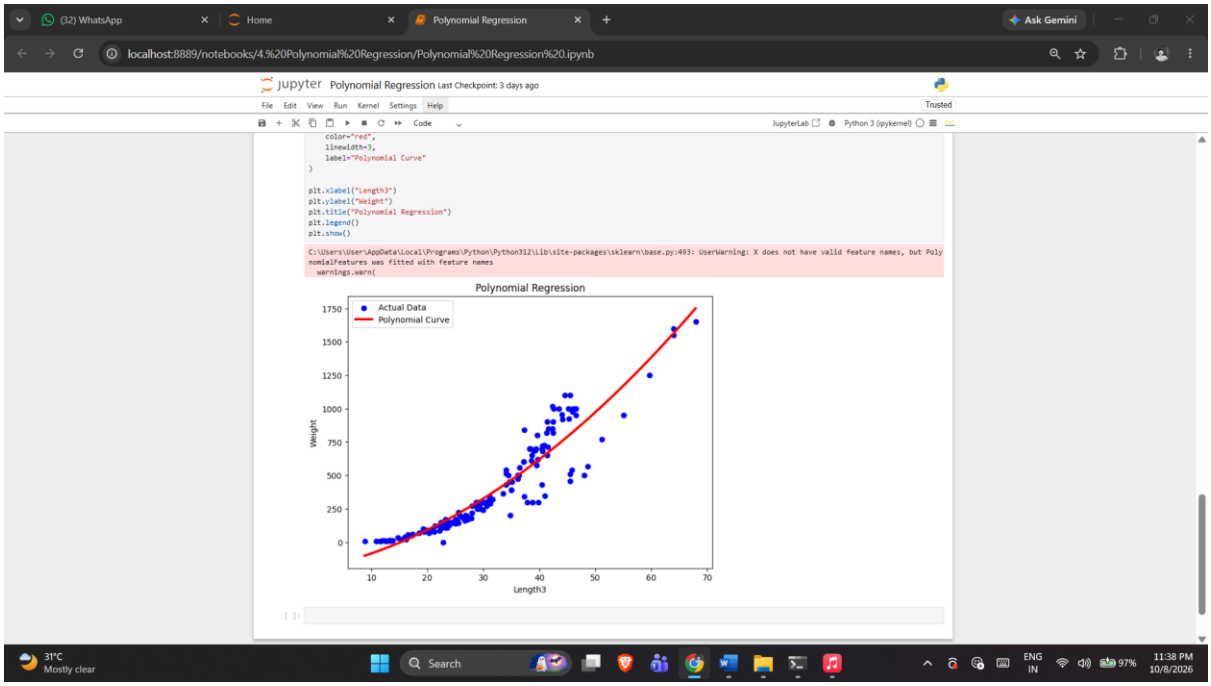
    plt.xlabel("Length3")
    plt.ylabel("Weight")
    plt.title("Polynomial Regression")
    plt.legend()
    plt.show()
```

4. OUTPUT




(32) WhatsApp
Home
Polynomial Regression
Ask Gemini
localhost:8889/notebooks/4.%20Polynomial%20Regression/Polynomial%20Regression%20.ipynb
jupyter Polynomial Regression Last Checkpoint: 3 days ago
File Edit View Run Kernel Settings Help
Trusted
JupyterLab
Python 3 (ipykernel)
memory usage: 8.8+ KB
[4]: df.shape
[4]: (159, 7)
[5]: df = pd.get_dummies(df, columns=['Species'], drop_first=True)
[6]: X = df.drop("Weight", axis=1)
y = df["Weight"]
[7]: x1, x2, y_train, y_test = train_test_split(
X,
y,
test_size=0.2,
random_state=42
)
[8]: poly = PolynomialFeatures(degree=2)
X_train_poly = poly.fit_transform(x1)
X_test_poly = poly.transform(x2)
[9]: model = LinearRegression()
model.fit(X_train_poly, y_train)
Prediction
y_pred = model.predict(X_test_poly)
[10]: mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)
mape = mean_absolute_percentage_error(y_test, y_pred)
evs = explained_variance_score(y_test, y_pred)
maxerr = max_error(y_test, y_pred)
[11]: # Adjusted R²
n = X_test_poly.shape[0]
p = X_test_poly.shape[1] - 1
31°C
Mostly clear
Search
ENG
IN
97%
11:37 PM
10/8/2026


(32) WhatsApp
Home
Polynomial Regression
Ask Gemini
localhost:8889/notebooks/4.%20Polynomial%20Regression/Polynomial%20Regression%20.ipynb
jupyter Polynomial Regression Last Checkpoint: 3 days ago
File Edit View Run Kernel Settings Help
Trusted
JupyterLab
Python 3 (ipykernel)
model.fit(X_train_poly, y_train)
Prediction
y_pred = model.predict(X_test_poly)
[10]: mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)
mape = mean_absolute_percentage_error(y_test, y_pred)
evs = explained_variance_score(y_test, y_pred)
maxerr = max_error(y_test, y_pred)
[11]: # Adjusted R²
n = X_test_poly.shape[0]
p = X_test_poly.shape[1] - 1
adj_r2 = 1 - ((1-r2)*(n-1)/(n-p-1))
print("\n========== Evaluation Metrics ==========")
print("MAE :", mae)
print("MSE :", mse)
print("RMSE :", rmse)
print("R2 Score :", r2)
print("Adjusted R2 :", adj_r2)
print("MAPE :", mape)
print("Explained Variance :", evs)
print("Maximum Error :", maxerr)
========== Evaluation Metrics ==========
MAE : 53.619221496582036
MSE : 6847.622175206095
RMSE : 82.75036057448
R2 Score : 0.9518584328737167
Adjusted R2 : 1.0324432300198865
MAPE : 0.25062390306883204
Explained Variance : 0.9545323484665024
Maximum Error : 290.5704345703125
[12]: X = df[['Length3']]
y = df['Weight']
poly = PolynomialFeatures(degree=2)
31°C
Mostly clear
Search
ENG
IN
97%
11:37 PM
10/8/2026



5. RESULT

Thus, the Polynomial Regression practical was implemented successfully using the program from the corresponding Jupyter Notebook.

Exercise -5

Regularization Techniques

1. AIM

To implement and demonstrate Regularization Techniques using the dataset, preprocessing, model or analysis workflow provided in the corresponding Jupyter Notebook.

2. ALGORITHM

1. Load and prepare the regression data.

2. Separate predictors and target.

3. Split the data.

4. Train the regularization models implemented in the notebook.

5. Generate predictions.

6. Compare models using the notebook's evaluation metrics.

3. PROGRAM

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, PolynomialFeatures
from sklearn.linear_model import LinearRegression, Ridge, Lasso, ElasticNet
from sklearn.metrics import r2_score

data = 'Dataset/Housing.csv'
df = pd.read_csv(data)
df.head()

df = pd.get_dummies(df, drop_first=True)

df.head()



# In[5]:



X = df.drop("price", axis=1)
y = df["price"]



# In[6]:



X_train,X_test,y_train,y_test = train_test_split(
```

```
X,
y,
test_size=0.2,
random_state=42
)



# In[7]:



scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)



# In[80]:



poly = PolynomialFeatures(
    degree=4,
    include_bias=False
)

X_train_poly = poly.fit_transform(X_train_scaled)
X_test_poly = poly.transform(X_test_scaled)

print("Original Features :", X_train.shape[1])
print("Polynomial Features :", X_train_poly.shape[1])



# In[81]:



linear = LinearRegression()

linear.fit(X_train_poly, y_train)

train_pred = linear.predict(X_train_poly)
test_pred = linear.predict(X_test_poly)

train_r2 = r2_score(y_train, train_pred)
test_r2 = r2_score(y_test, test_pred)

print("LINEAR REGRESSION")
print("Training R2 :", round(train_r2,4))
print("Testing R2 :", round(test_r2,4))



# In[83]:



ridge = Ridge(alpha=100)
```

```
ridge.fit(X_train_poly, y_train)

train_pred = ridge.predict(X_train_poly)
test_pred = ridge.predict(X_test_poly)

ridge_train = r2_score(y_train, train_pred)
ridge_test = r2_score(y_test, test_pred)

print("RIDGE")
print("Training R2 :", round(ridge_train,4))
print("Testing R2 :", round(ridge_test,4))



# In[ ]:



lasso = Lasso(
    alpha=0.1,
    max_iter=50000
)

lasso.fit(X_train_poly, y_train)

train_pred = lasso.predict(X_train_poly)
test_pred = lasso.predict(X_test_poly)

lasso_train = r2_score(y_train, train_pred)
lasso_test = r2_score(y_test, test_pred)

print("LASSO")
print("Training R2 :", round(lasso_train,4))
print("Testing R2 :", round(lasso_test,4))



# In[77]:



elastic = ElasticNet(
    alpha=0.1,
    l1_ratio=0.5,
    max_iter=50000
)

elastic.fit(X_train_poly, y_train)

train_pred = elastic.predict(X_train_poly)
test_pred = elastic.predict(X_test_poly)

elastic_train = r2_score(y_train, train_pred)
elastic_test = r2_score(y_test, test_pred)

print("ELASTIC NET")
print("Training R2 :", round(elastic_train,4))
print("Testing R2 :", round(elastic_test,4))
```

```
# In[78]:


comparison = pd.DataFrame({

    "Model":[
        "Linear Regression",
        "Ridge",
        "Lasso",
        "Elastic Net"
    ],

    "Train R2":[
        train_r2,
        ridge_train,
        lasso_train,
        elastic_train
    ],

    "Test R2":[
        test_r2,
        ridge_test,
        lasso_test,
        elastic_test
    ]
})

print(comparison)


# In[79]:







# In[62]:

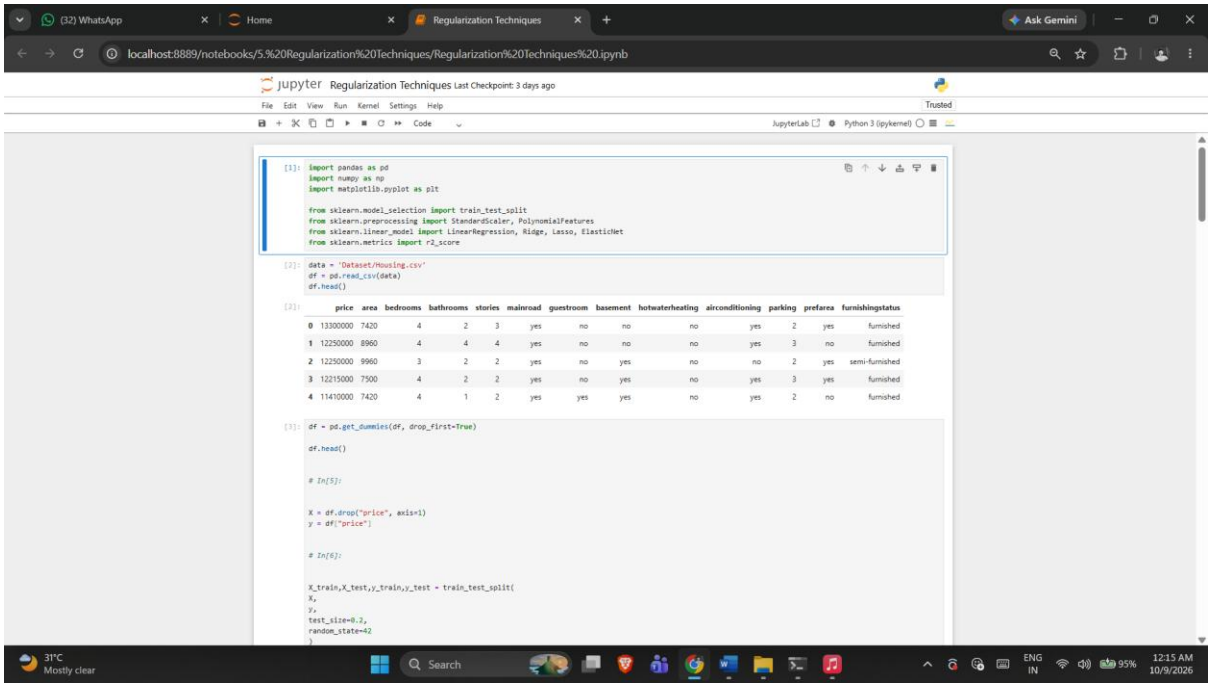

comparison["Difference"] = comparison["Train R2"] - comparison["Test R2"]

print(comparison)

comparison.plot(
    x="Model",
    y=["Train R2", "Test R2"],
    kind="bar",
    figsize=(10,6)
)

plt.title("Reducing Overfitting using Regularization")
plt.ylabel("R² Score")
plt.xticks(rotation=10)
plt.grid(axis="y")
plt.show()
```

4. OUTPUT



```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, PolynomialFeatures
from sklearn.linear_model import LinearRegression, Ridge, Lasso, ElasticNet
from sklearn.metrics import r2_score
```

```
data = 'Dataset/Housing.csv'
df = pd.read_csv(data)
df.head()
```

	price	area	bedrooms	bathrooms	stories	mainroad	guestroom	basement	hotwaterheating	airconditioning	parking	prefarea	furnishingstatus
0	13300000	7420	4	2	3	yes	no	no	no	yes	2	yes	furnished
1	12250000	8960	4	4	4	yes	no	no	no	yes	3	no	furnished
2	12250000	9960	3	2	2	yes	no	yes	no	no	2	yes	semi-furnished
3	12215000	7500	4	2	2	yes	no	yes	no	yes	3	yes	furnished
4	11410000	7420	4	1	2	yes	yes	yes	no	yes	2	no	furnished

```
df = pd.get_dummies(df, drop_first=True)

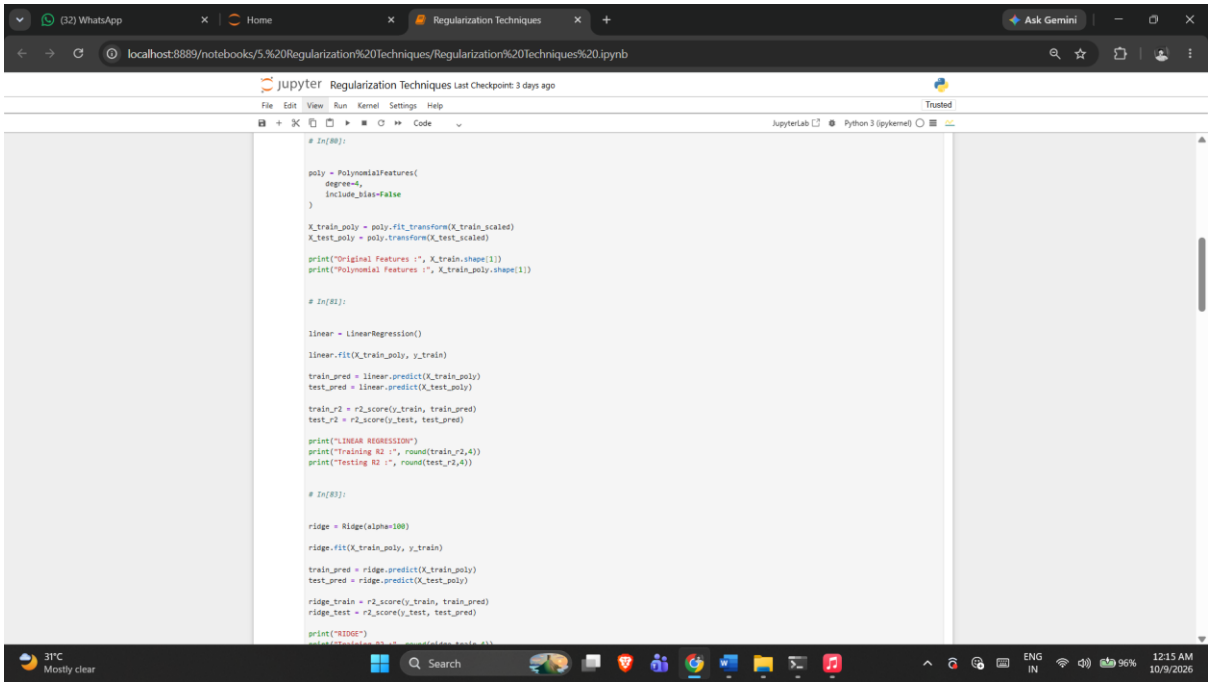
df.head()


# In[5]:


X = df.drop("price", axis=1)
y = df["price"]


# In[6]:


X_train,X_test,y_train,y_test = train_test_split(
X,
y,
test_size=0.2,
random_state=42
)
```



```
# In[80]:


poly = PolynomialFeatures(
    degree=4,
    include_bias=False
)

X_train_poly = poly.fit_transform(X_train_scaled)
X_test_poly = poly.transform(X_test_scaled)

print("Original Features :", X_train.shape[1])
print("Polynomial Features :", X_train_poly.shape[1])


# In[81]:


linear = LinearRegression()

linear.fit(X_train_poly, y_train)

train_pred = linear.predict(X_train_poly)
test_pred = linear.predict(X_test_poly)

train_r2 = r2_score(y_train, train_pred)
test_r2 = r2_score(y_test, test_pred)

print("LINEAR REGRESSION")
print("Training R2 :", round(train_r2,4))
print("Testing R2 :", round(test_r2,4))


# In[83]:

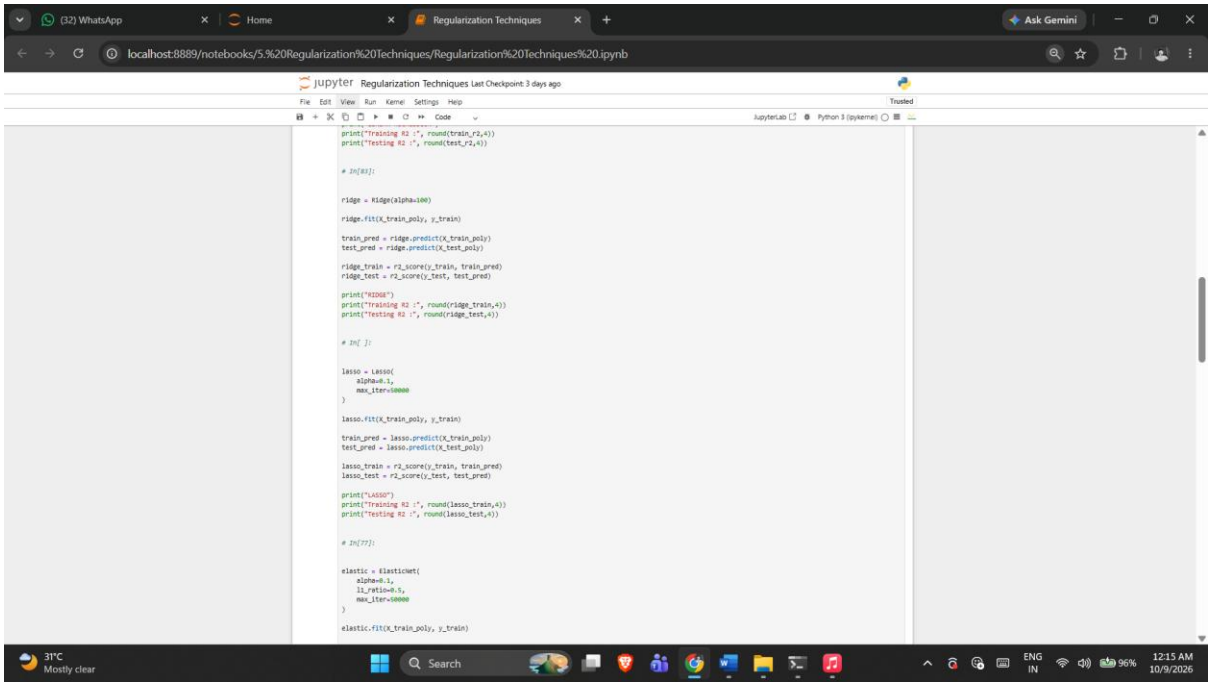

ridge = Ridge(alpha=100)

ridge.fit(X_train_poly, y_train)

train_pred = ridge.predict(X_train_poly)
test_pred = ridge.predict(X_test_poly)

ridge_train = r2_score(y_train, train_pred)
ridge_test = r2_score(y_test, test_pred)

print("RIDGE")
```



```
print("Training R2 :", round(train_r2,4))
print("Testing R2 :", round(test_r2,4))


# In[83]:


ridge = Ridge(alpha=100)

ridge.fit(X_train_poly, y_train)

train_pred = ridge.predict(X_train_poly)
test_pred = ridge.predict(X_test_poly)

ridge_train = r2_score(y_train, train_pred)
ridge_test = r2_score(y_test, test_pred)

print("RIDGE")
print("Training R2 :", round(ridge_train,4))
print("Testing R2 :", round(ridge_test,4))


# In[ ]:


lasso = Lasso(
    alpha=0.1,
    max_iter=50000
)

lasso.fit(X_train_poly, y_train)

train_pred = lasso.predict(X_train_poly)
test_pred = lasso.predict(X_test_poly)

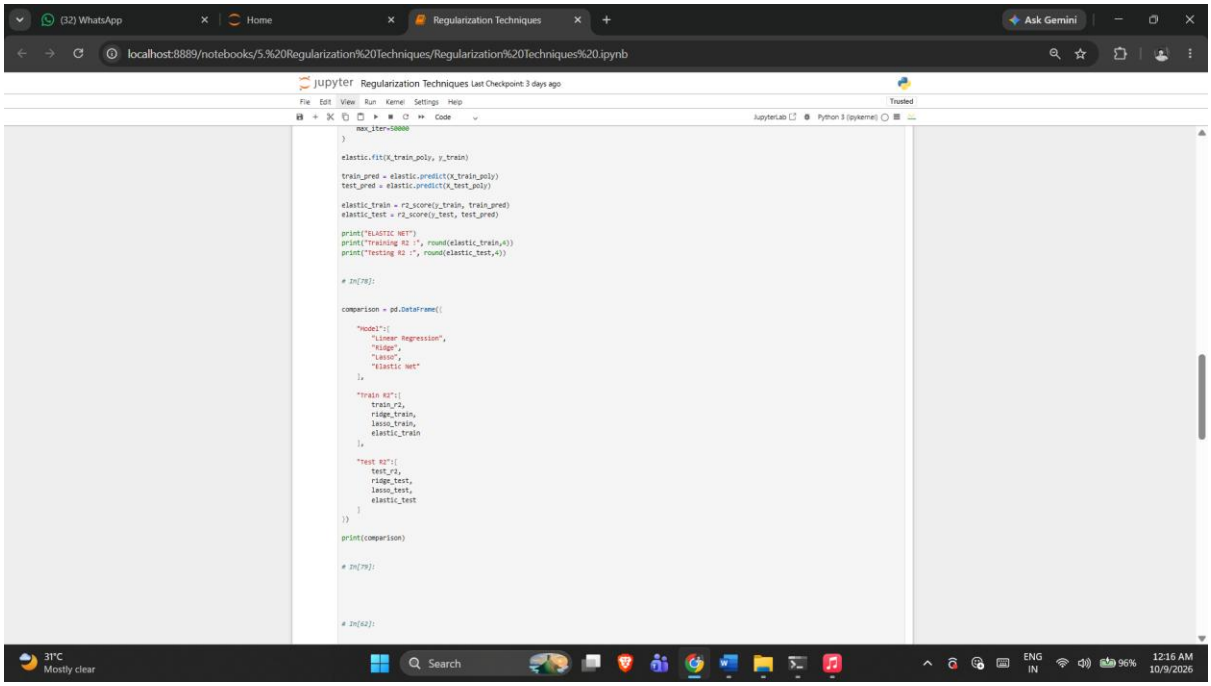
lasso_train = r2_score(y_train, train_pred)
lasso_test = r2_score(y_test, test_pred)

print("LASSO")
print("Training R2 :", round(lasso_train,4))
print("Testing R2 :", round(lasso_test,4))


# In[77]:


elastic = ElasticNet(
    alpha=0.1,
    l1_ratio=0.5,
    max_iter=50000
)

elastic.fit(X_train_poly, y_train)
```



```
    max_iter=50000
)

elastic.fit(X_train_poly, y_train)

train_pred = elastic.predict(X_train_poly)
test_pred = elastic.predict(X_test_poly)

elastic_train = r2_score(y_train, train_pred)
elastic_test = r2_score(y_test, test_pred)

print("ELASTIC NET")
print("Training R2 :", round(elastic_train,4))
print("Testing R2 :", round(elastic_test,4))


# In[78]:


comparison = pd.DataFrame({

    "Model":[
        "Linear Regression",
        "Ridge",
        "Lasso",
        "Elastic Net"
    ],

    "Train R2":[
        train_r2,
        ridge_train,
        lasso_train,
        elastic_train
    ],

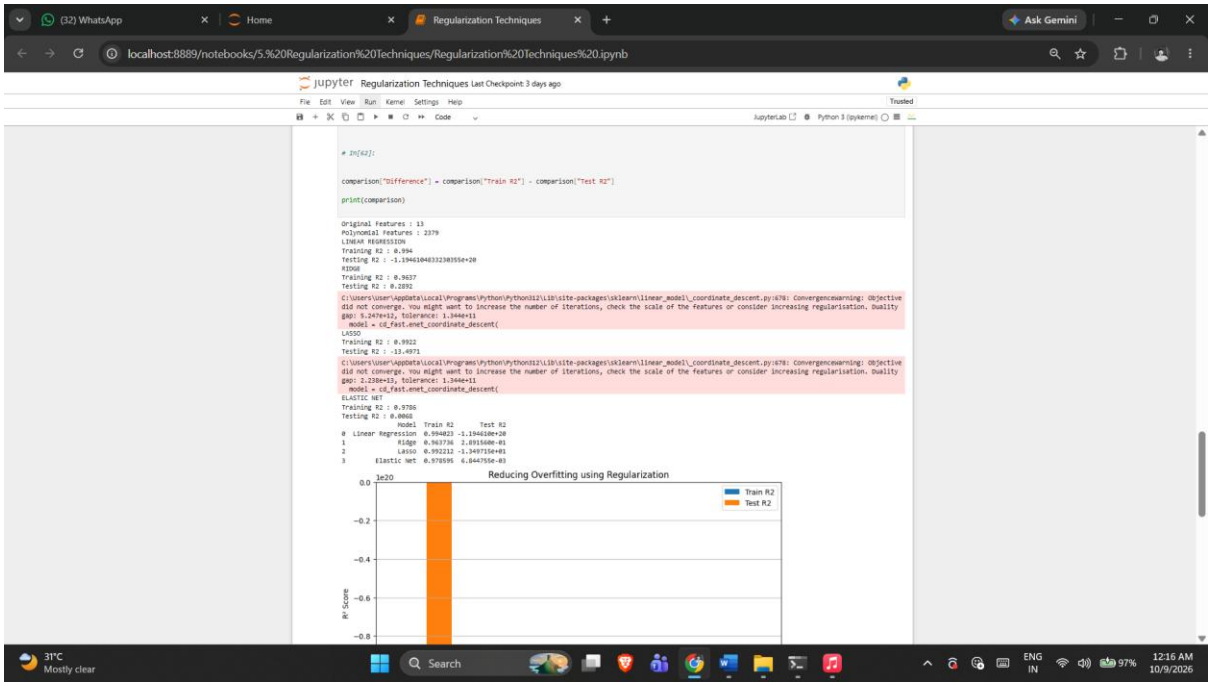
    "Test R2":[
        test_r2,
        ridge_test,
        lasso_test,
        elastic_test
    ]
})

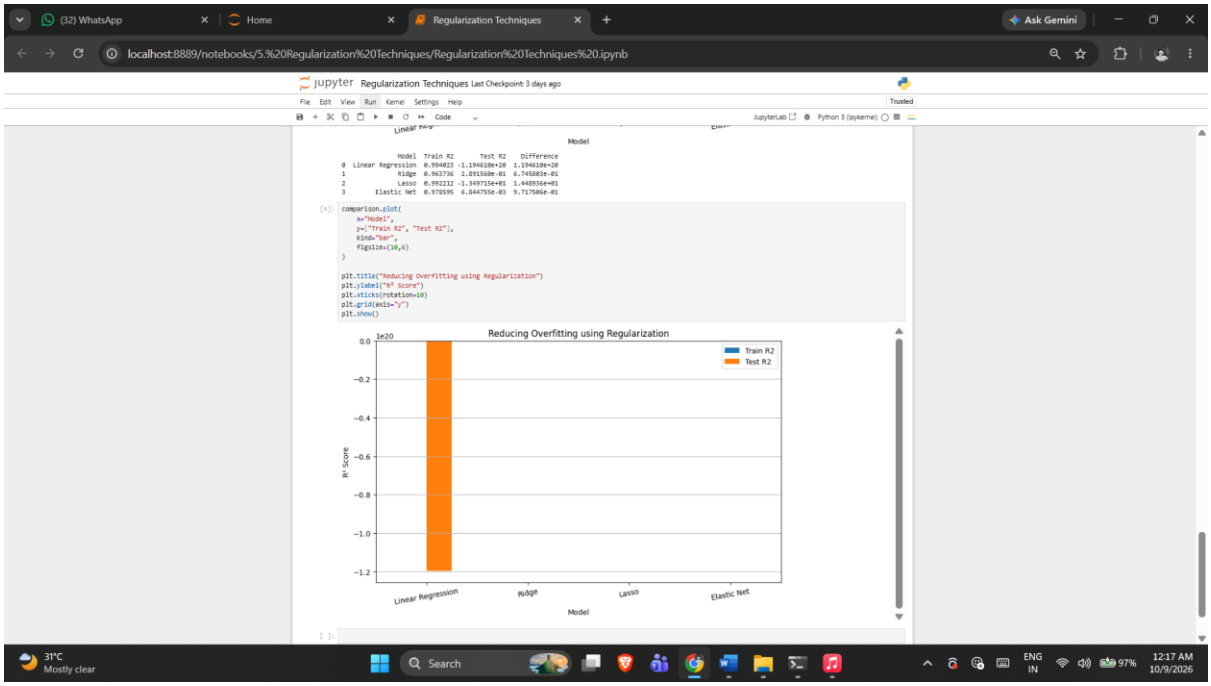
print(comparison)


# In[79]:



# In[62]:
```





5. RESULT

Thus, the Regularization Techniques practical was implemented successfully using the program from the corresponding Jupyter Notebook.

Exercise -6

Logistic Regression

1. AIM

To implement Logistic Regression for binary heart disease classification and evaluate the model using the metrics implemented in the notebook.

2. ALGORITHM

1. Load the dataset and separate features and target.

2. Identify numerical and categorical features.

3. Create the preprocessing pipeline.

4. Split the data into training and testing sets.

5. Train the Logistic Regression model.

6. Predict the test set and calculate the notebook's metrics.

7. Display the model performance.

3. PROGRAM

```
#!/usr/bin/env python
# coding: utf-8

import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import MinMaxScaler

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report,
    roc_curve,
    auc
)

df = pd.read_csv("heart_disease_dataset.csv")

print("First 5 rows:")
print(df.head())
```

```
print("\nDataset Shape:")
print(df.shape)

X = df.drop("Heart Disease", axis=1)
y = df["Heart Disease"]

print("\nFeatures:")
print(X.columns)

print("\nTarget:")
print(y.name)

categorical_columns = X.select_dtypes(
    include=["object"]
).columns

numerical_columns = X.select_dtypes(
    exclude=["object"]
).columns

print("\nCategorical Columns:")
print(list(categorical_columns))

print("\nNumerical Columns:")
print(list(numerical_columns))

imputer = SimpleImputer(strategy="most_frequent")

X[categorical_columns] = imputer.fit_transform(
    X[categorical_columns]
)

print("\nMissing values after imputation:")
print(X.isnull().sum())

label_encoders = {}

for column in categorical_columns:

    encoder = LabelEncoder()

    X[column] = encoder.fit_transform(
        X[column]
    )

    label_encoders[column] = encoder



print("\nData after Label Encoding:")
print(X.head())

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
```

```
    random_state=42,
    stratify=y
)

print("\nTraining data:", X_train.shape)
print("Testing data :", X_test.shape)

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)

# minmax_scaler = MinMaxScaler()

# X_train_scaled = minmax_scaler.fit_transform(X_train)

# X_test_scaled = minmax_scaler.transform(X_test)

model = LogisticRegression(
    max_iter=1000,
    random_state=42
)
model.fit(
    X_train_scaled,
    y_train
)

print("\nLogistic Regression model trained successfully.")

y_pred = model.predict(X_test_scaled)
# Probability of class 1
y_probability = model.predict_proba(X_test_scaled)[:, 1]

accuracy = accuracy_score(
    y_test,
    y_pred
)

precision = precision_score(
    y_test,
    y_pred
)

recall = recall_score(
    y_test,
    y_pred
)

f1 = f1_score(
    y_test,
    y_pred
)



print("\n================================")
print("CLASSIFICATION METRICS")
```

```
print("================================")

print("Accuracy  :", round(accuracy, 4))
print("Precision :", round(precision, 4))
print("Recall    :", round(recall, 4))
print("F1 Score  :", round(f1, 4))

cm = confusion_matrix(
    y_test,
    y_pred
)

print("\nConfusion Matrix:")
print(cm)

print("\nClassification Report:")
print(
    classification_report(
        y_test,
        y_pred
    )
)

# ==========================================
# 15. ROC Curve
# ==========================================

fpr, tpr, thresholds = roc_curve(
    y_test,
    y_probability
)



# ==========================================
# 16. Calculate AUC
# ==========================================

roc_auc = auc(
    fpr,
    tpr
)

print("\nROC-AUC Score:",
      round(roc_auc, 4))



# ==========================================
# 17. Plot ROC Curve
# ==========================================

plt.figure(figsize=(8, 6))

plt.plot(
    fpr,
    tpr,
    label="Logistic Regression (AUC = {:.3f})".format(
        roc_auc
    )
```

```
)

# Random classifier line
plt.plot(
    [0, 1],
    [0, 1],
    linestyle="--",
    label="Random Classifier"
)

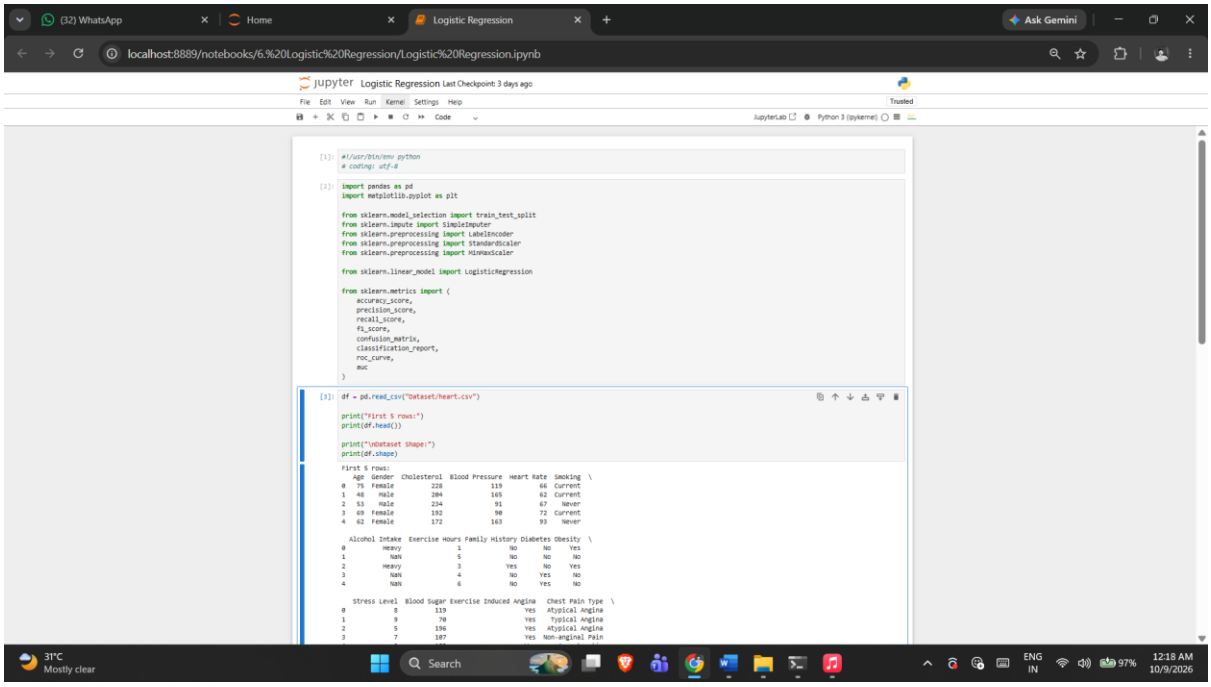
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")

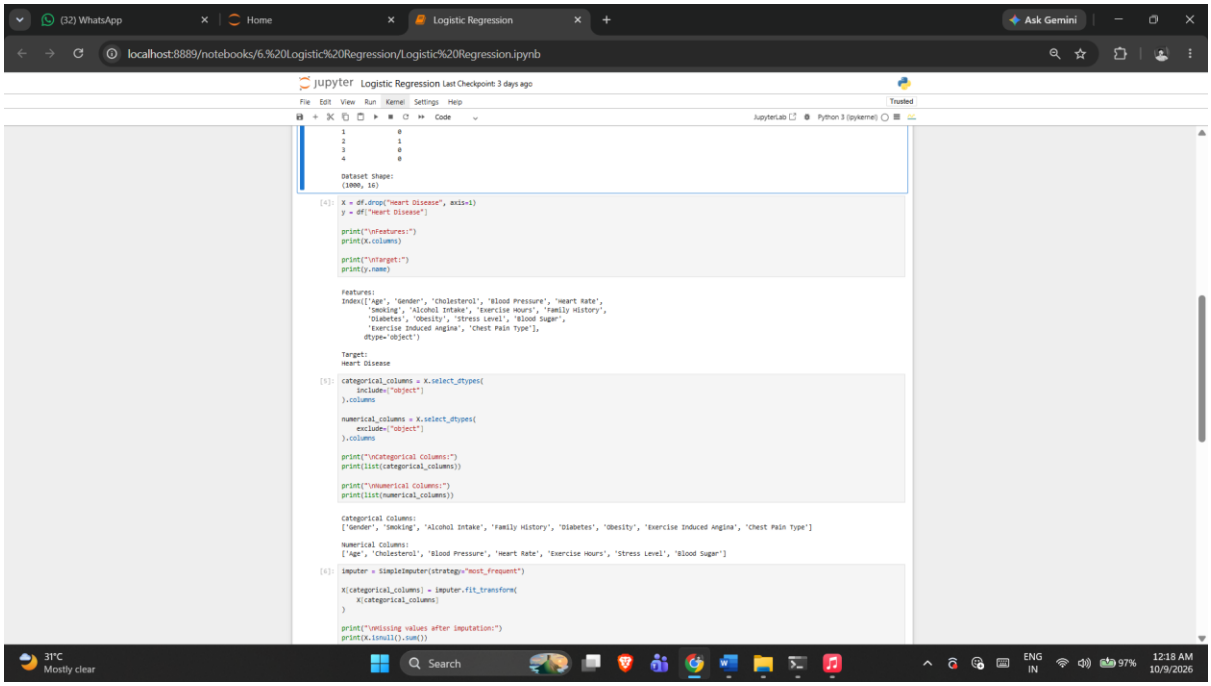
plt.title(
    "ROC Curve - Heart Disease Prediction"
)

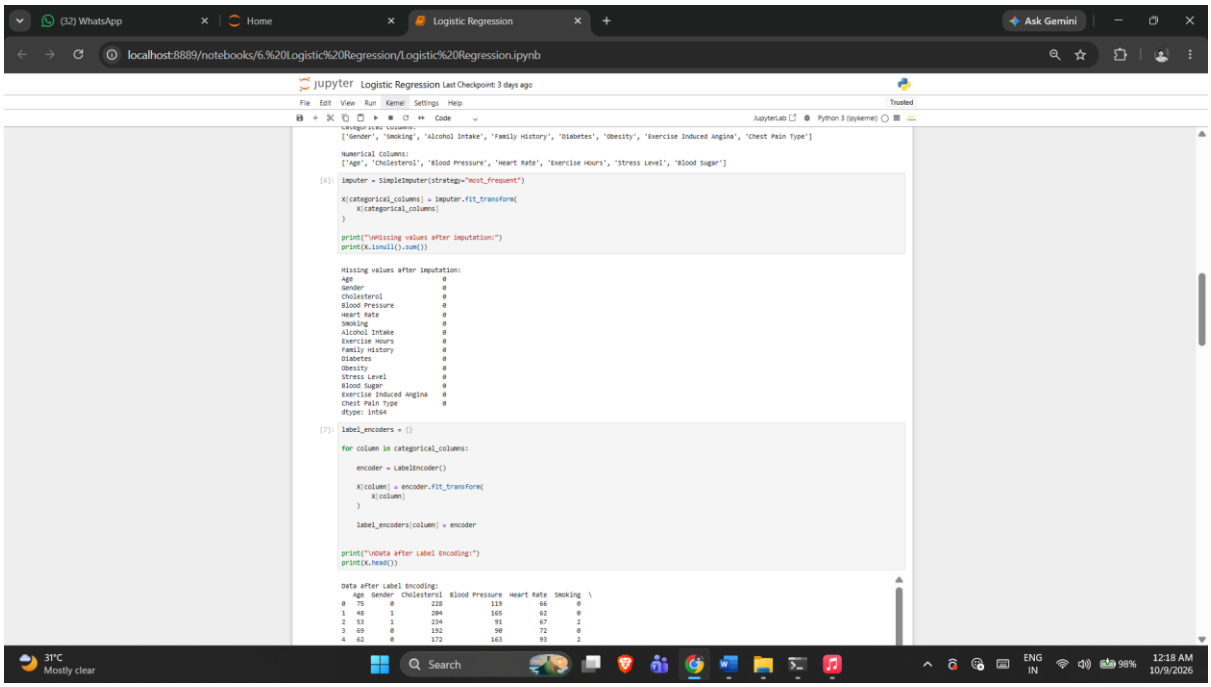
plt.legend()

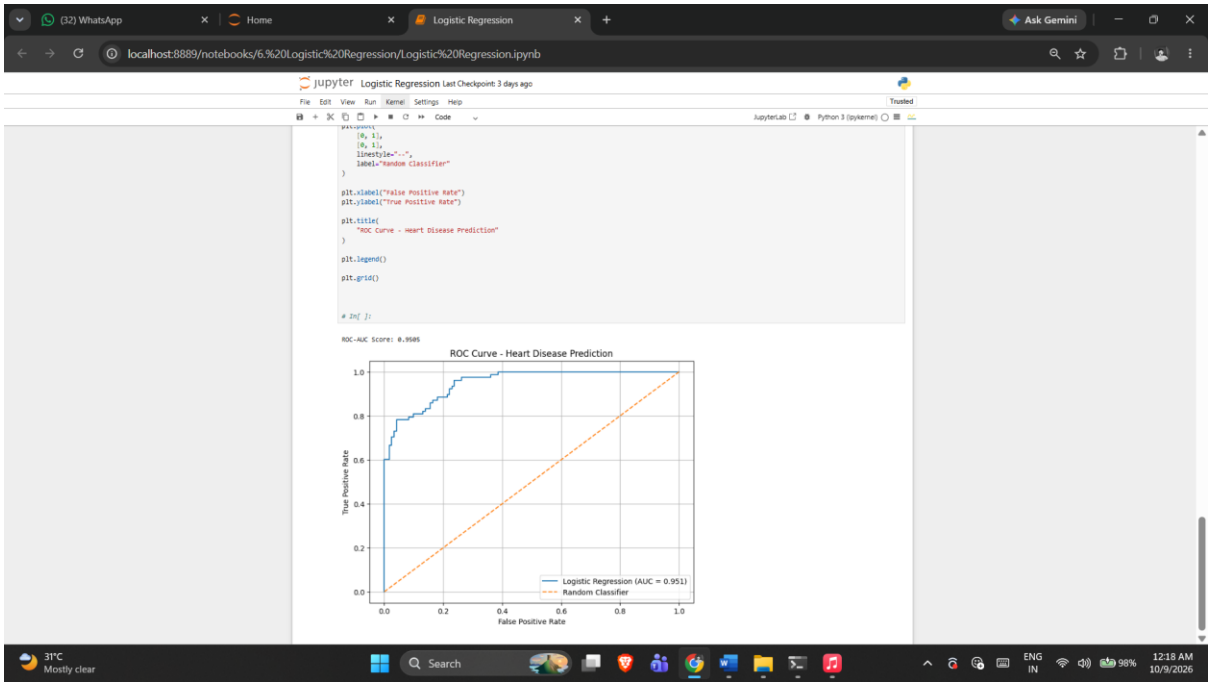
plt.grid()
```

4. OUTPUT




(32) WhatsApp
Home
Logistic Regression
Ask Gemini
localhost:8889/notebooks/6.%20Logistic%20Regression/Logistic%20Regression.ipynb
jupyter Logistic Regression Last Checkpoint: 3 days ago
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
1 0
2 1
3 0
4 0
Dataset Shape:
(1000, 16)
[4]: X = df.drop("Heart Disease", axis=1)
y = df["Heart Disease"]
print("\nFeatures:")
print(X.columns)
print("\nTarget:")
print(y.name)
Features:
Index(['Age', 'Gender', 'Cholesterol', 'Blood Pressure', 'Heart Rate',
'Smoking', 'Alcohol Intake', 'Exercise Hours', 'Family History',
'Diabetes', 'Obesity', 'Stress Level', 'Blood Sugar',
'Exercise Induced Angina', 'Chest Pain Type'],
dtype='object')
Target:
Heart Disease
[5]: categorical_columns = X.select_dtypes(
include=["object"]
).columns
numerical_columns = X.select_dtypes(
exclude=["object"]
).columns
print("\nCategorical Columns:")
print(list(categorical_columns))
print("\nNumerical Columns:")
print(list(numerical_columns))
Categorical Columns:
['Gender', 'Smoking', 'Alcohol Intake', 'Family History', 'Diabetes', 'Obesity', 'Exercise Induced Angina', 'Chest Pain Type']
Numerical Columns:
['Age', 'Cholesterol', 'Blood Pressure', 'Heart Rate', 'Exercise Hours', 'Stress Level', 'Blood Sugar']
[6]: imputer = SimpleImputer(strategy="most_frequent")
X[categorical_columns] = imputer.fit_transform(
X[categorical_columns]
)
print("\nMissing values after imputation:")
print(X.isnull().sum())
31°C
Mostly clear
Search
ENG
IN
97%
12:18 AM
10/9/2026


(32) WhatsApp
Home
Logistic Regression
Ask Gemini
localhost:8889/notebooks/6.%20Logistic%20Regression/Logistic%20Regression.ipynb
jupyter Logistic Regression Last Checkpoint: 3 days ago
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
['Gender', 'Smoking', 'Alcohol Intake', 'Family History', 'Diabetes', 'Obesity', 'Exercise Induced Angina', 'Chest Pain Type']
Numerical Columns:
['Age', 'Cholesterol', 'Blood Pressure', 'Heart Rate', 'Exercise Hours', 'Stress Level', 'Blood Sugar']
[6]: imputer = SimpleImputer(strategy="most_frequent")
X[categorical_columns] = imputer.fit_transform(
X[categorical_columns]
)
print("\nMissing values after imputation:")
print(X.isnull().sum())
Missing values after imputation:
Age 0
Gender 0
Cholesterol 0
Blood Pressure 0
Heart Rate 0
Smoking 0
Alcohol Intake 0
Exercise Hours 0
Family History 0
Diabetes 0
Obesity 0
Stress Level 0
Blood Sugar 0
Exercise Induced Angina 0
Chest Pain Type 0
dtype: int64
[7]: label_encoders = {}
for column in categorical_columns:
encoder = LabelEncoder()
X[column] = encoder.fit_transform(
X[column]
)
label_encoders[column] = encoder
print("\nData after Label Encoding:")
print(X.head())
Data after Label Encoding:
Age Gender Cholesterol Blood Pressure Heart Rate Smoking \
0 75 0 228 119 66 0
1 48 1 204 165 62 0
2 53 1 234 91 67 2
3 69 0 192 90 72 0
4 62 0 172 163 93 2
31°C
Mostly clear
Search
ENG
IN
98%
12:18 AM
10/9/2026



5. RESULT

Thus, the Logistic Regression practical was implemented successfully using the program from the corresponding Jupyter Notebook.

Exercise -7

K-Nearest Neighbour

1. AIM

To implement K-Nearest Neighbour classification for heart disease prediction, compare multiple K values and select the best-performing K.

2. ALGORITHM

1. Load the dataset and separate features and target.

2. Identify numerical and categorical features.

3. Split the dataset with stratification.

4. Scale numerical features and encode categorical features.

5. Train KNN for the specified K values.

6. Calculate accuracy, precision, recall and F1 score.

7. Select the best K and evaluate the final model.

3. PROGRAM

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder

from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report,
    roc_curve,
    roc_auc_score
)

df = pd.read_csv("heart_disease_dataset.csv")

print("Dataset Shape:", df.shape)

print("\nFirst 5 Records:")
```

```
print(df.head())

print("\nDataset Information:")
print(df.info())

print("\nMissing Values:")
print(df.isnull().sum())

print("\nTarget Distribution:")
print(df["Heart Disease"].value_counts())

X = df.drop("Heart Disease", axis=1)

y = df["Heart Disease"]

numerical_features = X.select_dtypes(
    include=["int64", "float64"]
).columns

categorical_features = X.select_dtypes(
    include=["object"]
).columns

print("Numerical Features:")
print(list(numerical_features))

print("\nCategorical Features:")
print(list(categorical_features))

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Training Data:", X_train.shape)
print("Testing Data :", X_test.shape)

preprocessor = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), numerical_features),
        ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_features)
    ]
)

X_train_processed = preprocessor.fit_transform(X_train)
X_test_processed = preprocessor.transform(X_test)

K = 5
knn = KNeighborsClassifier(
    n_neighbors=5
)

knn.fit(
    X_train_processed,
    y_train
```

```
)

y_pred = knn.predict(
    X_test_processed
)

print("Predicted Values:")
print(y_pred[:20])

print(
    classification_report(
        y_test,
        y_pred
    )
)

k_values = [1, 3, 5, 7, 9, 11, 15, 21]

accuracy_values = []
precision_values = []
recall_values = []
f1_values = []

for k in k_values:

    knn = KNeighborsClassifier(
        n_neighbors=k
    )

    knn.fit(
        X_train_processed,
        y_train
    )

    y_pred = knn.predict(
        X_test_processed
    )

    accuracy_values.append(
        accuracy_score(
            y_test,
            y_pred
        )
    )

    precision_values.append(
        precision_score(
            y_test,
            y_pred
        )
    )

    recall_values.append(
        recall_score(
            y_test,
            y_pred
        )
    )
```

```
    f1_values.append(
        f1_score(
            y_test,
            y_pred
        )
    )

results = pd.DataFrame({
    "K": k_values,
    "Accuracy": accuracy_values,
    "Precision": precision_values,
    "Recall": recall_values,
    "F1 Score": f1_values
})

print(results)

plt.figure(figsize=(8, 5))

plt.plot(
    k_values,
    accuracy_values,
    marker="o"
)

plt.xlabel("K Value")
plt.ylabel("Accuracy")

plt.title("Effect of K Value on KNN Accuracy")

plt.xticks(k_values)

plt.grid()

plt.show()

best_index = np.argmax(f1_values)

best_k = k_values[best_index]

print("Best K:", best_k)

print(
    "Best Accuracy:",
    accuracy_values[best_index]
)

print(
    "Best Precision:",
    precision_values[best_index]
)

print(
    "Best Recall:",
    recall_values[best_index]
)
```

```
print(
    "Best F1 Score:",
    f1_values[best_index]
)

final_knn = KNeighborsClassifier(
    n_neighbors=best_k
)

final_knn.fit(
    X_train_processed,
    y_train
)

final_pred = final_knn.predict(
    X_test_processed
)

final_accuracy = accuracy_score(
    y_test,
    final_pred
)

final_precision = precision_score(
    y_test,
    final_pred
)

final_recall = recall_score(
    y_test,
    final_pred
)

final_f1 = f1_score(
    y_test,
    final_pred
)

final_cm = confusion_matrix(
    y_test,
    final_pred
)

TN, FP, FN, TP = final_cm.ravel()

final_specificity = TN / (TN + FP)

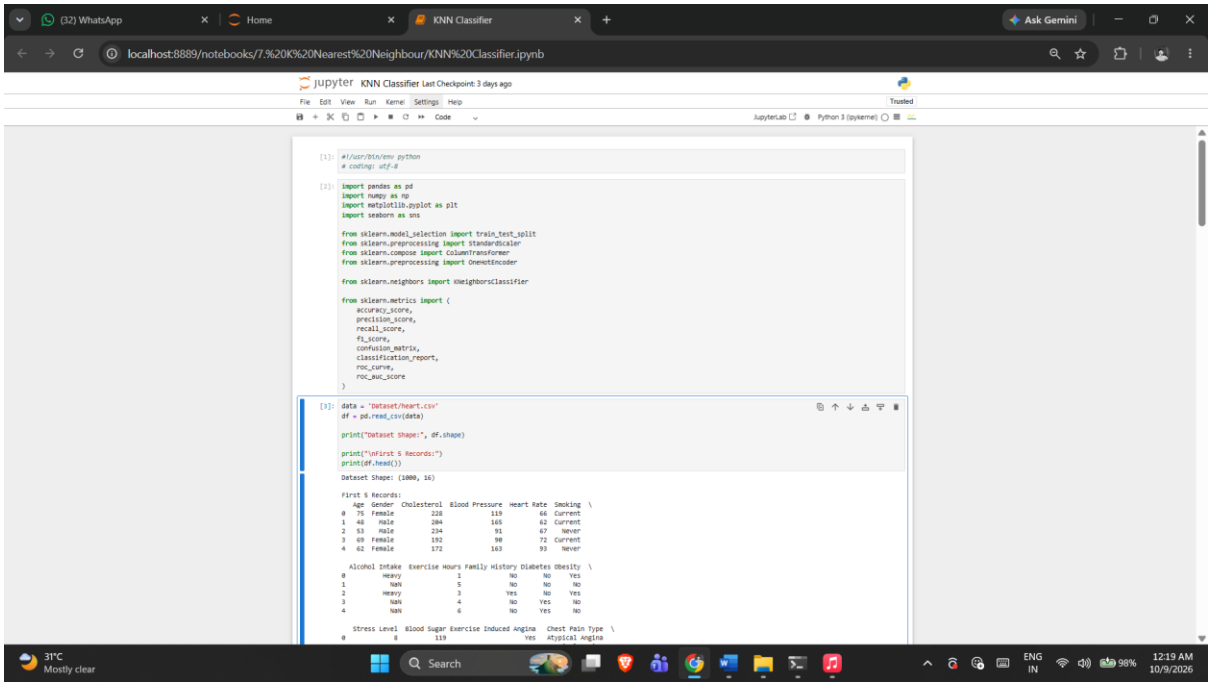
final_probability = final_knn.predict_proba(
    X_test_processed
)[:, 1]

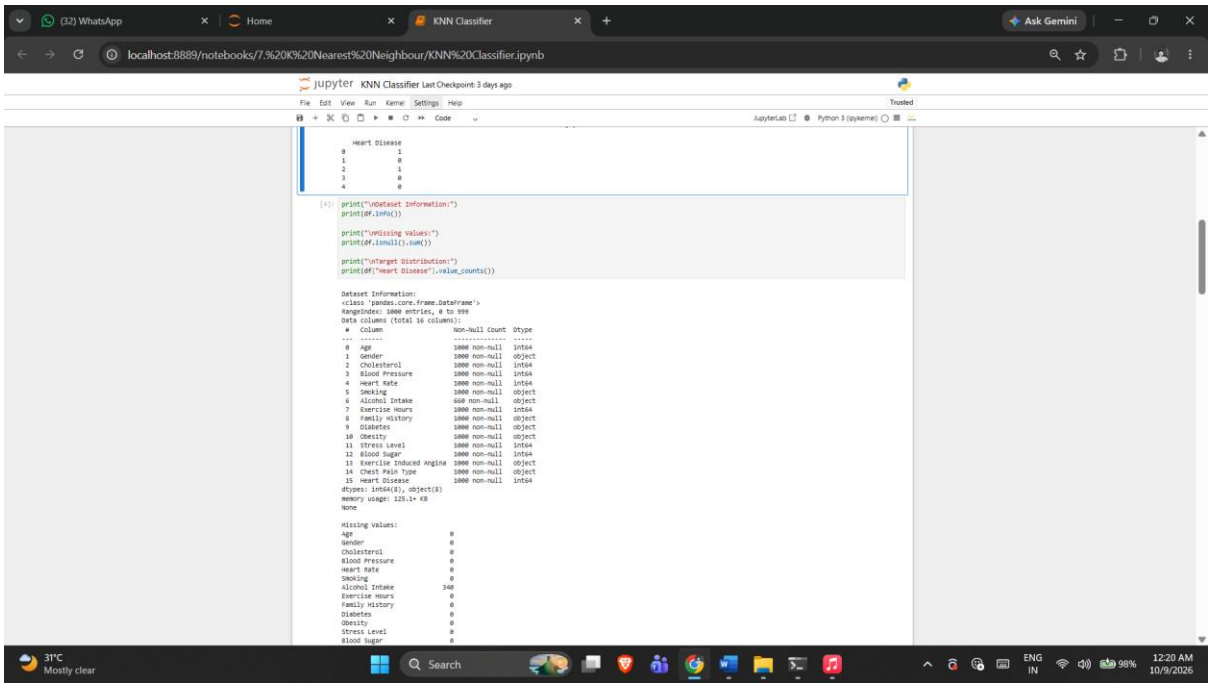
final_auc = roc_auc_score(
    y_test,
    final_probability
)

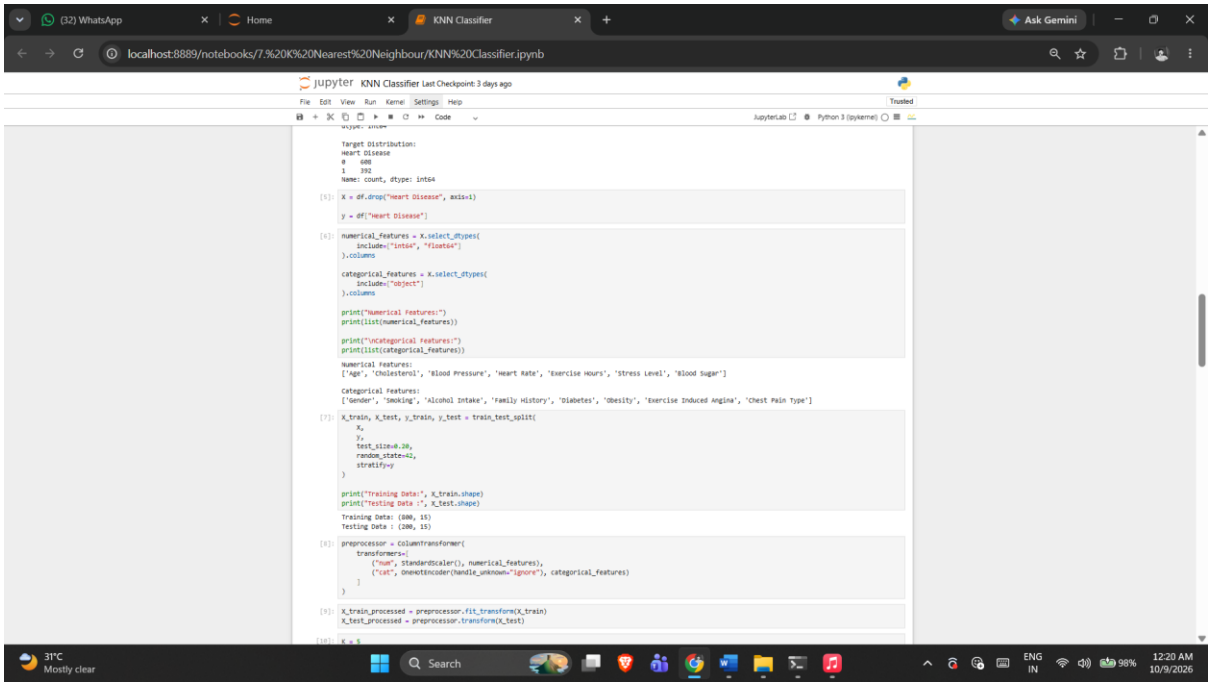
print("\n====================================")
print("FINAL KNN MODEL")
print("====================================")
```

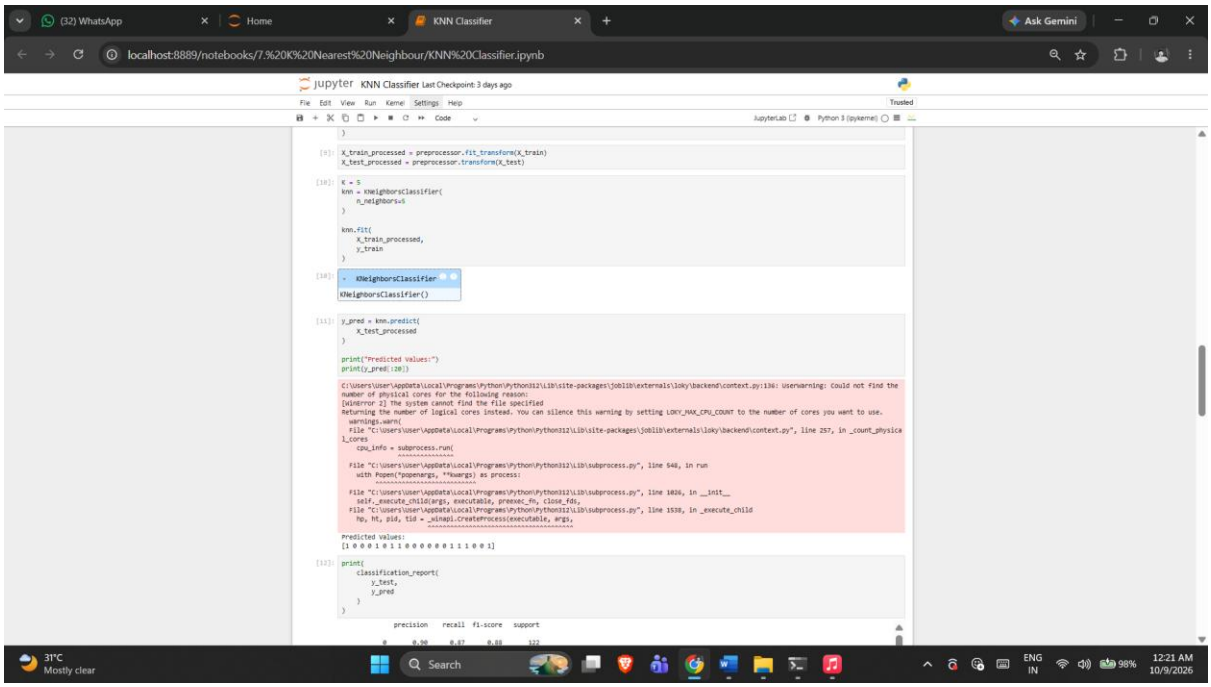
```
print("Best K       :", best_k)
print("Accuracy     :", final_accuracy)
print("Precision    :", final_precision)
print("Recall       :", final_recall)
print("F1 Score     :", final_f1)
print("Specificity  :", final_specificity)
print("AUC          :", final_auc)
```

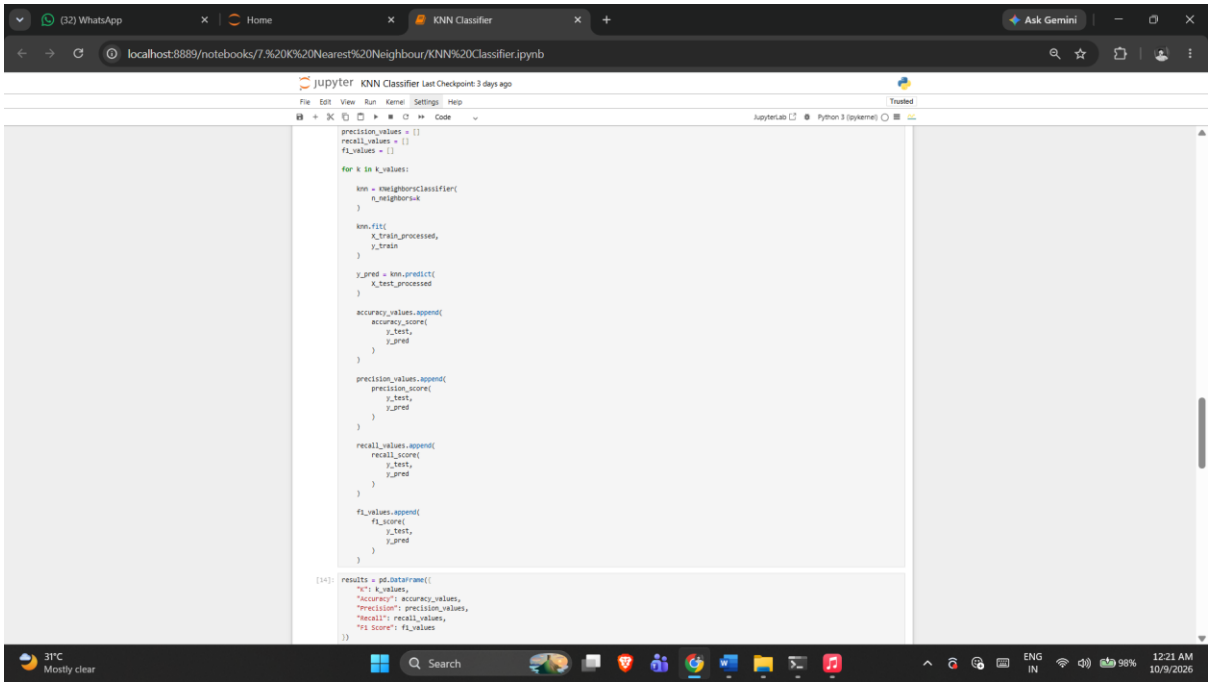
4. OUTPUT

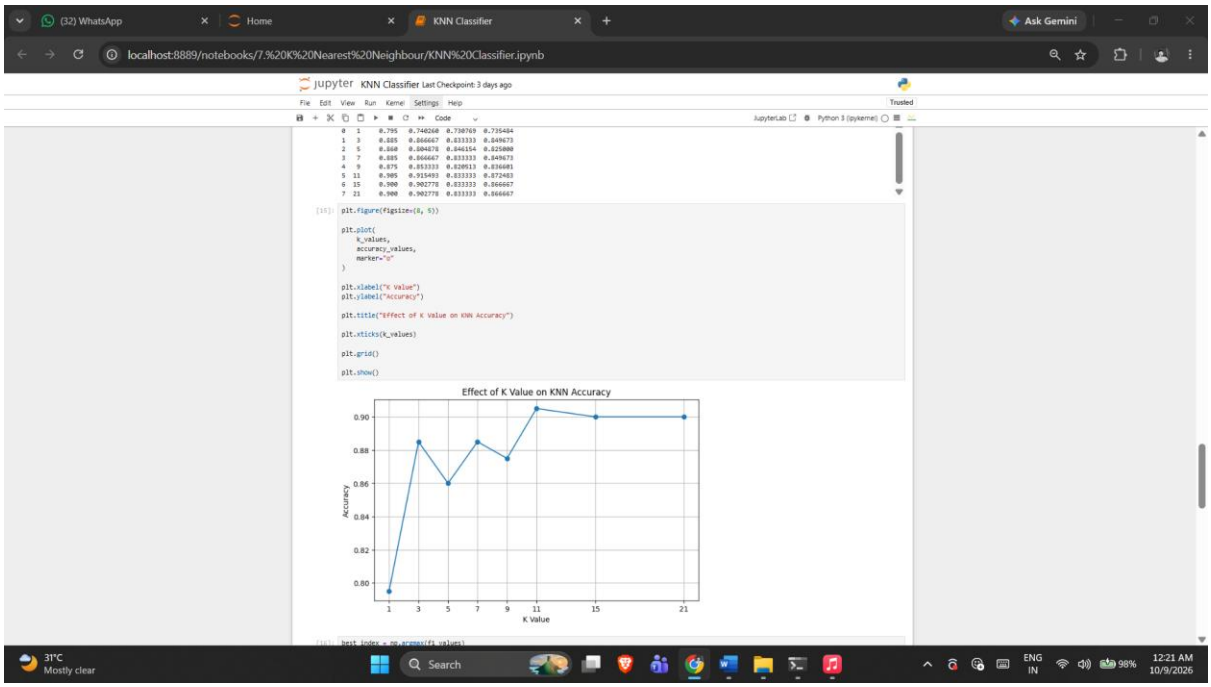





(32) WhatsApp
Home
KNN Classifier
Ask Gemini
localhost:8889/notebooks/7.%20K%20Nearest%20Neighbour/KNN%20Classifier.ipynb
jupyter KNN Classifier Last Checkpoint: 3 days ago
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
Target Distribution:
Heart Disease
0 600
1 392
Name: count, dtype: int64
[5]: X = df.drop("Heart Disease", axis=1)
y = df["Heart Disease"]
[6]: numerical_features = X.select_dtypes(
include=["int64", "float64"]
).columns
categorical_features = X.select_dtypes(
include="object"
).columns
print("Numerical Features:")
print(list(numerical_features))
print("\nCategorical Features:")
print(list(categorical_features))
Numerical Features:
['Age', 'Cholesterol', 'Blood Pressure', 'Heart Rate', 'Exercise Hours', 'Stress Level', 'Blood Sugar']
Categorical Features:
['Gender', 'Smoking', 'Alcohol Intake', 'Family History', 'Diabetes', 'Obesity', 'Exercise Induced Angina', 'Chest Pain Type']
[7]: X_train, X_test, y_train, y_test = train_test_split(
X,
y,
test_size=0.20,
random_state=42,
stratify=y
)
print("Training Data:', X_train.shape)
print("Testing Data :", X_test.shape)
Training Data: (800, 15)
Testing Data : (200, 15)
[8]: preprocessor = ColumnTransformer(
transformers=[
("num", StandardScaler(), numerical_features),
("cat", OneHotEncoder(handle_unknown="ignore"), categorical_features)
]
)
[9]: X_train_processed = preprocessor.fit_transform(X_train)
X_test_processed = preprocessor.transform(X_test)
31°C
Mostly clear
Search
ENG
IN
98%
12:20 AM
10/9/2026


(32) WhatsApp
Home
KNN Classifier
Ask Gemini
localhost:8889/notebooks/7.%20K%20Nearest%20Neighbour/KNN%20Classifier.ipynb
jupyter KNN Classifier Last Checkpoint: 3 days ago
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
[8]: X_train_processed = preprocessor.fit_transform(X_train)
X_test_processed = preprocessor.transform(X_test)
[10]: K = 5
knn = KNeighborsClassifier(
n_neighbors=5
)
knn.fit(
X_train_processed,
y_train
)
KNeighborsClassifier
KNeighborsClassifier()
[11]: y_pred = knn.predict(
X_test_processed
)
print("Predicted Values:")
print(y_pred[:20])
C:\Users\User\AppData\Local\Programs\Python\Python312\Lib\site-packages\joblib\externals\loky\backend\context.py:136: UserWarning: Could not find the number of physical cores for the following reason:
[WinError 2] The system cannot find the file specified
Returning the number of logical cores instead. You can silence this warning by setting LOKY_MAX_CPU_COUNT to the number of cores you want to use.
warnings.warn(
File "C:\Users\User\AppData\Local\Programs\Python\Python312\Lib\site-packages\joblib\externals\loky\backend\context.py", line 257, in _count_physical_cores
cpu_info = subprocess.run(
File "C:\Users\User\AppData\Local\Programs\Python\Python312\Lib\subprocess.py", line 548, in run
with Popen(*popenargs, **kwargs) as process:
File "C:\Users\User\AppData\Local\Programs\Python\Python312\Lib\subprocess.py", line 1026, in __init__
self._execute_child(args, executable, preexec_fn, close_fds,
File "C:\Users\User\AppData\Local\Programs\Python\Python312\Lib\subprocess.py", line 1538, in _execute_child
hp, ht, pid, tid = _winapi.CreateProcess(executable, args,
Predicted Values:
[1 0 0 0 1 0 1 1 0 0 0 0 0 0 1 1 1 0 0 1]
[12]: print(
classification_report(
y_test,
y_pred
)
)
precision recall f1-score support
0 0.90 0.87 0.88 122
31°C
Mostly clear
Search
ENG
IN
98%
12:21 AM
10/9/2026





5. RESULT

Thus, the K Nearest Neighbour practical was implemented successfully using the program from the corresponding Jupyter Notebook.

Exercise -8

Decision Tree Classification

1. AIM

To build a Decision Tree classifier for heart disease prediction and evaluate and visualize the trained model.

2. ALGORITHM

1. Load the dataset and separate features and target.

2. Identify numerical and categorical features.

3. Build the preprocessing pipeline.

4. Split the data into training and testing sets.

5. Train the Decision Tree classifier.

6. Predict and calculate the notebook's evaluation metrics.

7. Visualize the trained tree.

3. PROGRAM

```
#!/usr/bin/env python
# coding: utf-8

import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.pipeline import Pipeline

from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import plot_tree

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report,
    ConfusionMatrixDisplay
)

df = pd.read_csv("heart_disease_dataset.csv")

print("First 5 rows:")
print(df.head())
```

```
print("\nDataset Shape:")
print(df.shape)

X = df.drop("Heart Disease", axis=1)
y = df["Heart Disease"]

print("\nFeatures:")
print(X.columns)

print("\nTarget:")
print(y.name)

categorical_columns = X.select_dtypes(
    include=["object"]
).columns

numerical_columns = X.select_dtypes(
    exclude=["object"]
).columns

print("\nCategorical Columns:")
print(list(categorical_columns))

print("\nNumerical Columns:")
print(list(numerical_columns))

preprocessor = ColumnTransformer(
    transformers=[
        (
            "cat",
            OneHotEncoder(handle_unknown="ignore"),
            categorical_columns
        )
    ],
    remainder="passthrough"
)

decision_tree = DecisionTreeClassifier(
    criterion="gini",
    max_depth=4,
    random_state=42
)

model = Pipeline(
    steps=[
        ("preprocessor", preprocessor),
        ("classifier", decision_tree)
    ]
)

model = Pipeline(
    steps=[
        ("preprocessor", preprocessor),
        ("classifier", decision_tree)
    ]
)

X_train, X_test, y_train, y_test = train_test_split(
    X,
```

```
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("\nTraining Data:", X_train.shape)
print("Testing Data:", X_test.shape)

model.fit(X_train, y_train)

print("\nDecision Tree Model Trained Successfully!")

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

precision = precision_score(
    y_test,
    y_pred,
    zero_division=0
)

recall = recall_score(
    y_test,
    y_pred,
    zero_division=0
)

f1 = f1_score(
    y_test,
    y_pred,
    zero_division=0
)



print("\n========================================")
print("       DECISION TREE EVALUATION")
print("========================================")

print("Accuracy  :", round(accuracy, 4))
print("Precision :", round(precision, 4))
print("Recall    :", round(recall, 4))
print("F1 Score  :", round(f1, 4))

# ========================================
# 12. Classification Report
# ========================================

print("\nClassification Report:")
print(classification_report(y_test, y_pred))



# ========================================
# 13. Confusion Matrix
# ========================================
```

```
cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix:")
print(cm)

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["No Heart Disease", "Heart Disease"]
)

disp.plot()

plt.title("Confusion Matrix - Decision Tree")
plt.show()

# Get the trained classifier
tree_model = model.named_steps["classifier"]

# Get feature names after One-Hot Encoding
feature_names = model.named_steps[
    "preprocessor"
].get_feature_names_out()

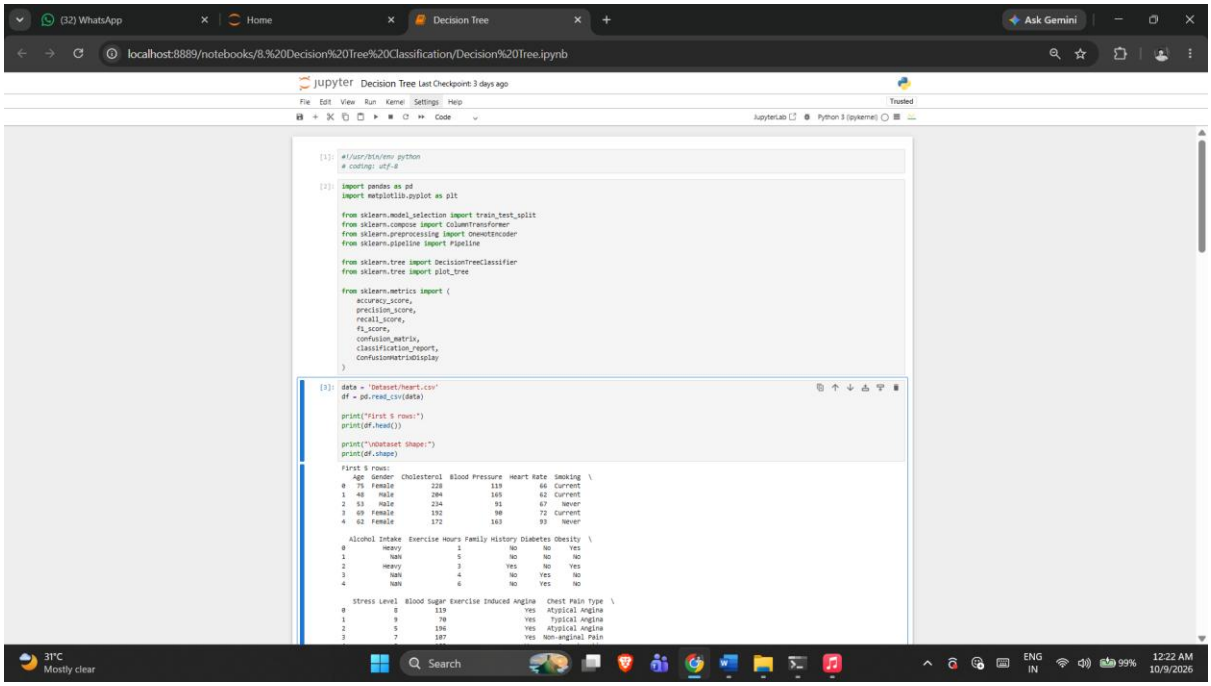
plt.figure(figsize=(25, 12))

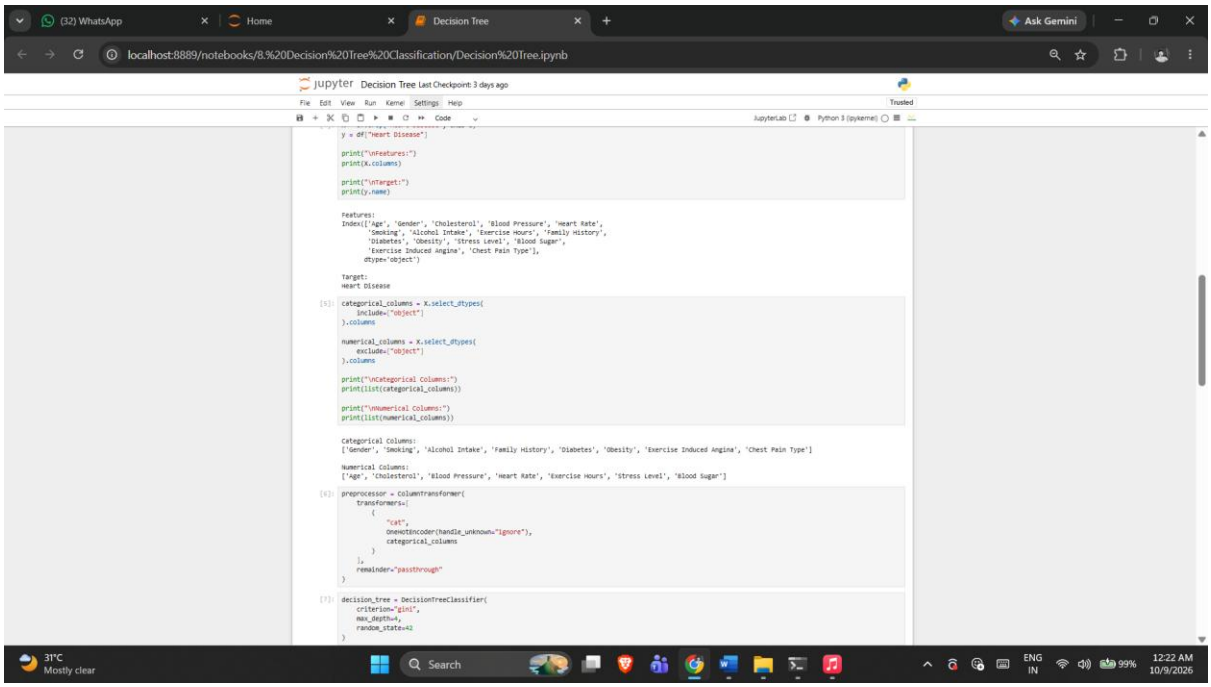
plot_tree(
    tree_model,
    feature_names=feature_names,
    class_names=["No Heart Disease", "Heart Disease"],
    filled=True,
    rounded=True,
    fontsize=8
)

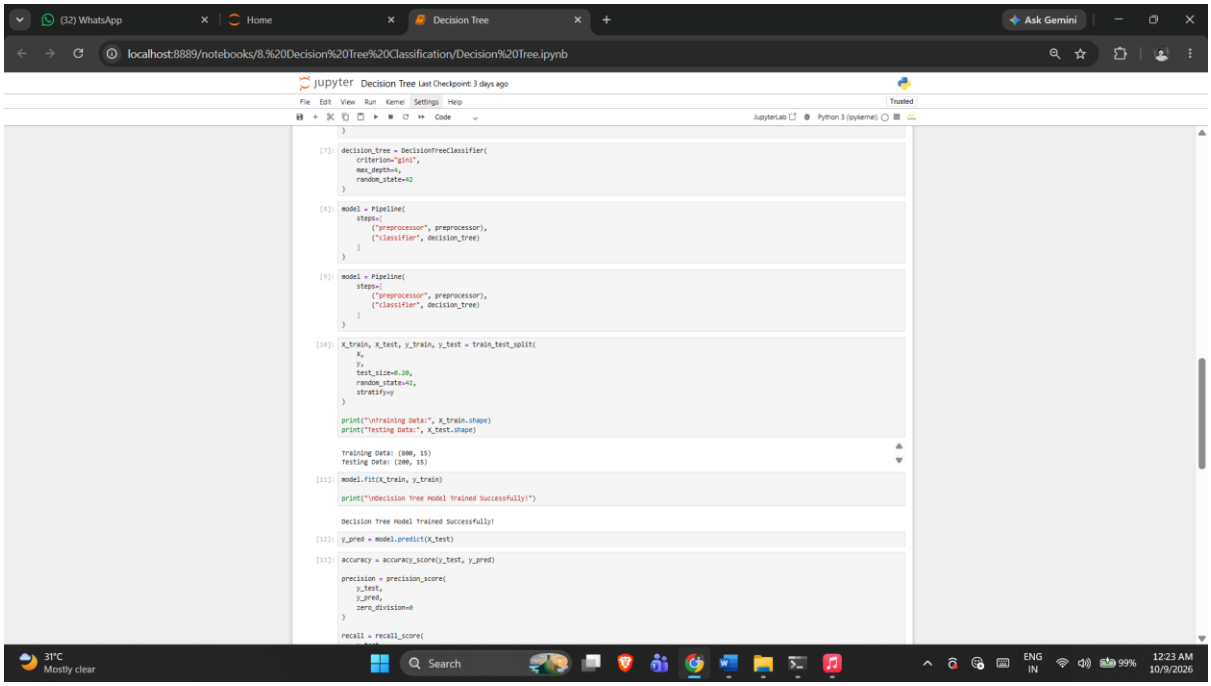
plt.title("Decision Tree for Heart Disease Classification")

plt.show()
```

4. OUTPUT


(32) WhatsApp
Home
Decision Tree
Ask Gemini
localhost:8889/notebooks/8.%20Decision%20Tree%20Classification/Decision%20Tree.ipynb
jupyter Decision Tree Last Checkpoint: 3 days ago
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
[1]:
#!/usr/bin/env python
coding: utf-8
[2]:
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import plot_tree
from sklearn.metrics import (
accuracy_score,
precision_score,
recall_score,
f1_score,
confusion_matrix,
classification_report,
ConfusionMatrixDisplay
)
[3]:
data = 'Dataset/heart.csv'
df = pd.read_csv(data)
print("First 5 rows:")
print(df.head())
print("\nDataset Shape:")
print(df.shape)
First 5 rows:
Age Gender Cholesterol Blood Pressure Heart Rate Smoking \
0 75 Female 228 119 66 Current
1 48 Male 204 165 62 Current
2 53 Male 234 91 67 Never
3 69 Female 192 90 72 Current
4 62 Female 172 163 93 Never
Alcohol Intake Exercise Hours Family History Diabetes Obesity \
0 Heavy 1 No No Yes
1 NaN 5 No No No
2 Heavy 3 Yes No Yes
3 NaN 4 No Yes No
4 NaN 6 No Yes No
Stress Level Blood Sugar Exercise Induced Angina Chest Pain Type \
0 8 119 Yes Atypical Angina
1 9 70 Yes Typical Angina
2 5 196 Yes Atypical Angina
3 7 107 Yes Non-anginal Pain
31°C
Mostly clear
Search
ENG
IN
99%
12:22 AM
10/9/2026


(32) WhatsApp
Home
Decision Tree
Ask Gemini
localhost:8889/notebooks/8.%20Decision%20Tree%20Classification/Decision%20Tree.ipynb
jupyter Decision Tree Last Checkpoint: 3 days ago
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
y = df["Heart Disease"]
print("\nFeatures:")
print(X.columns)
print("\nTarget:")
print(y.name)
Features:
Index(['Age', 'Gender', 'Cholesterol', 'Blood Pressure', 'Heart Rate',
'Smoking', 'Alcohol Intake', 'Exercise Hours', 'Family History',
'Diabetes', 'Obesity', 'Stress Level', 'Blood Sugar',
'Exercise Induced Angina', 'Chest Pain Type'],
dtype='object')
Target:
Heart Disease
[5]:
categorical_columns = X.select_dtypes(
include="object"
).columns
numerical_columns = X.select_dtypes(
exclude="object"
).columns
print("\nCategorical Columns:")
print(list(categorical_columns))
print("\nNumerical Columns:")
print(list(numerical_columns))
Categorical Columns:
['Gender', 'Smoking', 'Alcohol Intake', 'Family History', 'Diabetes', 'Obesity', 'Exercise Induced Angina', 'Chest Pain Type']
Numerical Columns:
['Age', 'Cholesterol', 'Blood Pressure', 'Heart Rate', 'Exercise Hours', 'Stress Level', 'Blood Sugar']
[6]:
preprocessor = ColumnTransformer(
transformers=[
(
"cat",
OneHotEncoder(handle_unknown="ignore"),
categorical_columns
)
],
remainder="passthrough"
)
[7]:
decision_tree = DecisionTreeClassifier(
criterion="gini",
max_depth=4,
random_state=42
)
31°C
Mostly clear
Search
ENG
IN
99%
12:22 AM
10/9/2026



```
)

decision_tree = DecisionTreeClassifier(
    criterion="gini",
    max_depth=4,
    random_state=42
)

model = Pipeline(
    steps=[
        ("preprocessor", preprocessor),
        ("classifier", decision_tree)
    ]
)

model = Pipeline(
    steps=[
        ("preprocessor", preprocessor),
        ("classifier", decision_tree)
    ]
)

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("\nTraining Data:", X_train.shape)
print("Testing Data:", X_test.shape)
```

```
Training Data: (800, 15)
Testing Data: (200, 15)
```

```
model.fit(X_train, y_train)

print("\nDecision Tree Model Trained Successfully!")
```

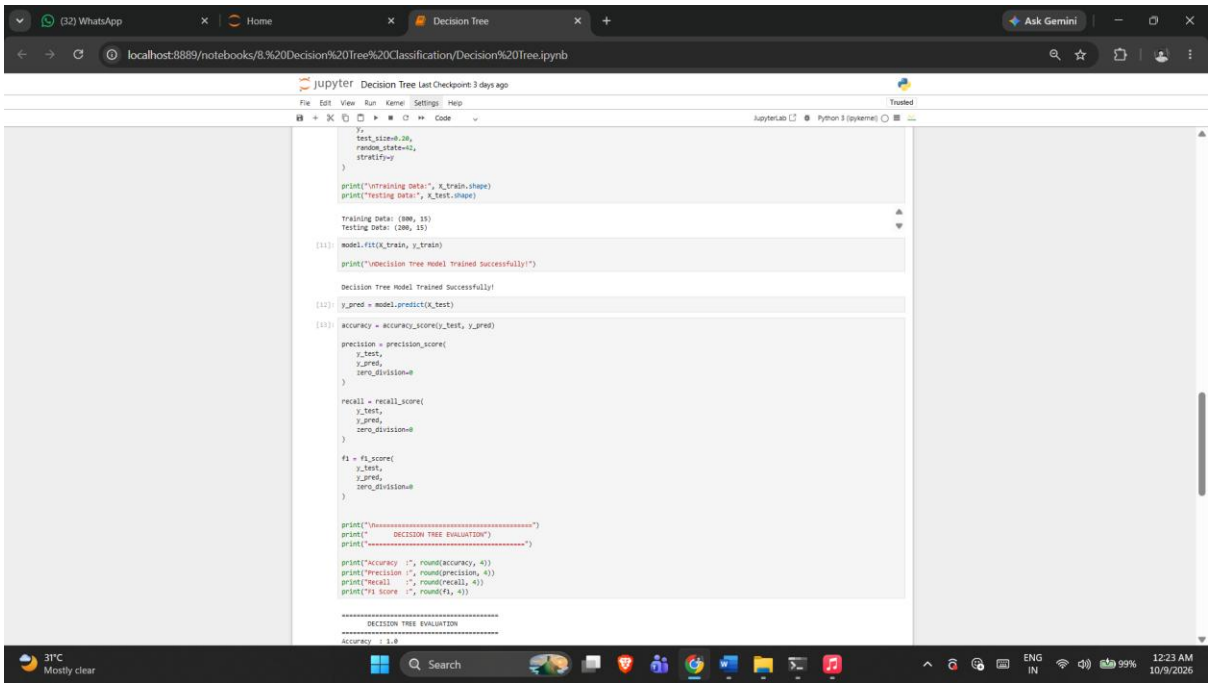
```
Decision Tree Model Trained Successfully!
```

```
y_pred = model.predict(X_test)
```

```
accuracy = accuracy_score(y_test, y_pred)

precision = precision_score(
    y_test,
    y_pred,
    zero_division=0
)

recall = recall_score(
```



```
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("\nTraining Data:", X_train.shape)
print("Testing Data:", X_test.shape)
```

```
Training Data: (800, 15)
Testing Data: (200, 15)
```

```
model.fit(X_train, y_train)

print("\nDecision Tree Model Trained Successfully!")
```

```
Decision Tree Model Trained Successfully!
```

```
y_pred = model.predict(X_test)
```

```
accuracy = accuracy_score(y_test, y_pred)

precision = precision_score(
    y_test,
    y_pred,
    zero_division=0
)

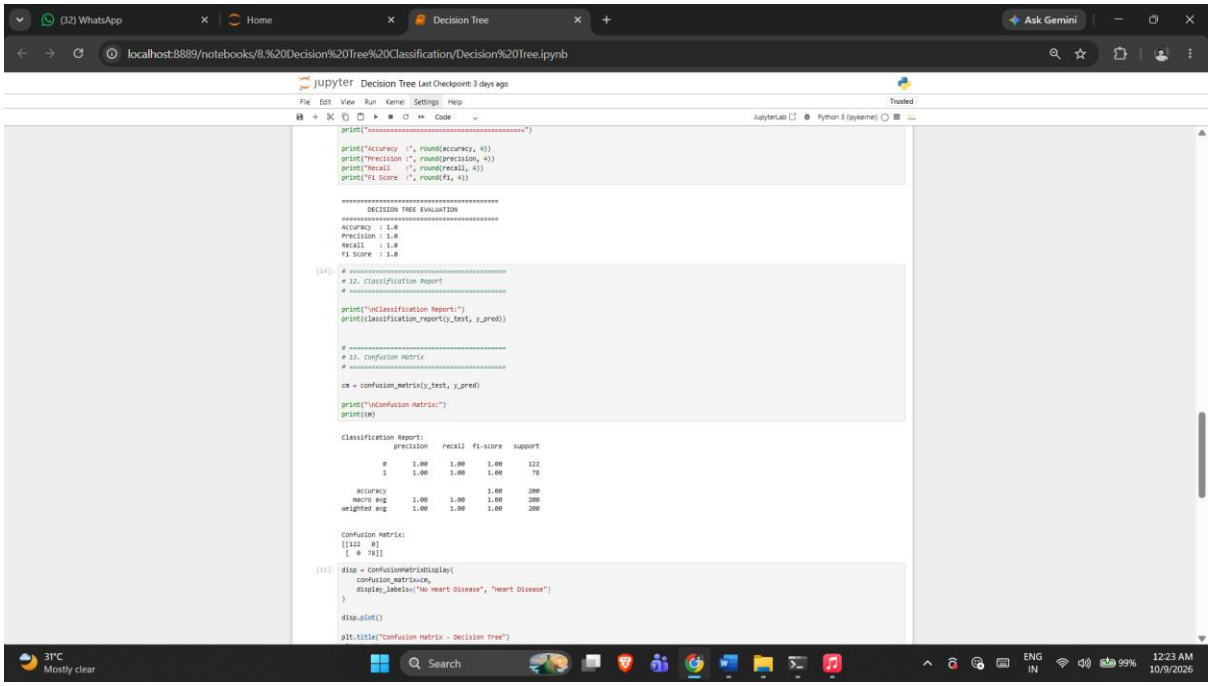
recall = recall_score(
    y_test,
    y_pred,
    zero_division=0
)

f1 = f1_score(
    y_test,
    y_pred,
    zero_division=0
)


print("\n==========================================")
print("      DECISION TREE EVALUATION")
print("==========================================")

print("Accuracy  :", round(accuracy, 4))
print("Precision :", round(precision, 4))
print("Recall    :", round(recall, 4))
print("F1 Score  :", round(f1, 4))
```

```
==========================================
      DECISION TREE EVALUATION
==========================================
Accuracy  : 1.0
```



```
print("========================================")

print("Accuracy  :", round(accuracy, 4))
print("Precision :", round(precision, 4))
print("Recall    :", round(recall, 4))
print("F1 Score  :", round(f1, 4))
```

```
========================================
      DECISION TREE EVALUATION
========================================
Accuracy  : 1.0
Precision : 1.0
Recall    : 1.0
F1 Score  : 1.0
```

```
# ========================================
# 12. Classification Report
# ========================================

print("\nClassification Report:")
print(classification_report(y_test, y_pred))


# ========================================
# 13. Confusion Matrix
# ========================================

cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix:")
print(cm)
```

```
Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00       122
           1       1.00      1.00      1.00        78

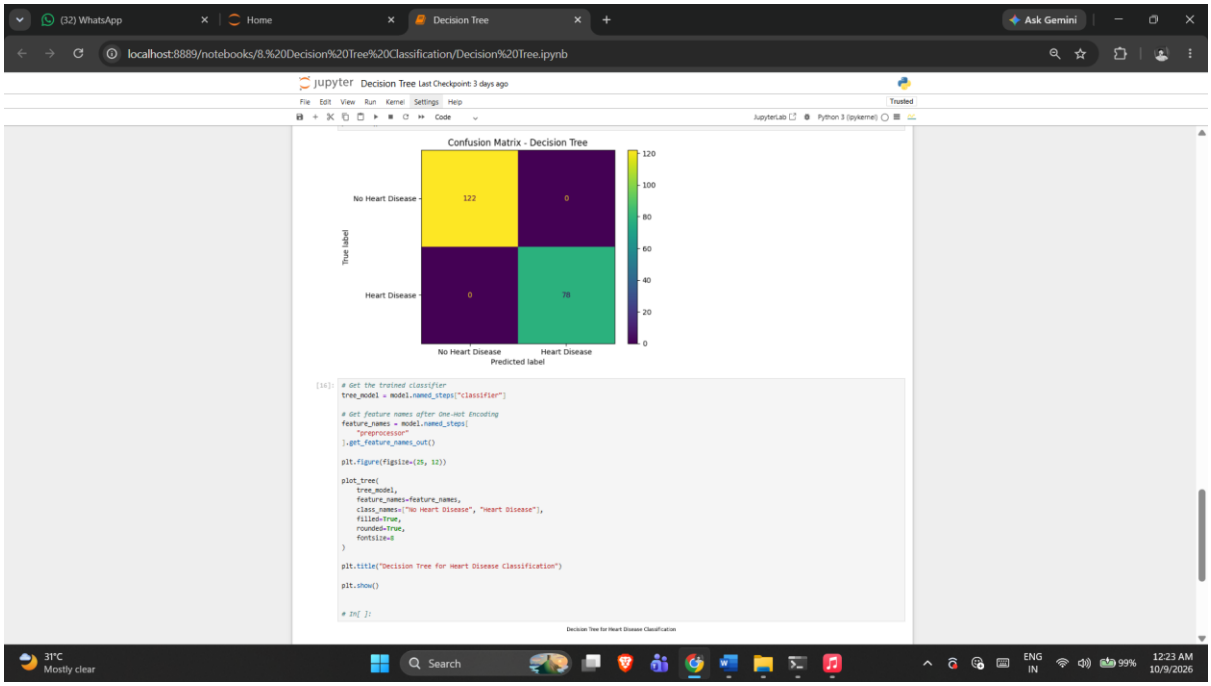
    accuracy                           1.00       200
   macro avg       1.00      1.00      1.00       200
weighted avg       1.00      1.00      1.00       200


Confusion Matrix:
[[122   0]
 [  0  78]]
```

```
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["No Heart Disease", "Heart Disease"]
)

disp.plot()

plt.title("Confusion Matrix - Decision Tree")
```




```
# Get the trained classifier
tree_model = model.named_steps["classifier"]

# Get feature names after One-Hot Encoding
feature_names = model.named_steps[
    "preprocessor"
].get_feature_names_out()

plt.figure(figsize=(25, 12))

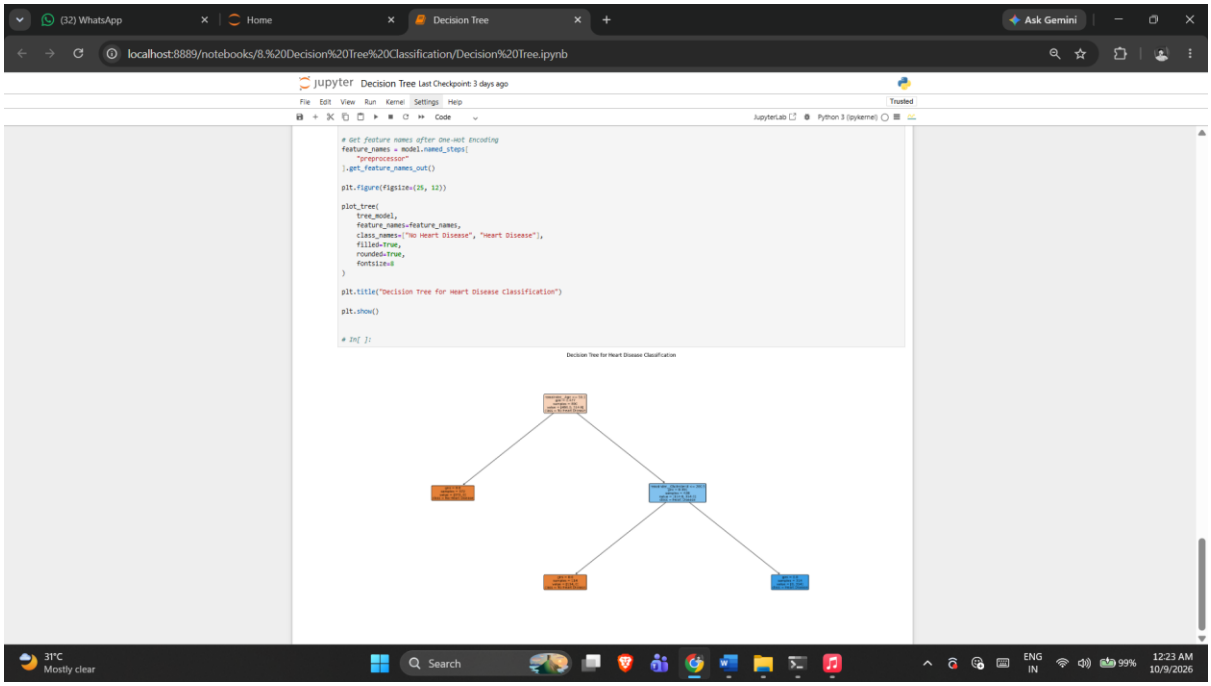
plot_tree(
    tree_model,
    feature_names=feature_names,
    class_names=["No Heart Disease", "Heart Disease"],
    filled=True,
    rounded=True,
    fontsize=8
)

plt.title("Decision Tree for Heart Disease Classification")

plt.show()


# In[ ]:
```

Decision Tree for Heart Disease Classification



5. RESULT

Thus, the Decision Tree Classifier practical was implemented successfully using the program from the corresponding Jupyter Notebook.

Exercise -9

Random Forest Classification

1. AIM

To build and evaluate a Random Forest classifier for heart disease prediction, including ROC-AUC analysis.

2. ALGORITHM

1. Load and inspect the dataset.

2. Check structure, data types, missing values and target distribution.

3. Separate features and target.

4. Prepare categorical preprocessing.

5. Create the Random Forest classifier with notebook parameters.

6. Split, train and predict.

7. Calculate classification metrics.

8. Generate ROC curve and ROC-AUC.

3. PROGRAM

```
#!/usr/bin/env python
# coding: utf-8


# Import required libraries


import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns


# Machine Learning libraries
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier


# Evaluation metrics
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report,
    confusion_matrix,
    ConfusionMatrixDisplay,
    roc_curve,
    roc_auc_score
```

```
)

# Load the dataset

df = pd.read_csv("heart_disease_dataset.csv")

# Display first 5 rows
df.head()

# Display number of rows and columns

print("Dataset Shape:", df.shape)

# Display column names

print(df.columns)

# Display data types

df.info()

# Check missing values

print(df.isnull().sum())

# Check target class distribution

print(df["Heart Disease"].value_counts())

# X contains input features
# y contains the target variable

X = df.drop("Heart Disease", axis=1)

y = df["Heart Disease"]

print("Features Shape:", X.shape)
print("Target Shape:", y.shape)

# Identify numerical columns

numerical_features = X.select_dtypes(
    include=["int64", "float64"]
).columns

# Identify categorical columns

categorical_features = X.select_dtypes(
    include=["object"]
).columns

print("Numerical Features:")
print(list(numerical_features))

print("\nCategorical Features:")
```

```
print(list(categorical_features))

# Preprocessing for numerical and categorical features

preprocessor = ColumnTransformer(
    transformers=[
        ("num", "passthrough", numerical_features),

        ("cat", OneHotEncoder(handle_unknown="ignore"),
         categorical_features)
    ]
)

# Create Random Forest classifier

rf_model = RandomForestClassifier(
    n_estimators=100,
    criterion="gini",
    max_depth=None,
    min_samples_split=2,
    min_samples_leaf=1,
    max_features="sqrt",
    random_state=42,
    n_jobs=-1
)

# Combine preprocessing and Random Forest model

model = Pipeline(
    steps=[
        ("preprocessor", preprocessor),
        ("classifier", rf_model)
    ]
)

# Split dataset into training and testing sets

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("Training Data:", X_train.shape)
print("Testing Data:", X_test.shape)

# Train the model

model.fit(X_train, y_train)

print("Random Forest Model Trained Successfully!")

# Predict on training data

y_train_pred = model.predict(X_train)
```

```
# Predict on testing data

y_test_pred = model.predict(X_test)

print("Predictions Generated Successfully!")

# Calculate evaluation metrics

train_accuracy = accuracy_score(y_train, y_train_pred)

test_accuracy = accuracy_score(y_test, y_test_pred)

precision = precision_score(y_test, y_test_pred)

recall = recall_score(y_test, y_test_pred)

f1 = f1_score(y_test, y_test_pred)

print("Training Accuracy :", round(train_accuracy, 4))
print("Testing Accuracy  :", round(test_accuracy, 4))
print("Precision         :", round(precision, 4))
print("Recall            :", round(recall, 4))
print("F1 Score          :", round(f1, 4))

# Predict probabilities

y_test_proba = model.predict_proba(X_test)[:, 1]

# Calculate ROC curve

fpr, tpr, thresholds = roc_curve(y_test, y_test_proba)

# Calculate AUC

auc_score = roc_auc_score(y_test, y_test_proba)

print("ROC-AUC Score:", round(auc_score, 4))

# Plot ROC Curve

plt.figure(figsize=(8, 6))

plt.plot(
    fpr,
    tpr,
    label=f"Random Forest (AUC = {auc_score:.2f})"
)

plt.plot(
    [0, 1],
    [0, 1],
    linestyle="--",
    label="Random Classifier"
)
```

```
plt.xlabel("False Positive Rate")

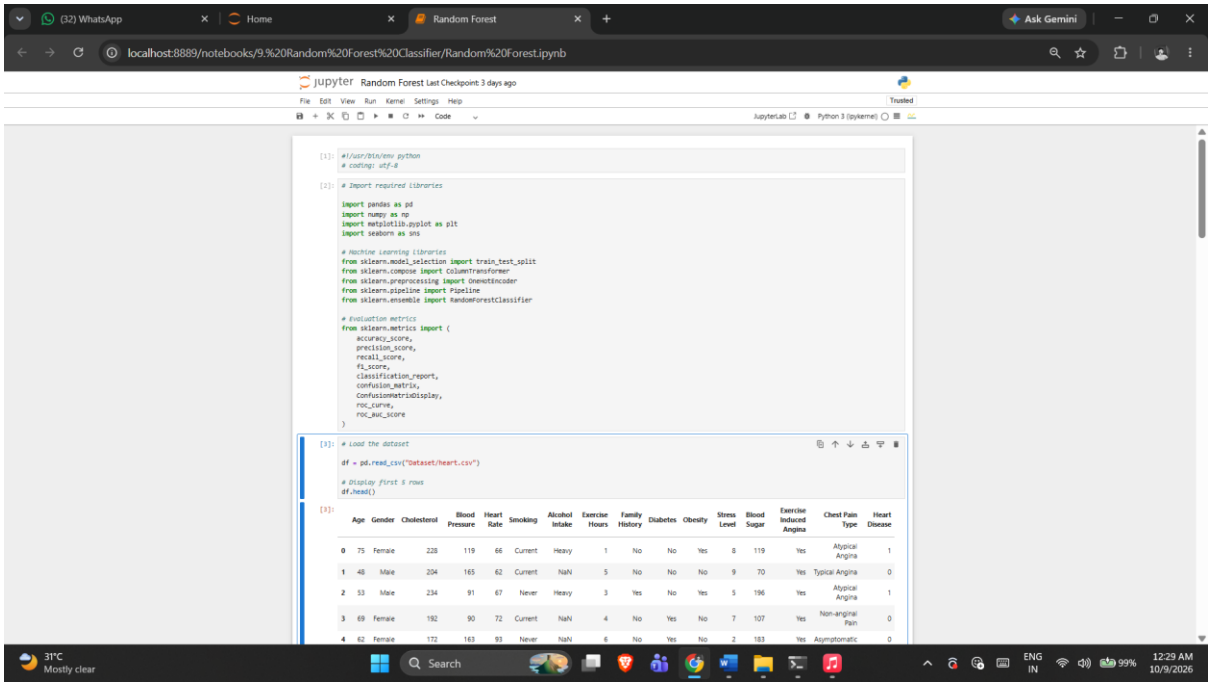
plt.ylabel("True Positive Rate")

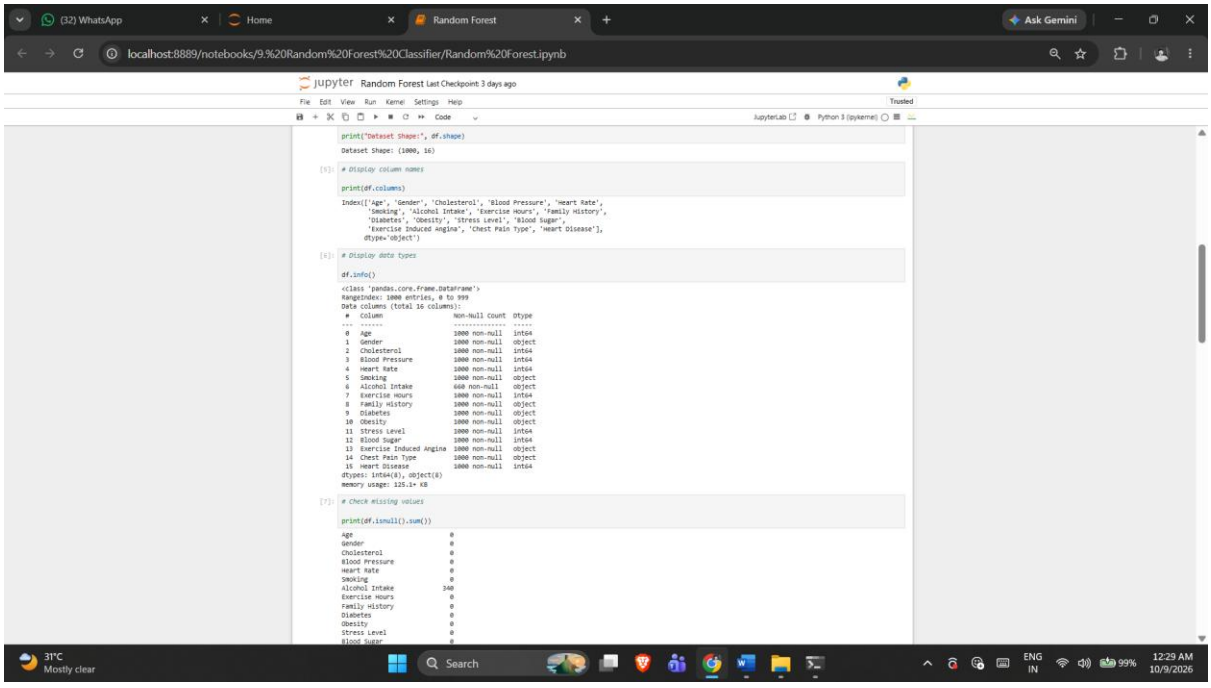
plt.title("ROC Curve - Random Forest")

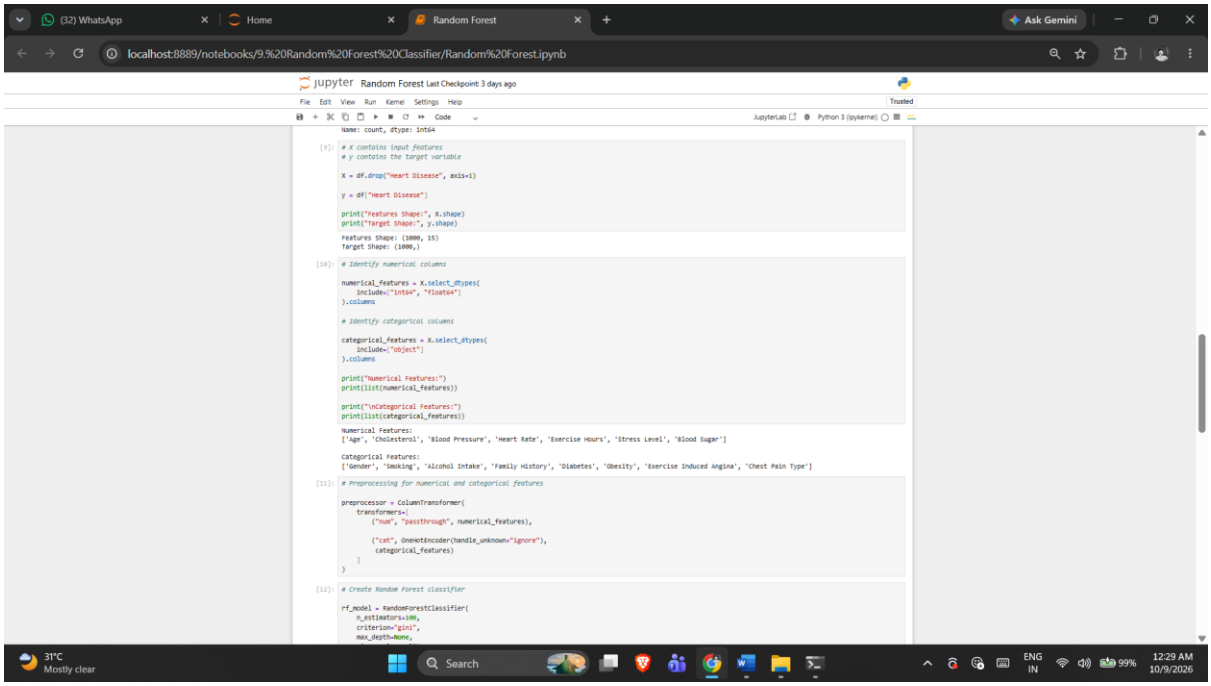
plt.legend()

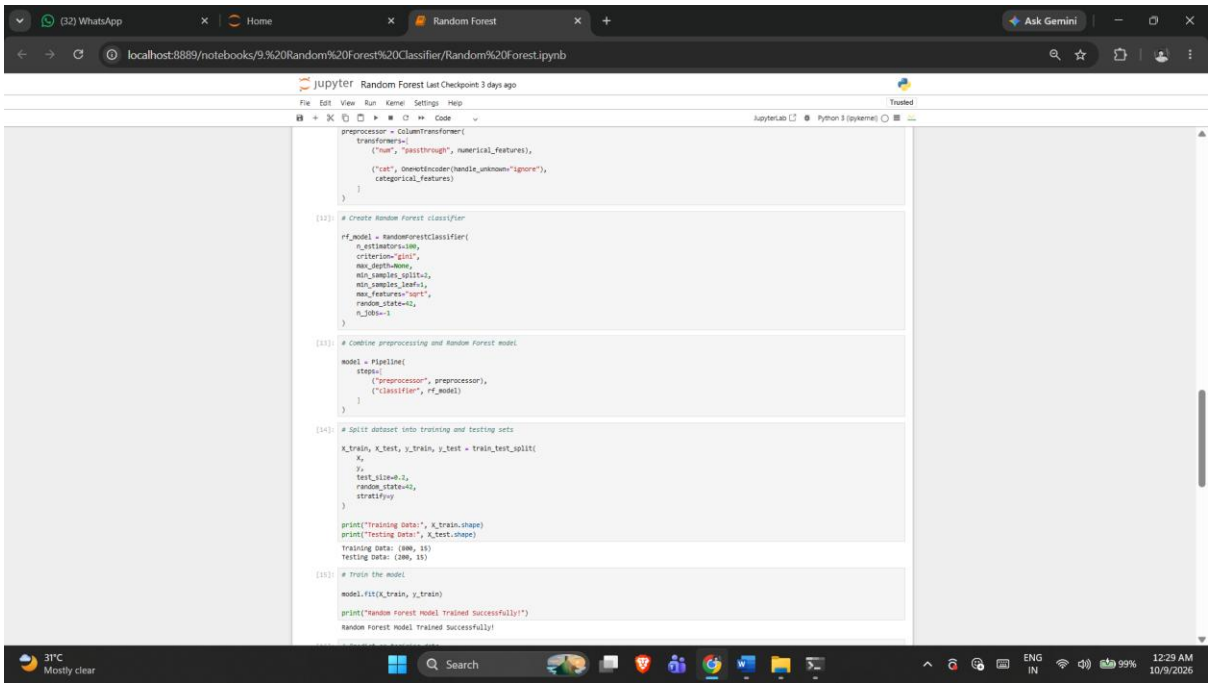
plt.show()
```

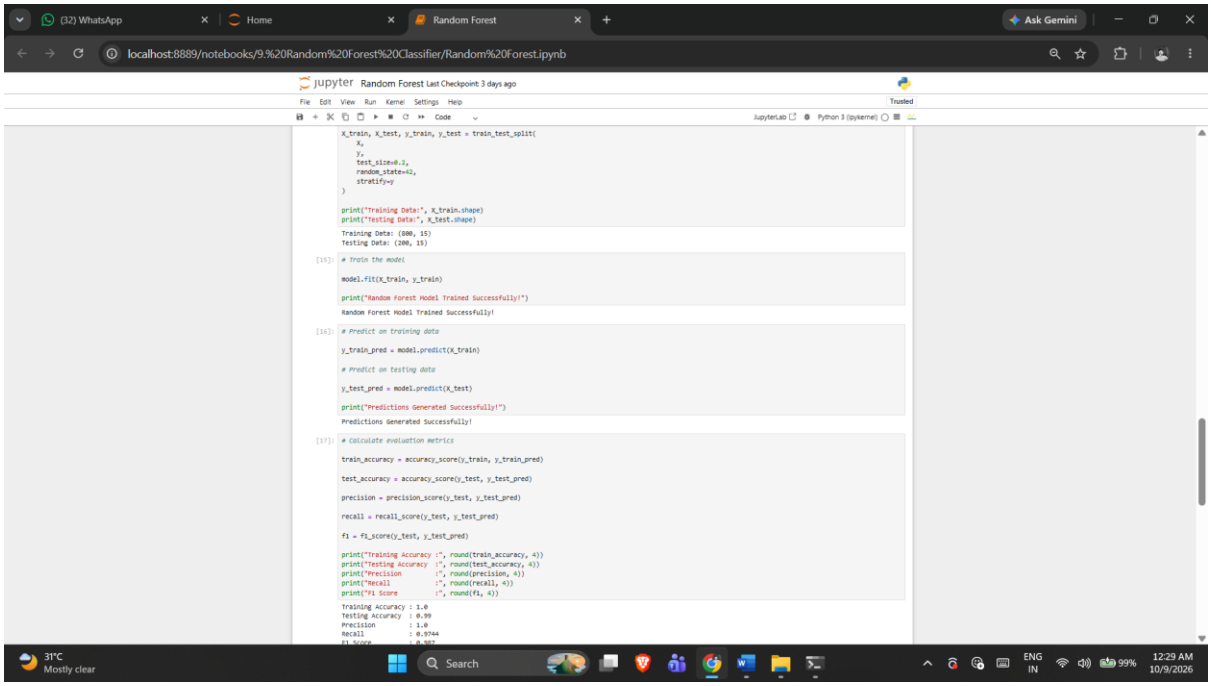
4. OUTPUT






(32) WhatsApp
Home
Random Forest
Ask Gemini
localhost:8889/notebooks/9.%20Random%20Forest%20Classifier/Random%20Forest.ipynb
jupyter Random Forest Last Checkpoint: 3 days ago
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
Name: count, dtype: int64
[9]:
X contains input features
y contains the target variable
X = df.drop("Heart Disease", axis=1)
y = df["Heart Disease"]
print("Features Shape:", X.shape)
print("Target Shape:", y.shape)
Features Shape: (1000, 15)
Target Shape: (1000,)
[10]:
Identify numerical columns
numerical_features = X.select_dtypes(
include=["int64", "float64"]
).columns
Identify categorical columns
categorical_features = X.select_dtypes(
include=["object"]
).columns
print("Numerical Features:")
print(list(numerical_features))
print("\nCategorical Features:")
print(list(categorical_features))
Numerical Features:
['Age', 'Cholesterol', 'Blood Pressure', 'Heart Rate', 'Exercise Hours', 'Stress Level', 'Blood Sugar']
Categorical Features:
['Gender', 'Smoking', 'Alcohol Intake', 'Family History', 'Diabetes', 'Obesity', 'Exercise Induced Angina', 'Chest Pain Type']
[11]:
Preprocessing for numerical and categorical features
preprocessor = ColumnTransformer(
transformers=[
("num", "passthrough", numerical_features),
("cat", OneHotEncoder(handle_unknown="ignore"),
categorical_features)
]
)
[12]:
Create Random Forest classifier
rf_model = RandomForestClassifier(
n_estimators=100,
criterion="gini",
max_depth=None,
31°C
Mostly clear
Search
ENG
IN
99%
12:29 AM
10/9/2026


(32) WhatsApp
Home
Random Forest
Ask Gemini
localhost:8889/notebooks/9.%20Random%20Forest%20Classifier/Random%20Forest.ipynb
jupyter Random Forest Last Checkpoint: 3 days ago
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
preprocessor = ColumnTransformer(
transformers=[
("num", "passthrough", numerical_features),
("cat", OneHotEncoder(handle_unknown="ignore"),
categorical_features)
]
)
[12]:
Create Random Forest classifier
rf_model = RandomForestClassifier(
n_estimators=100,
criterion="gini",
max_depth=None,
min_samples_split=2,
min_samples_leaf=1,
max_features="sqrt",
random_state=42,
n_jobs=-1
)
[13]:
Combine preprocessing and Random Forest model
model = Pipeline(
steps=[
("preprocessor", preprocessor),
("classifier", rf_model)
]
)
[14]:
Split dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
X,
y,
test_size=0.2,
random_state=42,
stratify=y
)
print("Training Data:", X_train.shape)
print("Testing Data:", X_test.shape)
Training Data: (800, 15)
Testing Data: (200, 15)
[15]:
Train the model
model.fit(X_train, y_train)
print("Random Forest Model Trained Successfully!")
Random Forest Model Trained Successfully!
31°C
Mostly clear
Search
ENG
IN
99%
12:29 AM
10/9/2026



```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("Training Data:", X_train.shape)
print("Testing Data:", X_test.shape)
```

```
Training Data: (800, 15)
Testing Data: (200, 15)
```

[15]:
```
# Train the model

model.fit(X_train, y_train)

print("Random Forest Model Trained Successfully!")
```

```
Random Forest Model Trained Successfully!
```

[16]:
```
# Predict on training data

y_train_pred = model.predict(X_train)

# Predict on testing data

y_test_pred = model.predict(X_test)

print("Predictions Generated Successfully!")
```

```
Predictions Generated Successfully!
```

[17]:
```
# Calculate evaluation metrics

train_accuracy = accuracy_score(y_train, y_train_pred)

test_accuracy = accuracy_score(y_test, y_test_pred)

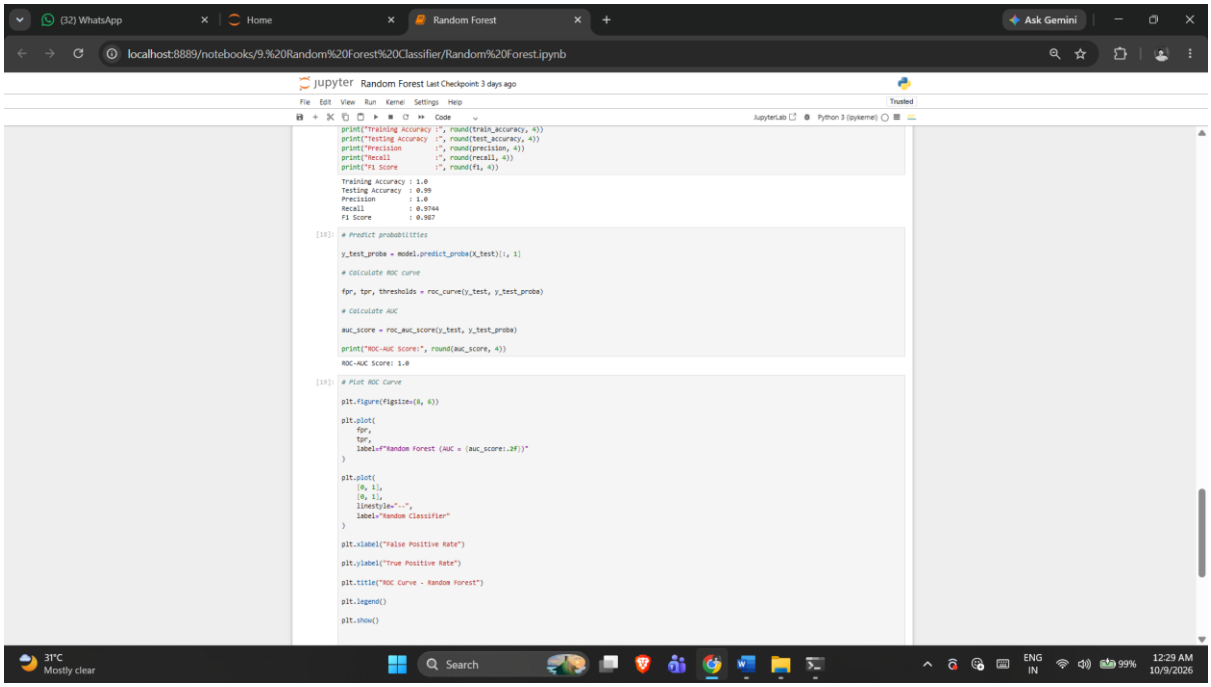
precision = precision_score(y_test, y_test_pred)

recall = recall_score(y_test, y_test_pred)

f1 = f1_score(y_test, y_test_pred)

print("Training Accuracy :", round(train_accuracy, 4))
print("Testing Accuracy  :", round(test_accuracy, 4))
print("Precision         :", round(precision, 4))
print("Recall            :", round(recall, 4))
print("F1 Score          :", round(f1, 4))
```

```
Training Accuracy : 1.0
Testing Accuracy  : 0.99
Precision         : 1.0
Recall            : 0.9744
F1 Score          : 0.987
```



```
print("Training Accuracy :", round(train_accuracy, 4))
print("Testing Accuracy  :", round(test_accuracy, 4))
print("Precision         :", round(precision, 4))
print("Recall            :", round(recall, 4))
print("F1 Score          :", round(f1, 4))
```

```
Training Accuracy : 1.0
Testing Accuracy  : 0.99
Precision         : 1.0
Recall            : 0.9744
F1 Score          : 0.987
```

[18]:
```
# Predict probabilities

y_test_proba = model.predict_proba(X_test)[:, 1]

# Calculate ROC curve

fpr, tpr, thresholds = roc_curve(y_test, y_test_proba)

# Calculate AUC

auc_score = roc_auc_score(y_test, y_test_proba)

print("ROC-AUC Score:", round(auc_score, 4))
```

```
ROC-AUC Score: 1.0
```

[19]:
```
# Plot ROC Curve

plt.figure(figsize=(8, 6))

plt.plot(
    fpr,
    tpr,
    label=f"Random Forest (AUC = {auc_score:.2f})"
)

plt.plot(
    [0, 1],
    [0, 1],
    linestyle="--",
    label="Random Classifier"
)

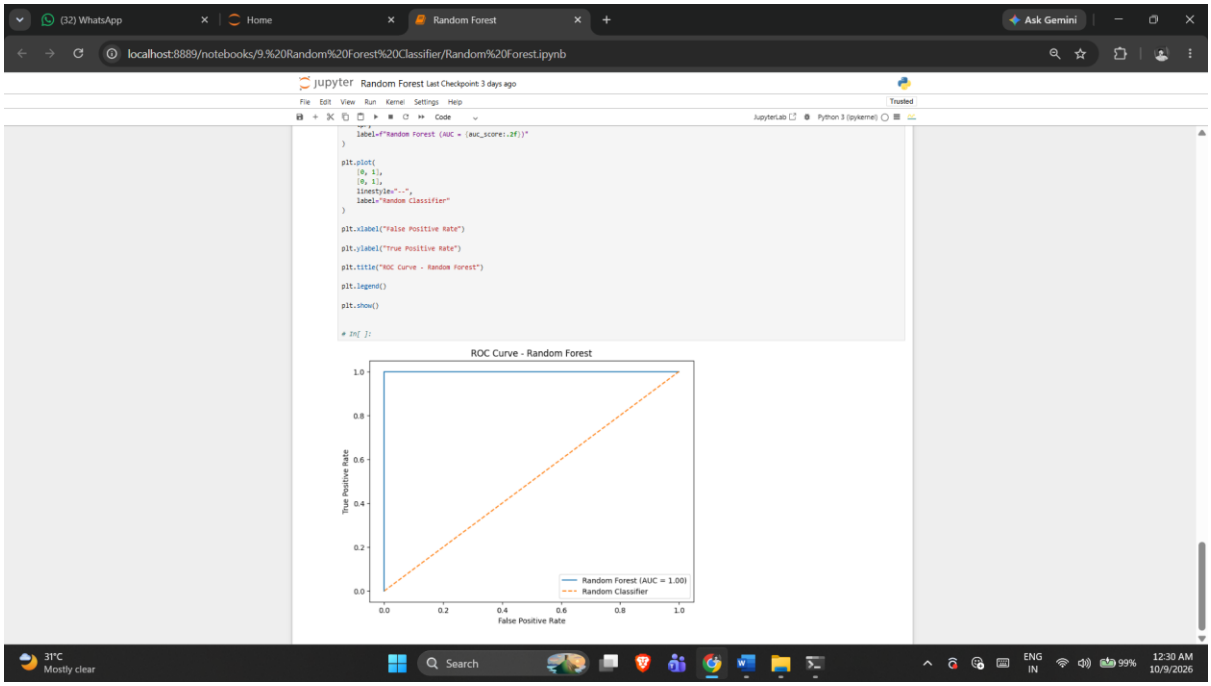
plt.xlabel("False Positive Rate")

plt.ylabel("True Positive Rate")

plt.title("ROC Curve - Random Forest")

plt.legend()

plt.show()
```



5. RESULT

Thus, the Random Forest Classifier practical was implemented successfully using the program from the corresponding Jupyter Notebook.

Exercise -10

Support Vector Machine

1. AIM

To perform SVM classification using Linear, Polynomial, RBF and Sigmoid kernels and compare their performance.

2. ALGORITHM

1. Load the dataset and separate features and target.

2. Identify numerical and categorical features.

3. Create scaling and encoding preprocessing.

4. Split the data with stratification.

5. Train SVM models using the four notebook kernels.

6. Calculate accuracy, precision, recall and F1 score.

7. Compare kernel performance in a table and plot.

3. PROGRAM

```
#!/usr/bin/env python
# coding: utf-8

# Step 1: Import libraries
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.svm import SVC
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report,
    confusion_matrix
)

df = pd.read_csv("heart_disease_dataset.csv")

print("Dataset Shape:", df.shape)
print("\nFirst 5 records:")
print(df.head())

X = df.drop("Heart Disease", axis=1)
y = df["Heart Disease"]
```

```
numeric_features = X.select_dtypes(include=["int64", "float64"]).columns
categorical_features = X.select_dtypes(include=["object"]).columns

print("\nNumerical Features:")
print(list(numeric_features))

print("\nCategorical Features:")
print(list(categorical_features))

preprocessor = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), numeric_features),
        ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_features)
    ]
)

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

kernels = ["linear", "poly", "rbf", "sigmoid"]

results = []

for kernel in kernels:

    model = Pipeline(
        steps=[
            ("preprocessor", preprocessor),
            ("svm", SVC(kernel=kernel, random_state=42))
        ]
    )

    model.fit(X_train, y_train)

    y_pred = model.predict(X_test)

    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred, zero_division=0)
    recall = recall_score(y_test, y_pred, zero_division=0)
    f1 = f1_score(y_test, y_pred, zero_division=0)

    results.append([
        kernel,
        accuracy,
        precision,
        recall,
        f1
    ])

    print("\n====================================")
    print("SVM Kernel:", kernel)
    print("====================================")
```

```
    print("Accuracy :", accuracy)
    print("Precision:", precision)
    print("Recall   :", recall)
    print("F1 Score :", f1)

    print("\nClassification Report:")
    print(classification_report(
        y_test,
        y_pred,
        zero_division=0
    ))

results_df = pd.DataFrame(
    results,
    columns=[
        "Kernel",
        "Accuracy",
        "Precision",
        "Recall",
        "F1 Score"
    ]
)

print("\n\nKernel Comparison:")
print(results_df)

plt.figure(figsize=(8, 5))

plt.bar(
    results_df["Kernel"],
    results_df["Accuracy"]
)

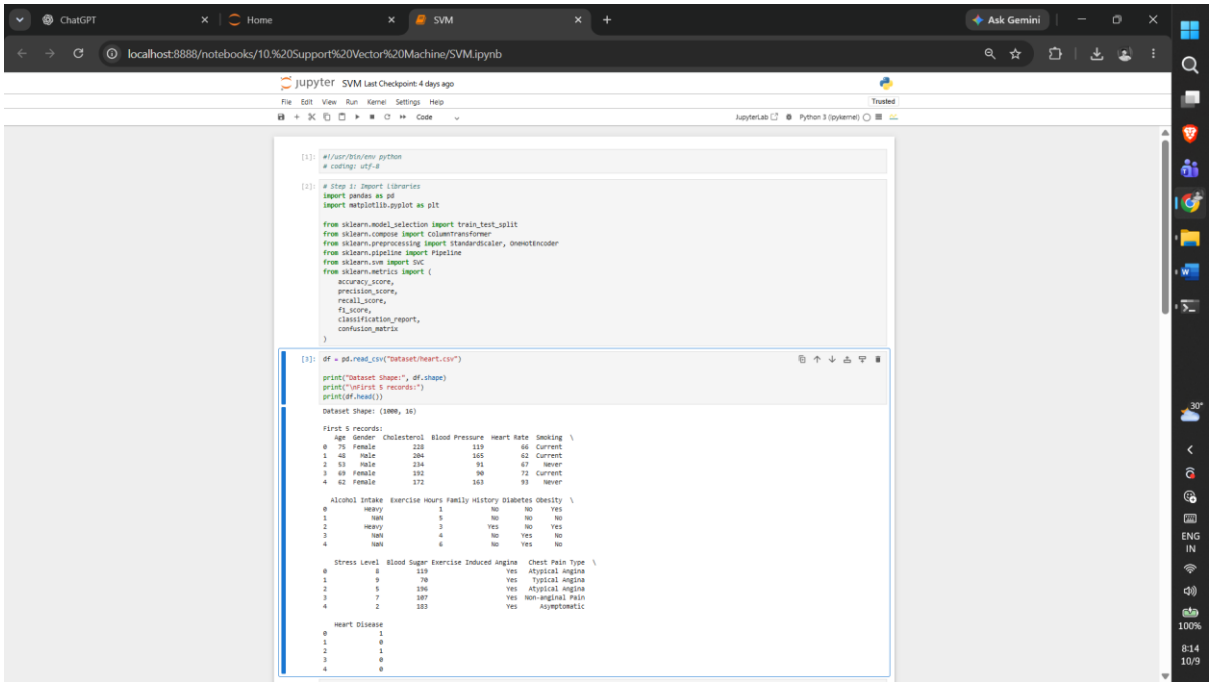
plt.xlabel("SVM Kernel")
plt.ylabel("Accuracy")
plt.title("SVM Performance with Different Kernels")
plt.ylim(0, 1)

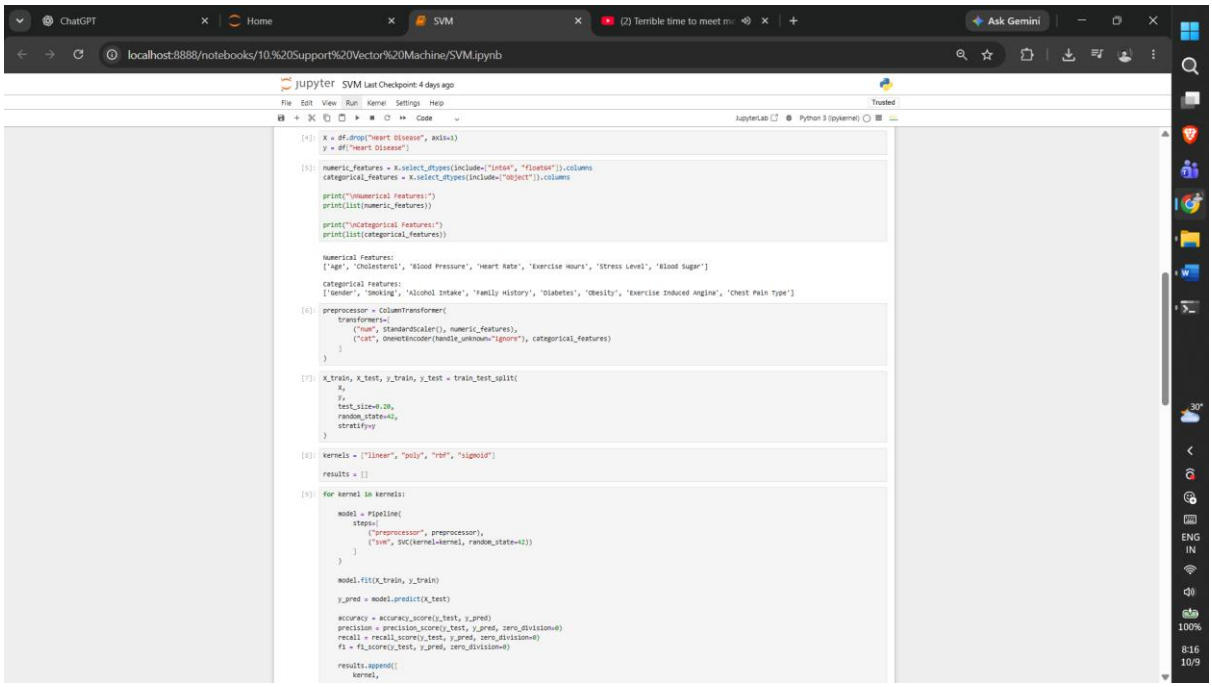
plt.show()

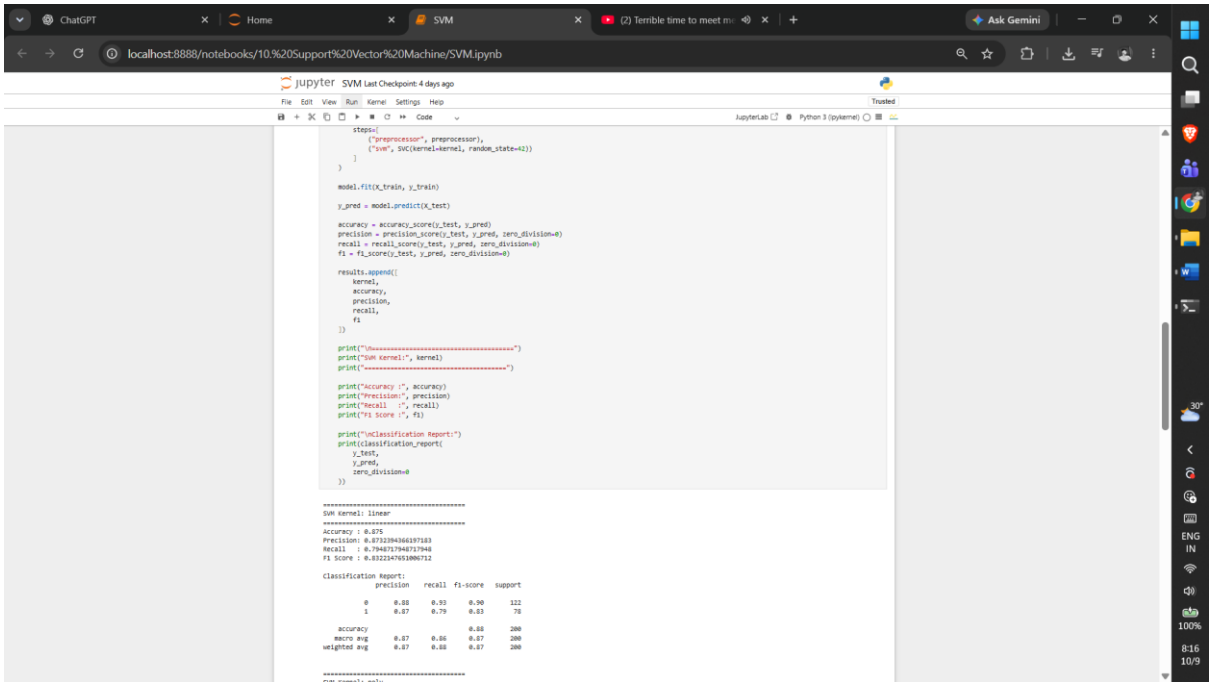
# ------------------------------------------------------------
# Result
# ------------------------------------------------------------

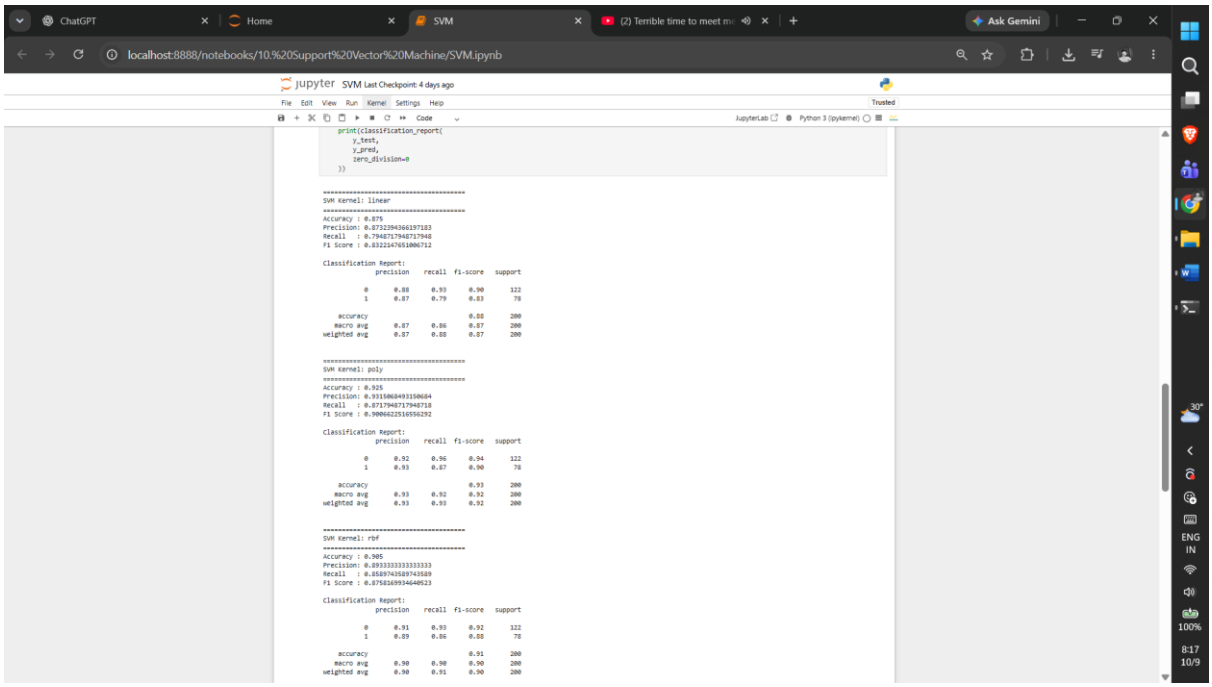
print("\nResult:")
print("SVM classification was successfully performed")
print("using Linear, Polynomial, RBF and Sigmoid kernels.")
```

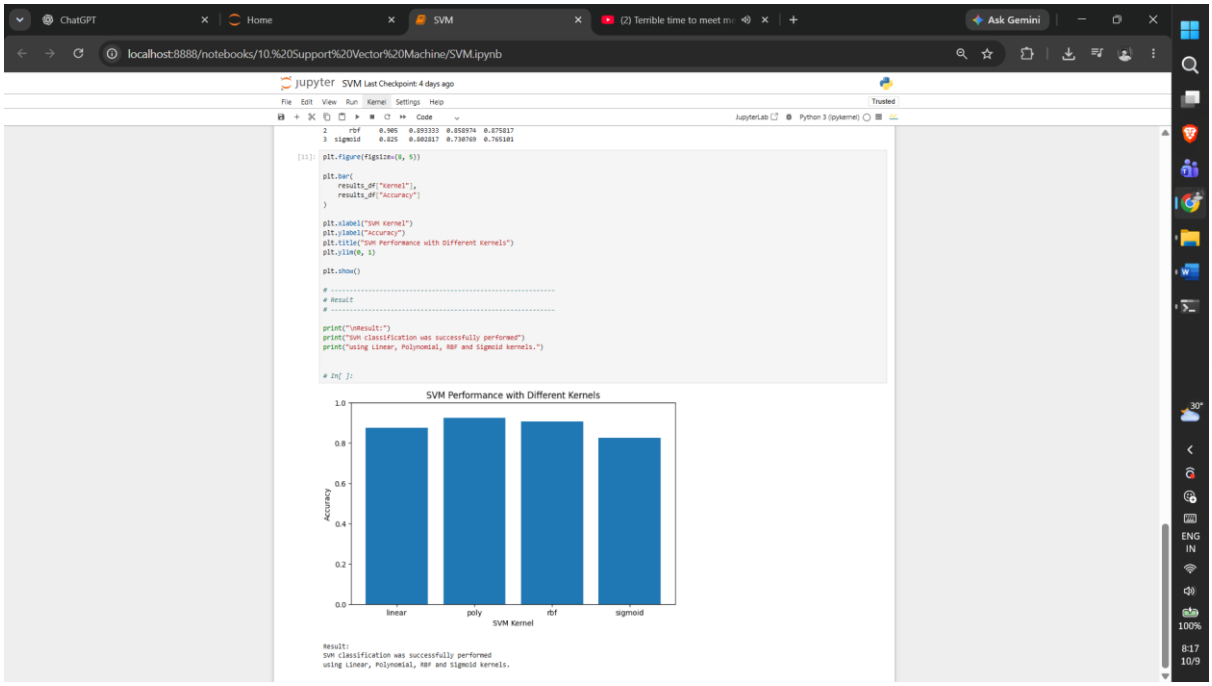
4. OUTPUT


ChatGPT
Home
SVM
Ask Gemini
localhost:8888/notebooks/10.%20Support%20Vector%20Machine/SVM.ipynb
jupyter SVM Last Checkpoint: 4 days ago
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
[1]:
#!/usr/bin/env python
coding: utf-8
[2]:
Step 1: Import Libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.svm import SVC
from sklearn.metrics import (
accuracy_score,
precision_score,
recall_score,
f1_score,
classification_report,
confusion_matrix
)
[3]:
df = pd.read_csv("Dataset/heart.csv")
print("Dataset Shape:", df.shape)
print("\nFirst 5 records:")
print(df.head())
Dataset Shape: (1000, 16)
First 5 records:
Age Gender Cholesterol Blood Pressure Heart Rate Smoking \
0 75 Female 228 119 66 Current
1 48 Male 204 165 62 Current
2 53 Male 234 91 67 Never
3 69 Female 192 90 72 Current
4 62 Female 172 163 93 Never
Alcohol Intake Exercise Hours Family History Diabetes Obesity \
0 Heavy 1 No No Yes
1 NaN 5 No No No
2 Heavy 3 Yes No Yes
3 NaN 4 No Yes No
4 NaN 6 No Yes No
Stress Level Blood Sugar Exercise Induced Angina Chest Pain Type \
0 8 119 Yes Atypical Angina
1 9 70 Yes Typical Angina
2 5 196 Yes Atypical Angina
3 7 107 Yes Non-anginal Pain
4 2 183 Yes Asymptomatic
Heart Disease
0 1
1 0
2 1
3 0
4 0


ChatGPT
Home
SVM
(2) Terrible time to meet m
Ask Gemini
localhost:8888/notebooks/10.%20Support%20Vector%20Machine/SVM.ipynb
jupyter SVM Last Checkpoint: 4 days ago
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
[4]:
X = df.drop("Heart Disease", axis=1)
y = df["Heart Disease"]
[5]:
numeric_features = X.select_dtypes(include=["int64", "float64"]).columns
categorical_features = X.select_dtypes(include=["object"]).columns
print("\nNumerical Features:")
print(list(numeric_features))
print("\nCategorical Features:")
print(list(categorical_features))
Numerical Features:
['Age', 'Cholesterol', 'Blood Pressure', 'Heart Rate', 'Exercise Hours', 'Stress Level', 'Blood Sugar']
Categorical Features:
['Gender', 'Smoking', 'Alcohol Intake', 'Family History', 'Diabetes', 'Obesity', 'Exercise Induced Angina', 'Chest Pain Type']
[6]:
preprocessor = ColumnTransformer(
transformers=[
("num", StandardScaler(), numeric_features),
("cat", OneHotEncoder(handle_unknown="ignore"), categorical_features)
]
)
[7]:
X_train, X_test, y_train, y_test = train_test_split(
X,
y,
test_size=0.20,
random_state=42,
stratify=y
)
[8]:
kernels = ["linear", "poly", "rbf", "sigmoid"]
results = []
[9]:
for kernel in kernels:
model = Pipeline(
steps=[
("preprocessor", preprocessor),
("svm", SVC(kernel=kernel, random_state=42))
]
)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, zero_division=0)
recall = recall_score(y_test, y_pred, zero_division=0)
f1 = f1_score(y_test, y_pred, zero_division=0)
results.append([
kernel,


ChatGPT
Home
SVM
(2) Terrible time to meet m
Ask Gemini
localhost:8888/notebooks/10.%20Support%20Vector%20Machine/SVM.ipynb
jupyter SVM Last Checkpoint: 4 days ago
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
steps=[
("preprocessor", preprocessor),
("svm", SVC(kernel=kernel, random_state=42))
]
)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, zero_division=0)
recall = recall_score(y_test, y_pred, zero_division=0)
f1 = f1_score(y_test, y_pred, zero_division=0)
results.append({
kernel,
accuracy,
precision,
recall,
f1
})
print("\n==========")
print("SVM Kernel:", kernel)
print("==========")
print("Accuracy :", accuracy)
print("Precision:", precision)
print("Recall :", recall)
print("F1 Score :", f1)
print("\nClassification Report:")
print(classification_report(
y_test,
y_pred,
zero_division=0
))
==========
SVM Kernel: linear
==========
Accuracy : 0.875
Precision: 0.8732394366197183
Recall : 0.7948717948717948
F1 Score : 0.8322147651006712
Classification Report:
precision recall f1-score support
0 0.88 0.93 0.90 122
1 0.87 0.79 0.83 78
accuracy 0.88 200
macro avg 0.87 0.86 0.87 200
weighted avg 0.87 0.88 0.87 200
==========
30°
ENG
IN
100%
8:16
10/9


ChatGPT
Home
SVM
(2) Terrible time to meet m
Ask Gemini
localhost:8888/notebooks/10.%20Support%20Vector%20Machine/SVM.ipynb
jupyter SVM Last Checkpoint: 4 days ago
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
print(classification_report(
y_test,
y_pred,
zero_division=0
))
==========
SVM Kernel: linear
==========
Accuracy : 0.875
Precision: 0.8732394366197183
Recall : 0.7948717948717948
F1 Score : 0.8322147651006712
Classification Report:
precision recall f1-score support
0 0.88 0.93 0.90 122
1 0.87 0.79 0.83 78
accuracy 0.88 200
macro avg 0.87 0.86 0.87 200
weighted avg 0.87 0.88 0.87 200
==========
SVM Kernel: poly
==========
Accuracy : 0.925
Precision: 0.9315068493150684
Recall : 0.8717948717948718
F1 Score : 0.9006622516556292
Classification Report:
precision recall f1-score support
0 0.92 0.96 0.94 122
1 0.93 0.87 0.90 78
accuracy 0.93 200
macro avg 0.93 0.92 0.92 200
weighted avg 0.93 0.93 0.92 200
==========
SVM Kernel: rbf
==========
Accuracy : 0.905
Precision: 0.8933333333333333
Recall : 0.8589743589743589
F1 Score : 0.8758169934640523
Classification Report:
precision recall f1-score support
0 0.91 0.93 0.92 122
1 0.89 0.86 0.88 78
accuracy 0.91 200
macro avg 0.90 0.90 0.90 200
weighted avg 0.90 0.91 0.90 200
30°
ENG
IN
100%
8:17
10/9



5. RESULT

Thus, the Support Vector Machine practical was implemented successfully using the program from the corresponding Jupyter Notebook.

Exercise -11

Hyperparameter Tuning using GridSearchCV

1. AIM

To tune the hyperparameters of a Support Vector Machine classifier for heart disease prediction using GridSearchCV and evaluate the best model on the test data.

2. ALGORITHM

1. Load the dataset and separate features and target.
2. Identify numerical and categorical features.
3. Create scaling and encoding preprocessing.
4. Build a pipeline of the preprocessor and the SVM classifier.
5. Split the data with stratification.
6. Define the parameter grid for C, kernel and gamma.
7. Run GridSearchCV with 5-fold cross-validation.
8. Find the best parameters and best cross-validation score.
9. Predict with the best model and calculate accuracy, precision, recall and F1 score.

3. PROGRAM

```
#!/usr/bin/env python
# coding: utf-8

import pandas as pd
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.svm import SVC
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report,
    confusion_matrix
)

df = pd.read_csv("heart_disease_dataset.csv")
print("Dataset Shape:", df.shape)
print(df.head())

X = df.drop("Heart Disease", axis=1)
y = df["Heart Disease"]

numeric_features = X.select_dtypes(
    include=["int64", "float64"]
).columns
categorical_features = X.select_dtypes(
    include=["object"]
).columns
preprocessor = ColumnTransformer(
```

```
    transformers=[
        ("num", StandardScaler(), numeric_features),
        ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_features)
    ]
)

pipeline = Pipeline(
    steps=[
        ("preprocessor", preprocessor),
        ("svm", SVC())
    ]
)

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

param_grid = {
    "svm__C": [0.1, 1, 10, 100],
    "svm__kernel": [
        "linear",
        "rbf",
        "poly"
    ],
    "svm__gamma": [
        "scale",
        "auto"
    ]
}

grid_search = GridSearchCV(
    estimator=pipeline,
    param_grid=param_grid,
    cv=5,
    scoring="accuracy",
    n_jobs=-1
)

print("\nTraining GridSearchCV...")
grid_search.fit(X_train, y_train)

print("\n====================================")
print("BEST PARAMETERS")
print("====================================")
print(grid_search.best_params_)
print("\nBest Cross Validation Score:")
print(grid_search.best_score_)

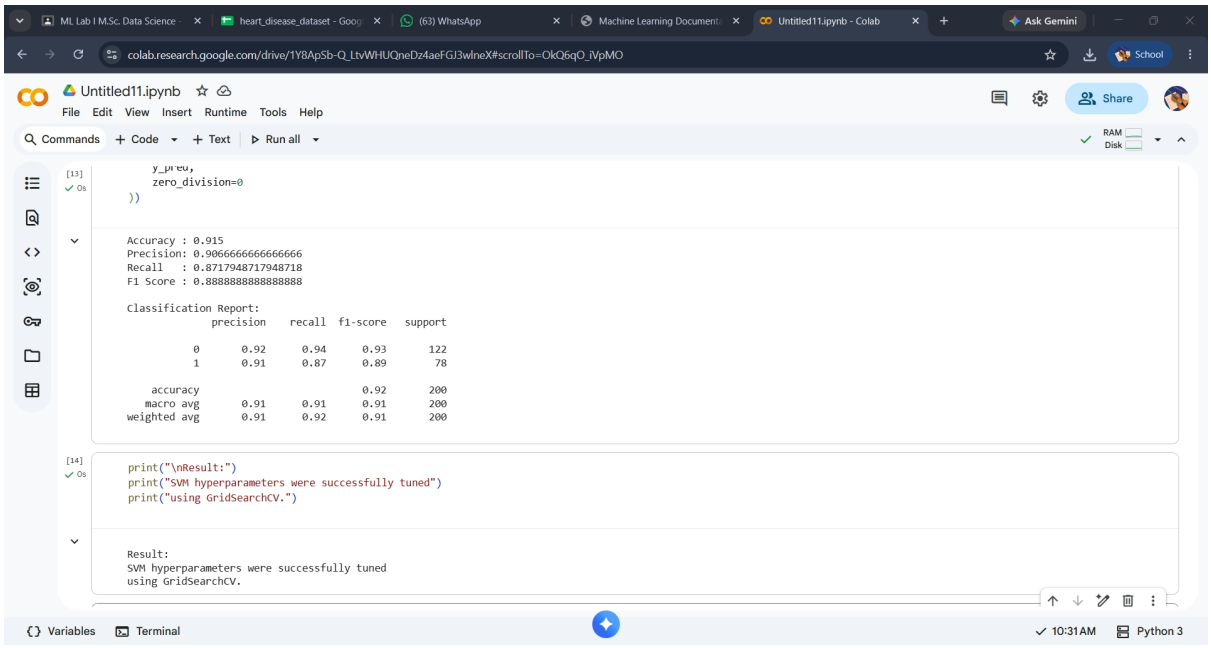
best_model = grid_search.best_estimator_
y_pred = best_model.predict(X_test)

print("Accuracy :",
      accuracy_score(y_test, y_pred))
print("Precision:",
      precision_score(y_test, y_pred, zero_division=0))
print("Recall   :",
      recall_score(y_test, y_pred, zero_division=0))
print("F1 Score :",
      f1_score(y_test, y_pred, zero_division=0))
```

```
print("\nClassification Report:")
print(classification_report(
    y_test,
    y_pred,
    zero_division=0
))

print("\nResult:")
print("SVM hyperparameters were successfully tuned")
print("using GridSearchCV.")
```

4. OUTPUT



5. RESULT

Thus, the Hyperparameter Tuning using GridSearchCV practical was implemented successfully using the program from the corresponding Jupyter Notebook.

Exercise -12

K Means Clustering

1. AIM

To perform K-Means clustering and use the Elbow Method to determine a suitable number of clusters and visualize the resulting groups.

2. ALGORITHM

1. Load and prepare the dataset.

2. Preprocess the features as implemented in the notebook.

3. Run K-Means for the specified range of K values.

4. Calculate the clustering inertia values.

5. Plot the Elbow curve.

6. Fit and visualize the selected clustering solution.

3. PROGRAM

```
#!/usr/bin/env python
# coding: utf-8

import pandas as pd
import matplotlib.pyplot as plt

from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.cluster import KMeans

df = pd.read_csv("heart_disease_dataset.csv")

print("Dataset Shape:", df.shape)
print("\nFirst 5 records:")
print(df.head())

X = df.drop("Heart Disease", axis=1)

numeric_features = X.select_dtypes(
    include=["int64", "float64"]
).columns

categorical_features = X.select_dtypes(
    include=["object"]
).columns

preprocessor = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), numeric_features),
        ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_features)
    ]
```

```
)

X_processed = preprocessor.fit_transform(X)

# Check whether X_processed is sparse or already a NumPy array
if hasattr(X_processed, "toarray"):
    X_processed = X_processed.toarray()

print("\nProcessed Data Shape:")
print(X_processed.shape)

# ------------------------------------------------------------
# Step 7: Apply Elbow Method
# ------------------------------------------------------------

inertia = []

K = range(1, 11)

for k in K:

    kmeans = KMeans(
        n_clusters=k,
        random_state=42,
        n_init=10
    )

    kmeans.fit(X_processed)

    inertia.append(kmeans.inertia_)

plt.figure(figsize=(8, 5))

plt.plot(
    K,
    inertia,
    marker="o"
)

plt.xlabel("Number of Clusters (K)")
plt.ylabel("Inertia")
plt.title("Elbow Method for Optimal K")

plt.xticks(K)

plt.grid(True)

plt.show()

# Change this value after observing the elbow plot.
optimal_k = 3

print("\nSelected Optimal K:", optimal_k)

kmeans = KMeans(
    n_clusters=optimal_k,
```

```
    random_state=42,
    n_init=10
)

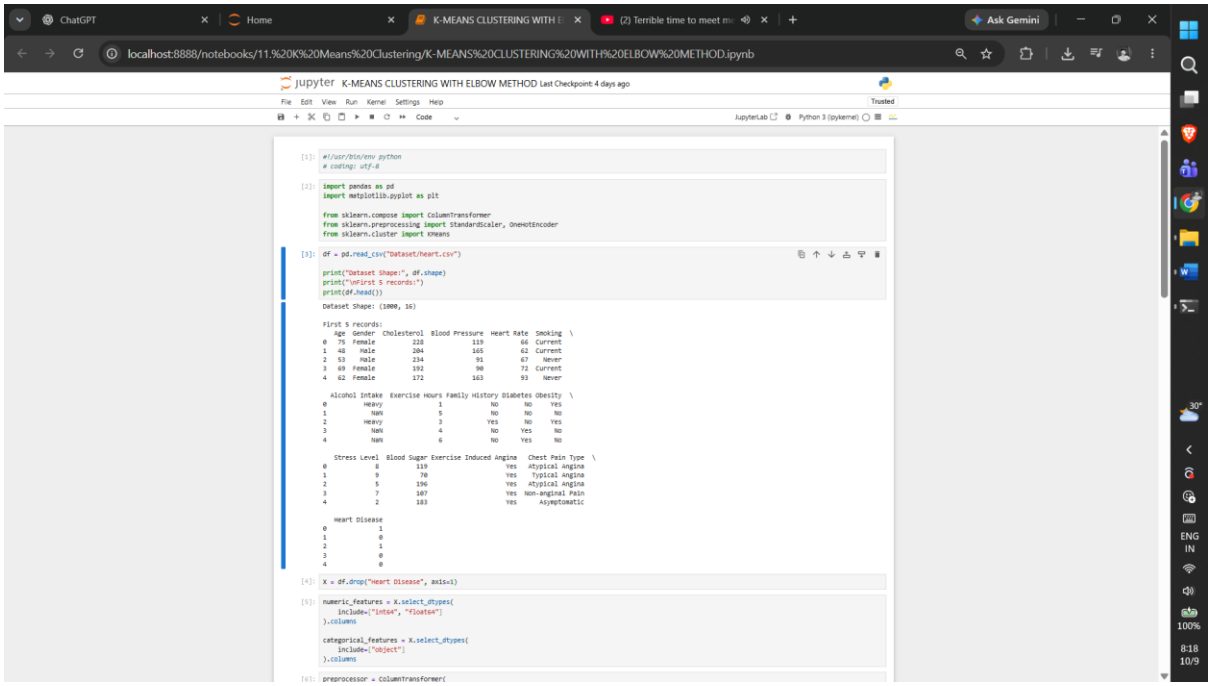
clusters = kmeans.fit_predict(X_processed)

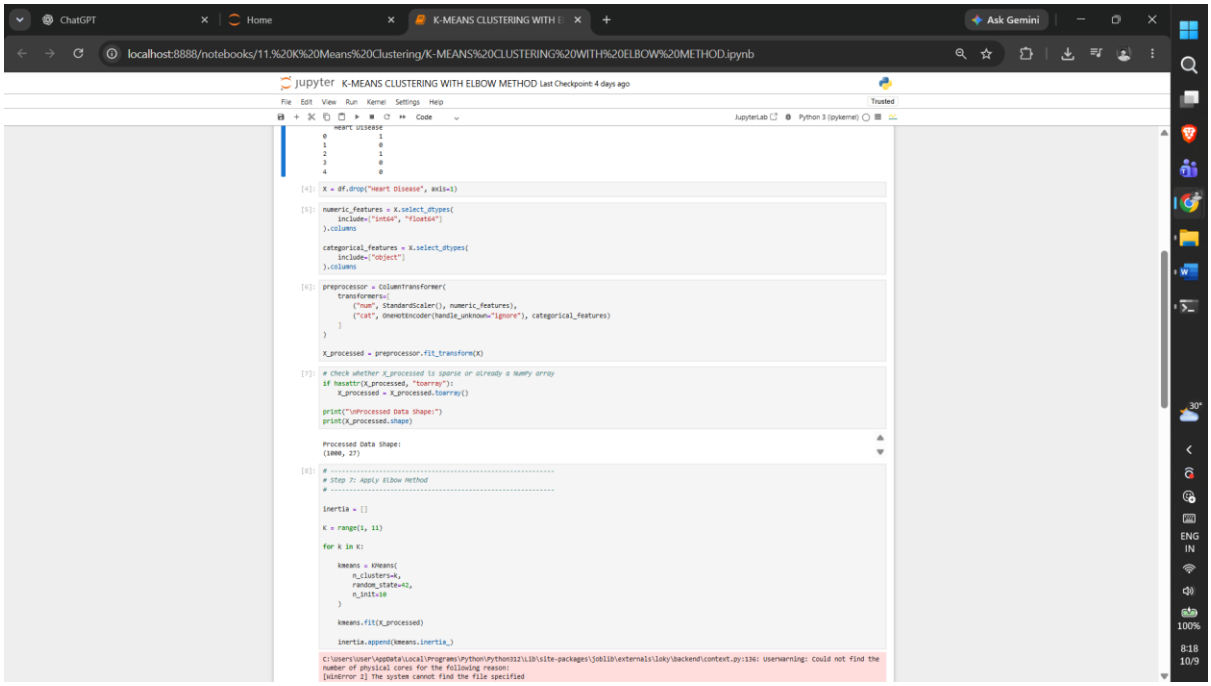
df["Cluster"] = clusters

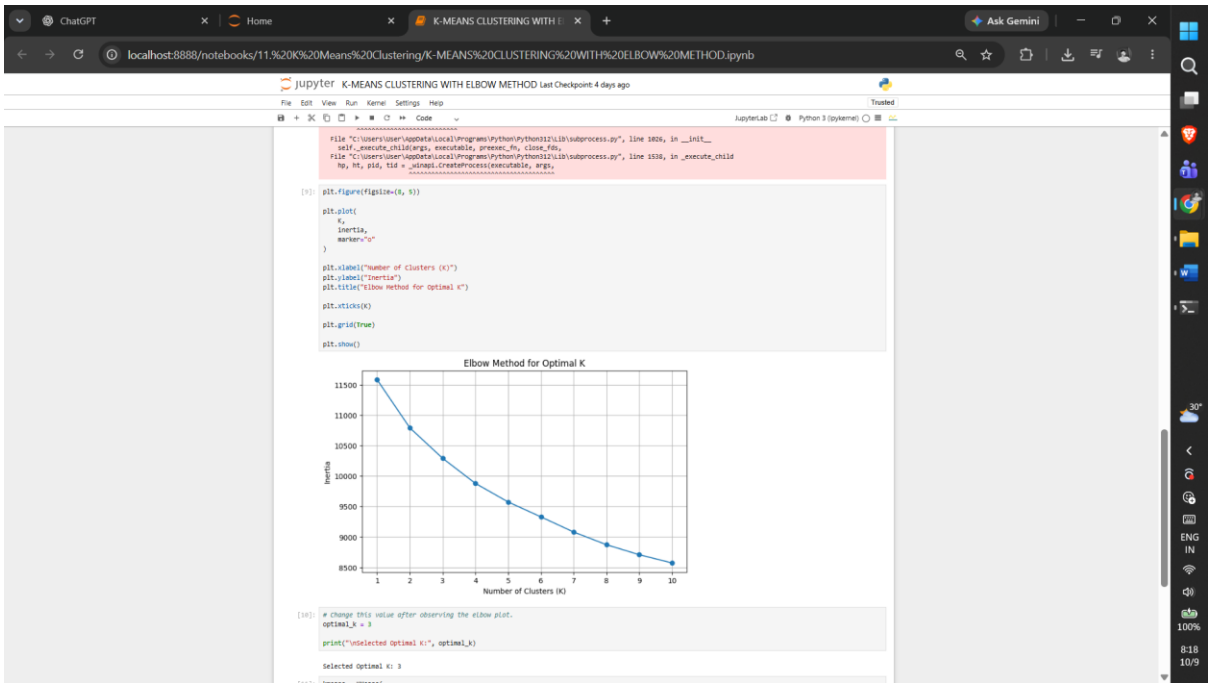
print("\nClustered Dataset:")
print(df.head(10))

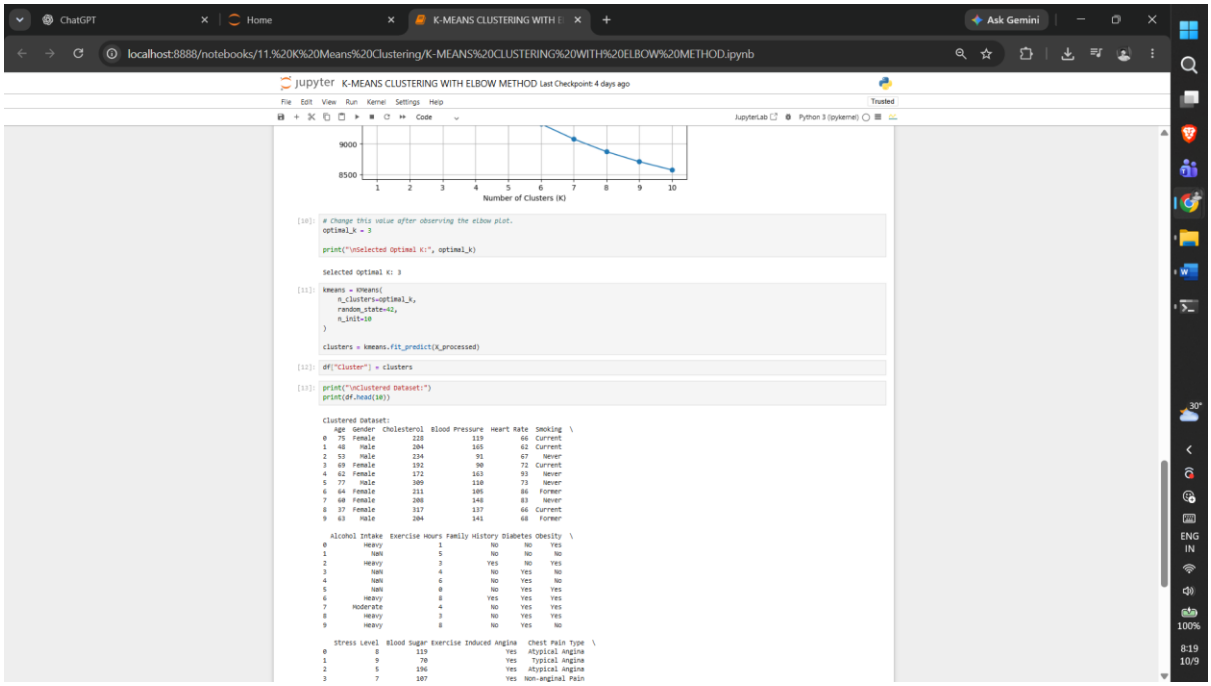
print("\nNumber of records in each cluster:")
print(df["Cluster"].value_counts().sort_index())
```

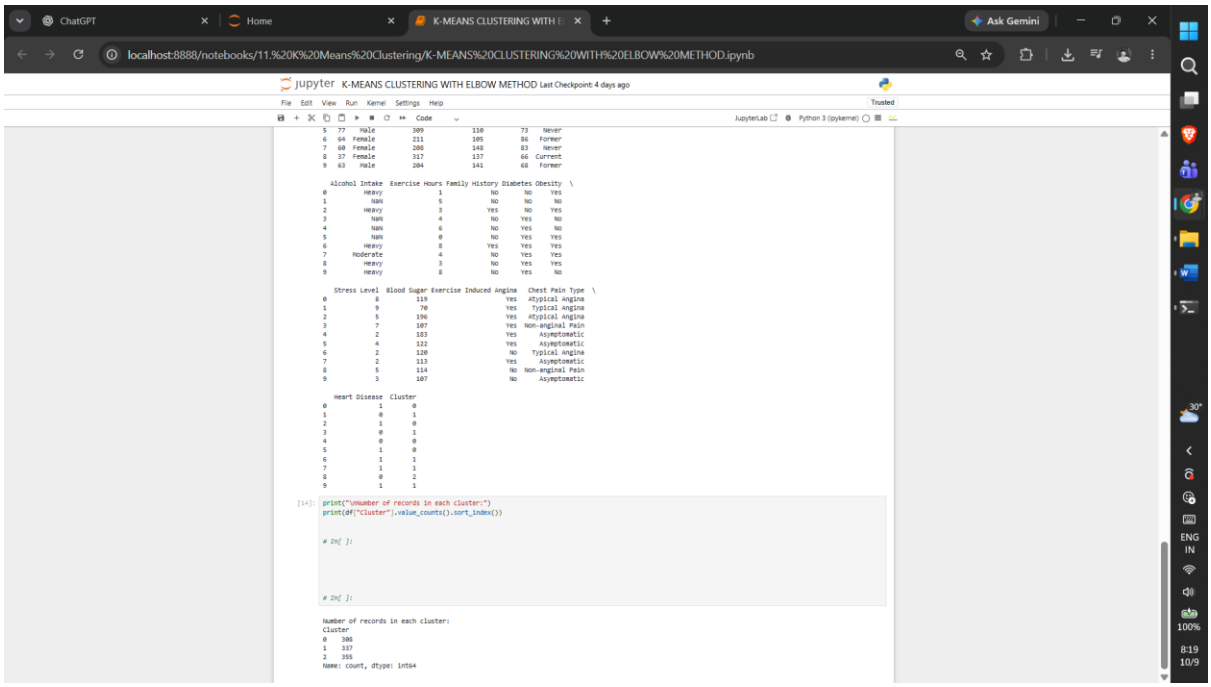
4. OUTPUT




ChatGPT
Home
K-MEANS CLUSTERING WITH E
Ask Gemini
localhost:8888/notebooks/11.%20K%20Means%20Clustering/K-MEANS%20CLUSTERING%20WITH%20ELBOW%20METHOD.ipynb
jupyter K-MEANS CLUSTERING WITH ELBOW METHOD Last Checkpoint: 4 days ago
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
Heart Disease
0 1
1 0
2 1
3 0
4 0
[4]: X = df.drop("Heart Disease", axis=1)
[5]: numeric_features = X.select_dtypes(
include=["int64", "float64"]
).columns
categorical_features = X.select_dtypes(
include="object"
).columns
[6]: preprocessor = ColumnTransformer(
transformers=[
("num", StandardScaler(), numeric_features),
("cat", OneHotEncoder(handle_unknown="ignore"), categorical_features)
]
)
X_processed = preprocessor.fit_transform(X)
[7]: # Check whether X_processed is sparse or already a NumPy array
if hasattr(X_processed, "toarray"):
X_processed = X_processed.toarray()
print("\nProcessed Data Shape:")
print(X_processed.shape)
Processed Data Shape:
(1000, 27)
[8]: # --
Step 7: Apply Elbow Method
--
inertia = []
K = range(1, 11)
for k in K:
kmeans = KMeans(
n_clusters=k,
random_state=42,
n_init=10
)
kmeans.fit(X_processed)
inertia.append(kmeans.inertia_)
C:\Users\User\AppData\Local\Programs\Python\Python312\Lib\site-packages\joblib\externals\loky\backend\context.py:136: UserWarning: Could not find the number of physical cores for the following reason:
[WinError 2] The system cannot find the file specified


ChatGPT
Home
K-MEANS CLUSTERING WITH E
Ask Gemini
localhost:8888/notebooks/11.%20K%20Means%20Clustering/K-MEANS%20CLUSTERING%20WITH%20ELBOW%20METHOD.ipynb
jupyter K-MEANS CLUSTERING WITH ELBOW METHOD Last Checkpoint: 4 days ago
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
File "C:\Users\User\AppData\Local\Programs\Python\Python312\Lib\subprocess.py", line 1026, in __init__
self._execute_child(args, executable, preexec_fn, close_fds,
File "C:\Users\User\AppData\Local\Programs\Python\Python312\Lib\subprocess.py", line 1538, in _execute_child
hp, ht, pid, tid = _winapi.CreateProcess(executable, args,
[9]: plt.figure(figsize=(8, 5))
plt.plot(
K,
inertia,
marker="o"
)
plt.xlabel("Number of Clusters (K)")
plt.ylabel("Inertia")
plt.title("Elbow Method for Optimal K")
plt.xticks(K)
plt.grid(True)
plt.show()
Elbow Method for Optimal K
Inertia
Number of Clusters (K)
11500
11000
10500
10000
9500
9000
8500
1
2
3
4
5
6
7
8
9
10
[10]: # Change this value after observing the elbow plot.
optimal_k = 3
print("\nSelected Optimal K:", optimal_k)
Selected Optimal K: 3





5. RESULT

Thus, the K Means Clustering practical was implemented successfully using the program from the corresponding Jupyter Notebook.

Exercise -13

Hierarchical Clustering

1. AIM

To perform hierarchical clustering on the preprocessed heart disease dataset using Ward linkage and visualize the hierarchy with a dendrogram.

2. ALGORITHM

1. Load the dataset and remove the target.

2. Identify numerical and categorical features.

3. Standardize numerical data and one-hot encode categorical data.

4. Create the processed feature matrix.

5. Perform Ward hierarchical linkage.

6. Plot the dendrogram.

3. PROGRAM

```
#!/usr/bin/env python
# coding: utf-8

import pandas as pd
import matplotlib.pyplot as plt

from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

from scipy.cluster.hierarchy import (
    dendrogram,
    linkage
)

df = pd.read_csv("heart_disease_dataset.csv")

print("Dataset Shape:", df.shape)
print("\nFirst 5 records:")
print(df.head())

X = df.drop("Heart Disease", axis=1)

numeric_features = X.select_dtypes(
    include=["int64", "float64"]
).columns

categorical_features = X.select_dtypes(
    include=["object"]
).columns

preprocessor = ColumnTransformer(
```

```
        transformers=[
            ("num", StandardScaler(), numeric_features),
            ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_features)
        ]
    )

    X_processed = preprocessor.fit_transform(X)

    # Check whether X_processed is sparse or already a NumPy array
    if hasattr(X_processed, "toarray"):
        X_processed = X_processed.toarray()

    print("\nProcessed Data Shape:")
    print(X_processed.shape)

    linked = linkage(
        X_processed,
        method="ward"
    )

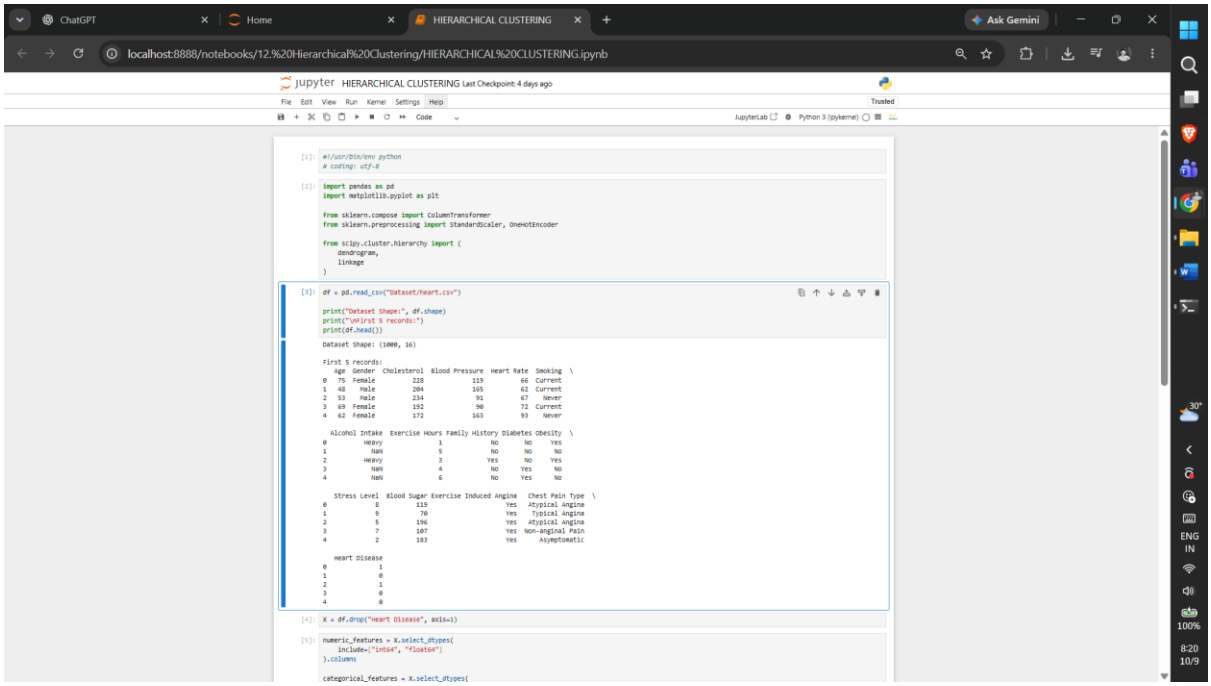
    plt.figure(figsize=(14, 7))

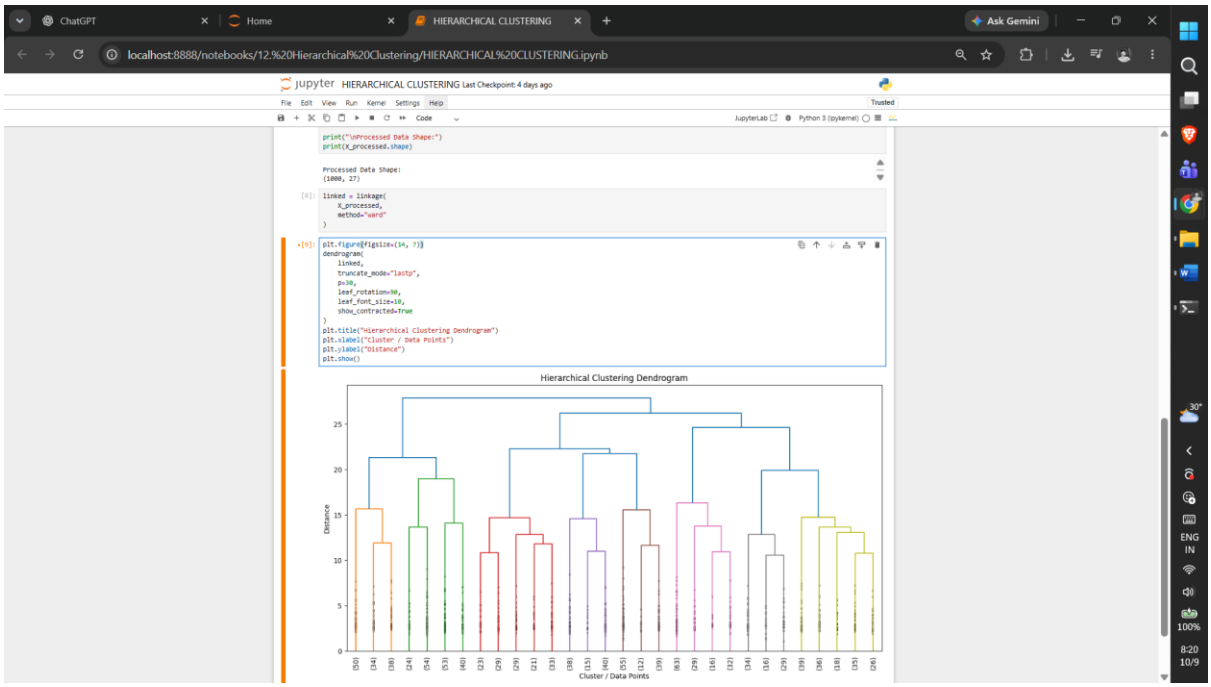
    dendrogram(
        linked,
        truncate_mode="lastp",
        p=30,
        leaf_rotation=90,
        leaf_font_size=10,
        show_contracted=True
    )

    plt.title("Hierarchical Clustering Dendrogram")
    plt.xlabel("Cluster / Data Points")
    plt.ylabel("Distance")

plt.show()
```

4. OUTPUT





5. RESULT

Thus, the Hierarchical Clustering practical was implemented successfully using the program from the corresponding Jupyter Notebook.

Exercise -14

DBSCAN Clustering

1. AIM

To perform DBSCAN clustering on the heart disease dataset, identify clusters and noise points, and visualize the clustering result using PCA.

2. ALGORITHM

1. Load the dataset and separate the target.

2. Identify numerical and categorical features.

3. Scale numerical features and one-hot encode categorical features.

4. Apply DBSCAN using the notebook parameters.

5. Assign labels, identify noise points and count clusters.

6. Apply PCA for two-dimensional visualization.

3. PROGRAM

```
#!/usr/bin/env python
# coding: utf-8

import pandas as pd
import matplotlib.pyplot as plt

from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.cluster import DBSCAN
from sklearn.decomposition import PCA

df = pd.read_csv("heart_disease_dataset.csv")

print("Dataset Shape:", df.shape)
print("\nFirst 5 records:")
print(df.head())

X = df.drop("Heart Disease", axis=1)

numeric_features = X.select_dtypes(
    include=["int64", "float64"]
).columns

categorical_features = X.select_dtypes(
    include=["object"]
).columns

preprocessor = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), numeric_features),
        ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_features)
```

```
    ]
)

X_processed = preprocessor.fit_transform(X)

X_processed = X_processed

dbscan = DBSCAN(
    eps=1.5,
    min_samples=5
)

labels = dbscan.fit_predict(X_processed)

df["Cluster"] = labels

print("\nCluster Labels:")
print(df["Cluster"].value_counts().sort_index())

noise_points = df[df["Cluster"] == -1]

print("\nNumber of Noise Points:")
print(len(noise_points))

print("\nNoise Points:")
print(noise_points.head(10))

number_of_clusters = len(
    set(labels)
) - (1 if -1 in labels else 0)

print("\nNumber of Clusters:")
print(number_of_clusters)

pca = PCA(n_components=2)

X_pca = pca.fit_transform(X_processed)

plt.figure(figsize=(9, 6))

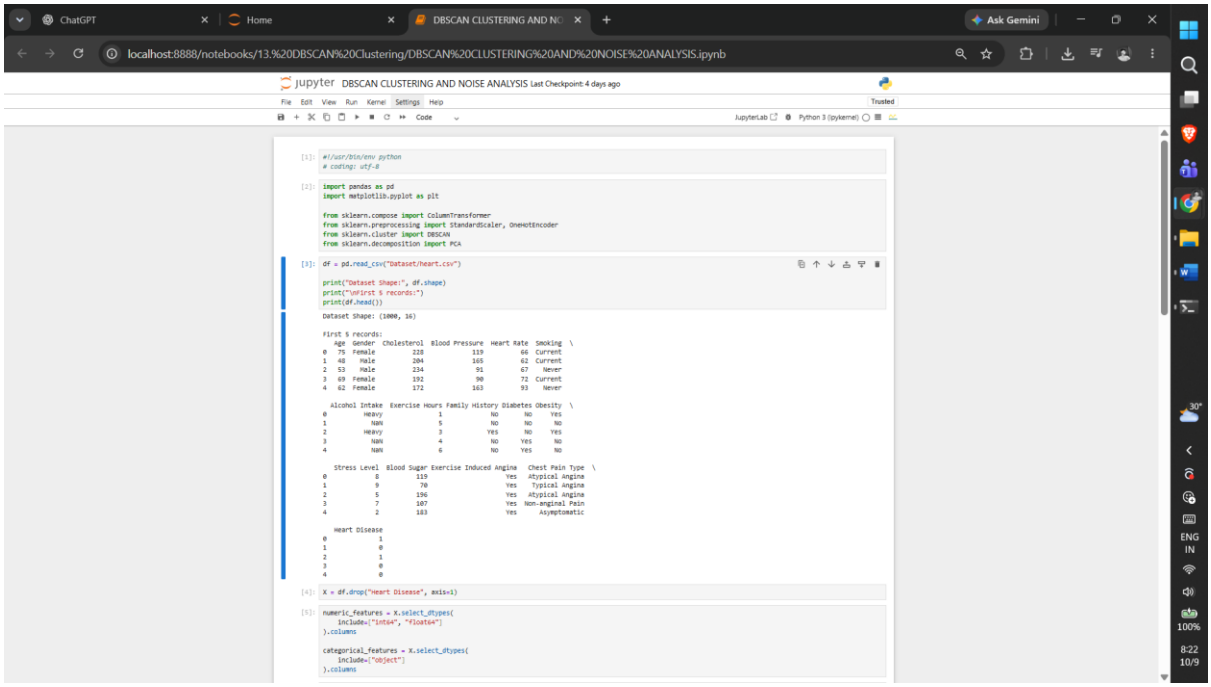
plt.scatter(
    X_pca[:, 0],
    X_pca[:, 1],
    c=labels,
    cmap="viridis",
    s=40
)

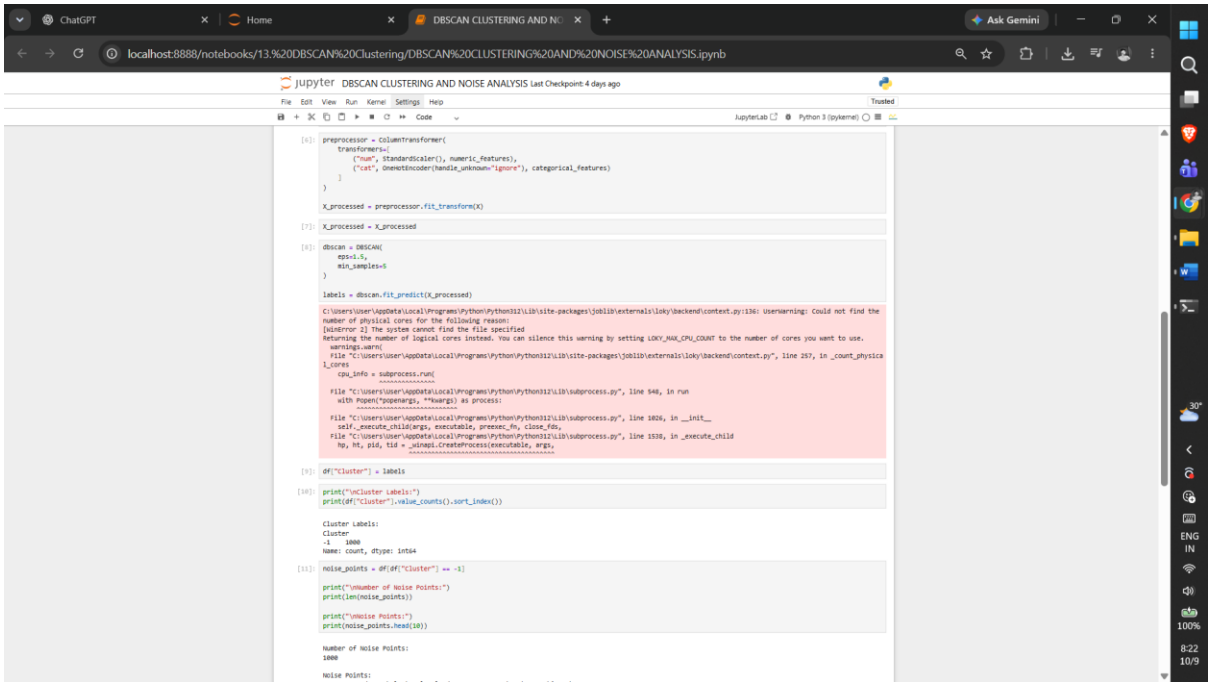
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("DBSCAN Clustering")

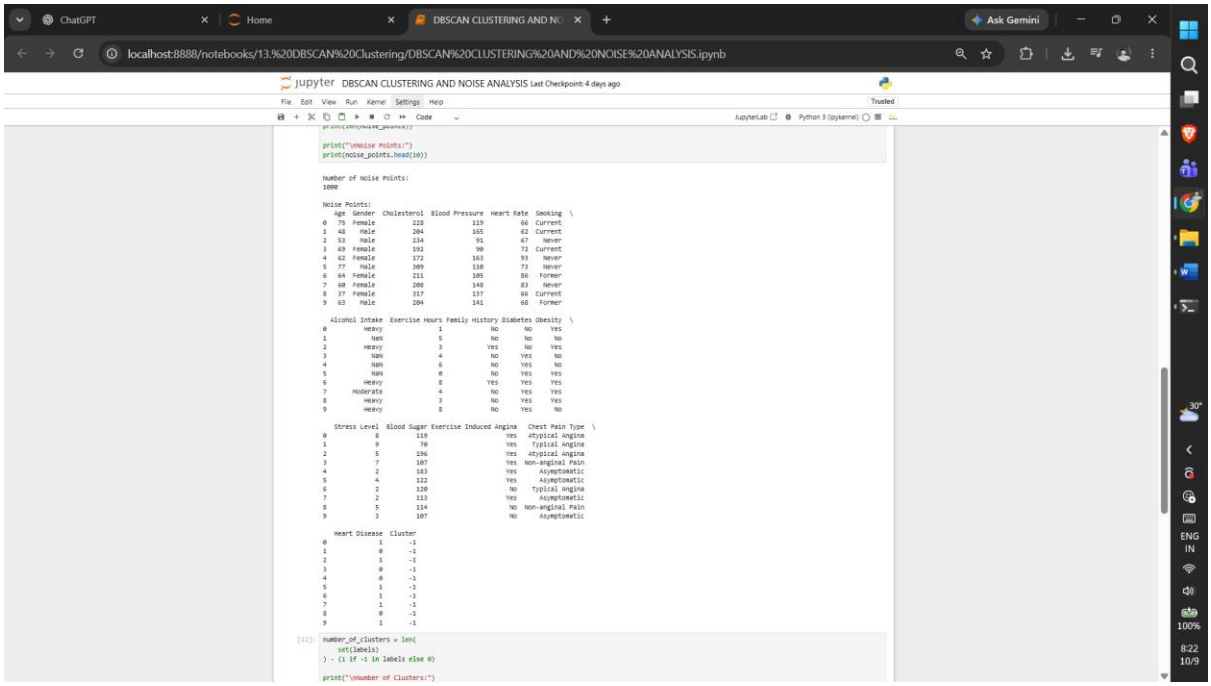
plt.colorbar(label="Cluster")

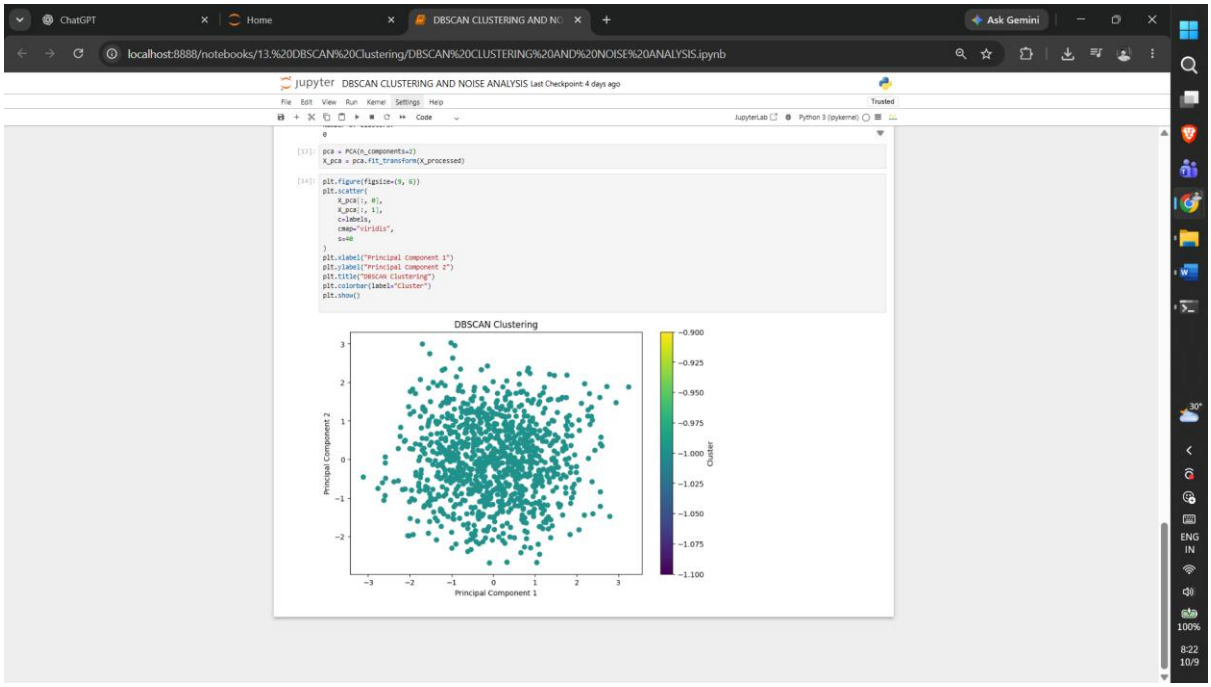
plt.show()
```

4. OUTPUT


ChatGPT
Home
DBSCAN CLUSTERING AND NO
Ask Gemini
localhost:8888/notebooks/13.%20DBSCAN%20Clustering/DBSCAN%20CLUSTERING%20AND%20NOISE%20ANALYSIS.ipynb
jupyter DBSCAN CLUSTERING AND NOISE ANALYSIS Last Checkpoint: 4 days ago
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
[1]: #!/usr/bin/env python
coding: utf-8
[2]: import pandas as pd
import matplotlib.pyplot as plt
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.cluster import DBSCAN
from sklearn.decomposition import PCA
[3]: df = pd.read_csv("Dataset/heart.csv")
print("Dataset Shape:", df.shape)
print("\nFirst 5 records:")
print(df.head())
Dataset Shape: (1000, 16)
First 5 records:
Age Gender Cholesterol Blood Pressure Heart Rate Smoking \
0 75 Female 228 119 66 Current
1 48 Male 204 165 62 Current
2 53 Male 234 91 67 Never
3 69 Female 192 90 72 Current
4 62 Female 172 163 93 Never
Alcohol Intake Exercise Hours Family History Diabetes Obesity \
0 Heavy 1 No No Yes
1 NAN 5 No No No
2 Heavy 3 Yes No Yes
3 NAN 4 No Yes No
4 NAN 6 No Yes No
Stress Level Blood Sugar Exercise Induced Angina Chest Pain Type \
0 8 115 Yes Atypical Angina
1 9 70 Yes Typical Angina
2 5 196 Yes Atypical Angina
3 7 107 Yes Non-anginal Pain
4 2 183 Yes Asymptomatic
Heart Disease
0 1
1 0
2 1
3 0
4 0
[4]: X = df.drop("Heart Disease", axis=1)
[5]: numeric_features = X.select_dtypes(
include=["int64", "float64"]
).columns
categorical_features = X.select_dtypes(
include=["object"]
).columns
ENG
IN
100%
8:22
10/9


ChatGPT
Home
DBSCAN CLUSTERING AND NO
Ask Gemini
localhost:8888/notebooks/13.%20DBSCAN%20Clustering/DBSCAN%20CLUSTERING%20AND%20NOISE%20ANALYSIS.ipynb
jupyter DBSCAN CLUSTERING AND NOISE ANALYSIS Last Checkpoint: 4 days ago
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
[6]: preprocessor = ColumnTransformer(
transformers=[
("num", StandardScaler(), numeric_features),
("cat", OneHotEncoder(handle_unknown="ignore"), categorical_features)
]
)
X_processed = preprocessor.fit_transform(X)
[7]: X_processed = X_processed
[8]: dbscan = DBSCAN(
eps=1.5,
min_samples=5
)
labels = dbscan.fit_predict(X_processed)
C:\Users\User\AppData\Local\Programs\Python\Python312\Lib\site-packages\joblib\externals\loky\backend\context.py:136: UserWarning: Could not find the number of physical cores for the following reason:
[WinError 2] The system cannot find the file specified
Returning the number of logical cores instead. You can silence this warning by setting LOKY_MAX_CPU_COUNT to the number of cores you want to use.
warnings.warn(
File "C:\Users\User\AppData\Local\Programs\Python\Python312\Lib\site-packages\joblib\externals\loky\backend\context.py", line 257, in _count_physical_cores
cpu_info = subprocess.run(
File "C:\Users\User\AppData\Local\Programs\Python\Python312\Lib\subprocess.py", line 548, in run
with Popen(*popenargs, **kwargs) as process:
File "C:\Users\User\AppData\Local\Programs\Python\Python312\Lib\subprocess.py", line 1026, in __init__
self._execute_child(args, executable, preexec_fn, close_fds,
File "C:\Users\User\AppData\Local\Programs\Python\Python312\Lib\subprocess.py", line 1538, in _execute_child
hp, ht, pid, tid = _winapi.CreateProcess(executable, args,
[9]: df["Cluster"] = labels
[10]: print("\nCluster Labels:")
print(df["Cluster"].value_counts().sort_index())
Cluster Labels:
Cluster
-1 1000
Name: count, dtype: int64
[11]: noise_points = df[df["Cluster"] == -1]
print("\nNumber of Noise Points:")
print(len(noise_points))
print("\nNoise Points:")
print(noise_points.head(10))
Number of Noise Points:
1000
Noise Points:
ENG
IN
100%
8:22
10/9





5. RESULT

Thus, the DBSCAN Clustering practical was implemented successfully using the program from the corresponding Jupyter Notebook.

Exercise -15

Principal Component Analysis

1. AIM

To apply Principal Component Analysis to reduce the preprocessed heart disease feature space to two components and measure explained variance.

2. ALGORITHM

1. Load the dataset and separate the target.

2. Identify numerical and categorical features.

3. Scale numerical features and one-hot encode categorical features.

4. Convert the processed data when required.

5. Fit PCA with two components.

6. Transform the data and calculate explained variance.

7. Plot the two principal components.

3. PROGRAM

```
#!/usr/bin/env python
# coding: utf-8

import pandas as pd
import matplotlib.pyplot as plt

from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.decomposition import PCA

df = pd.read_csv("heart_disease_dataset.csv")

print("Dataset Shape:", df.shape)

print("\nFirst 5 records:")
print(df.head())

X = df.drop("Heart Disease", axis=1)

numeric_features = X.select_dtypes(
    include=["int64", "float64"]
).columns

categorical_features = X.select_dtypes(
    include=["object"]
).columns

preprocessor = ColumnTransformer(
```

```
    transformers=[
        ("num", StandardScaler(), numeric_features),
        ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_features)
    ]
)

X_processed = preprocessor.fit_transform(X)

# Check whether X_processed is sparse or already a NumPy array
if hasattr(X_processed, "toarray"):
    X_processed = X_processed.toarray()

print("\nProcessed Data Shape:")
print(X_processed.shape)

pca = PCA(
    n_components=2
)

X_pca = pca.fit_transform(X_processed)

pca_df = pd.DataFrame(
    X_pca,
    columns=[
        "Principal Component 1",
        "Principal Component 2"
    ]
)

print("\nPCA Transformed Data:")
print(pca_df.head())

print("\nExplained Variance Ratio:")

print(pca.explained_variance_ratio_)

total_variance = pca.explained_variance_ratio_.sum()

print("\nTotal Variance Explained:")
print(total_variance)

print(
    "\nPercentage of Variance Explained:",
    total_variance * 100,
    "%"
)

plt.figure(figsize=(9, 6))

plt.scatter(
    X_pca[:, 0],
    X_pca[:, 1],
    s=40
)

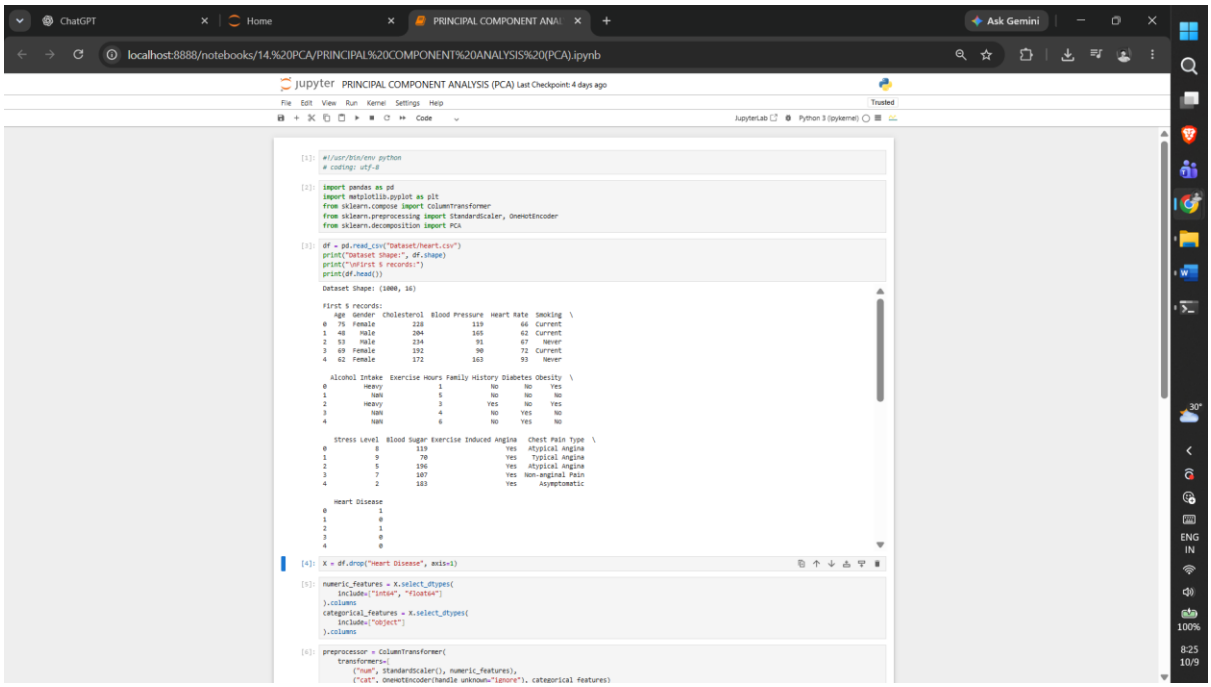
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
```

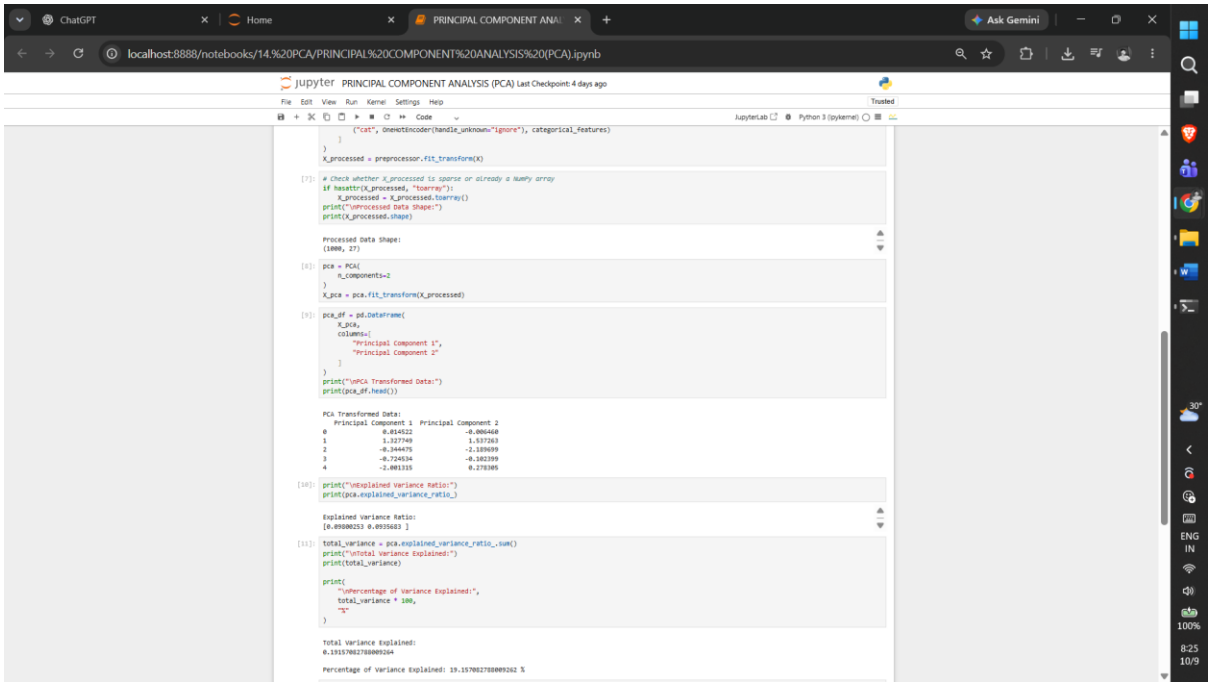
```
plt.title("PCA Visualization")

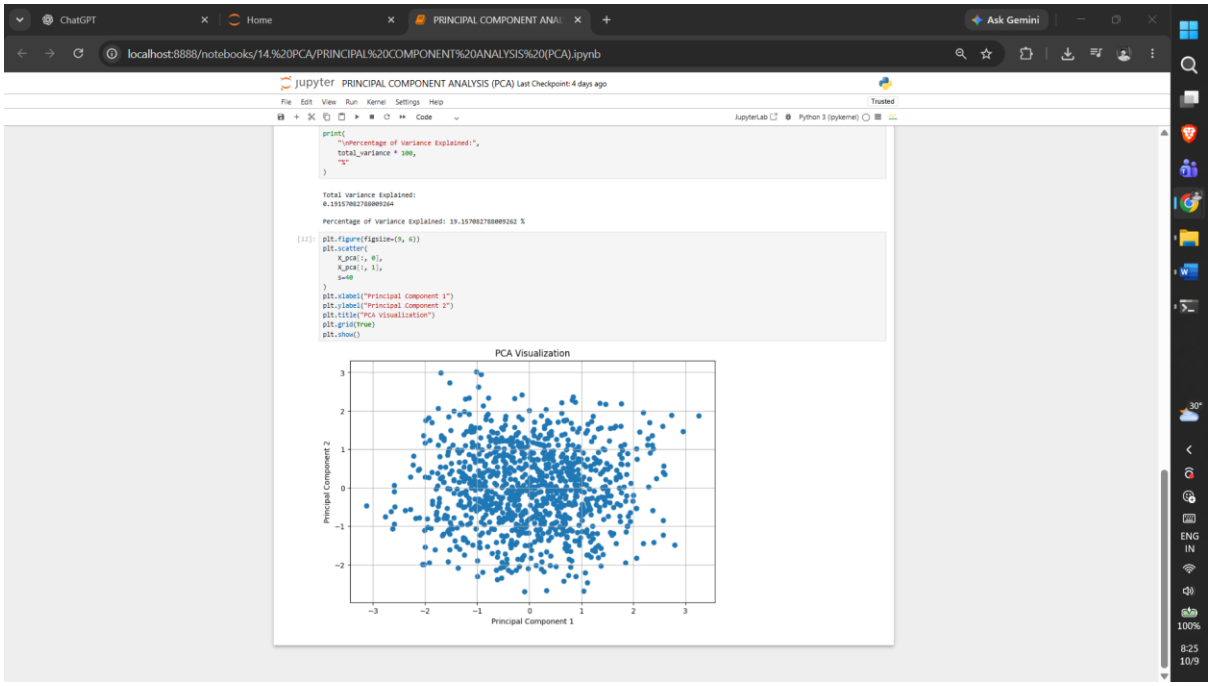
plt.grid(True)

plt.show()
```

4. OUTPUT







5. RESULT

Thus, the PCA practical was implemented successfully using the program from the corresponding Jupyter Notebook.